

Chapter 7

The Grand Unified Theory of Computation

It may be desirable to explain, that by the word *operation*, we mean *any process which alters the mutual relation of two or more things*, be this relation of what kind it may. This is the most general definition, and would include all subjects in the universe. In abstract mathematics, of course operations alter those particular relations which are involved in the considerations of number and space. . . . But the science of operations, as derived from mathematics more especially, is a science of itself, and has its own abstract truth and value; just as logic has its own peculiar truth and value, independently of the subjects to which we may apply its reasonings and processes.

Augusta Ada, Countess of Lovelace, 1842

We have spent the first part of this book asking what problems can be solved in polynomial time, exponential time, and so on. But what if our time is unlimited? Let's call a problem *decidable*, or *computable*, if there is an algorithm that solves it in some finite amount of time. We place no bounds whatsoever on how long the algorithm takes—we just know that it will halt eventually. In some sense, the class Decidable of such problems is the ultimate complexity class. Is there anything outside it? Is every problem decidable, and every function computable, if we give our computer enough time?

For this question to make sense, we have to wrestle with an issue that is partly mathematical and partly philosophical: what is computation anyway? In Chapter 2 we argued that the class P is robust—that the running time of an algorithm doesn't depend very much on our model of computation or our choice of programming language. Based on this claim, we have allowed ourselves to define our algorithms using an informal “pseudocode.” We have assumed that an algorithm's behavior is essentially the same regardless of the details of how we implement it, or what hardware we run it on.

But do the details really not matter? *A priori*, it might be that each programming language, and each computing technology, is capable of solving a different set of problems—and therefore that there are as many definitions of computability and decidability as there are models of computation. Is there a universal definition of computation, with many equivalent forms, that captures every reasonable model?

Many of the greatest moments in the history of science have been *unifications*: realizations that seemingly disparate phenomena are all aspects of one underlying structure. Newton showed that apples and

planets are governed by the same laws of motion and gravity, creating for the first time a single explanatory framework that joins the terrestrial to the celestial. Maxwell showed that electricity and magnetism can be explained by a single electromagnetic field, and that light consists of its oscillations. This quest for unity drives physics to this day, as we hunt for a Grand Unified Theory that combines gravity with quantum mechanics.

In 1936, computer science had its own grand unification. In the early 20th century, driven by the desire to create a firm underpinning for mathematics, logicians and mathematicians invented various notions of an “effective calculation” or “mechanical procedure”—what we would call a program or algorithm today. Each of their definitions took a very different attitude towards what it means to compute. One of them defines functions recursively in terms of simpler ones; another unfolds the definition of a function by manipulating strings of symbols; and, most famously, the Turing machine reads, writes, and changes the data in its memory by following a set of simple instructions.

Turing, Kleene, Church, Post, and others showed that each of these models can simulate any of the others. What one can do, they all can do. This universal notion of computation, and the fact that it transcends what programming language we use, or what technology we possess, is the topic of this chapter.

We begin by describing Babbage’s vision of a mechanical device which can be programmed to carry out any calculation, and Hilbert’s dream of a mechanical procedure that can prove or refute any mathematical claim. We discuss the fact that any sufficiently powerful model of computation can talk about itself—programs can run other programs, or even run themselves.

The self-referential nature of computation has some surprising consequences, including the fact that some problems are *undecidable*—there is no program that solves them in any finite amount of time. Chief among these is the Halting Problem, which asks whether a given program will halt or run forever. We will see how these undecidable problems imply Gödel’s Incompleteness Theorem that no formal system is capable of proving all mathematical truths. Thus, while Babbage’s vision has now been realized, Hilbert’s vision can never be.

We then describe the three great models of computation proposed in the early 20th century—partial recursive functions, the λ -calculus, and Turing machines—and show that they are all equivalent to each other. We argue for the *Church–Turing Thesis*, which states that these models are strong enough to carry out any conceivable computation. Finally, we cast our net wide, and find universal computation and undecidability nearly everywhere we look: in products of fractions, tilings of the plane, and the motions of a chaotic system.

7.1 Babbage’s Vision and Hilbert’s Dream

There are two fundamental concepts at the heart of computer science. One is the *algorithm* or *program*: a sequence of instructions, written in some general but precise language, that carries out some calculation or solves some problem. The other is the *universal computer*: a programmable device that can carry out any algorithm we give to it.

These concepts emerged in the 19th and early 20th centuries, in forms that are surprisingly recognizable to us today. Regrettably, they appeared in two separate threads that, as far as we know, never interacted—the visions of an inventor, and the dream of an unshakeable foundation for mathematics. Let’s go back in time, and watch as our forebears set computer science in motion.

7.1.1 Computing With Steam

As soon as an Analytical Engine exists, it will necessarily guide the future course of the science.

Charles Babbage, *Passages from the Life of a Philosopher*

What shall we do to get rid of Mr. Babbage and his calculating machine? Surely if completed it would be worthless as far as science is concerned?

British Prime Minister Sir Robert Peel, 1842

For centuries, printed tables of trigonometric functions, logarithms, and planetary orbits were used as computational aids by scientists, merchants, and navigators. Being calculated by human computers, these tables were subject to human fallibility. But errors, especially those in navigational tables, can have severe consequences.

One day in 1821, while checking astronomical tables and discovering numerous errors, Charles Babbage declared "I wish to God that these calculations had been executed by steam!" Babbage, being not only a mathematician and philosopher, but also an inventor and mechanical engineer, decided to develop a machine that could calculate and print mathematical tables, automatically and free of errors.

Babbage knew that many functions can be closely approximated by polynomials of low degree. For example, the 4th-order Taylor series for the cosine,

$$\cos x \approx 1 - \frac{x^2}{2} + \frac{x^4}{24},$$

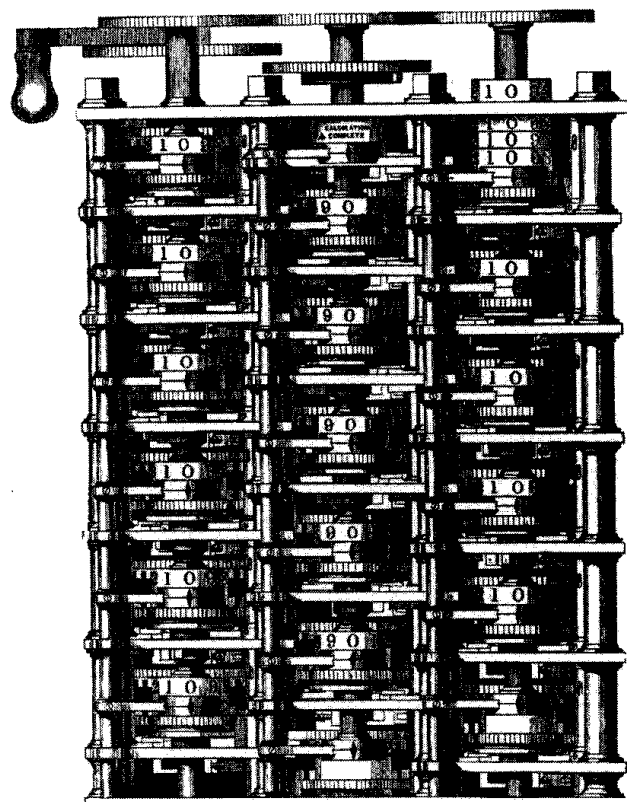
has an error of at most 0.02 for $0 \leq x \leq \pi/2$. In order to compute functions such as $\cos x$ and $\ln x$, Babbage designed his first machine, the Difference Engine, to tabulate polynomials. He proposed to do this in an ingenious way, called the *method of differences*.

If $f(x)$ is a polynomial of degree n then its difference $\Delta f(x) = f(x+1) - f(x)$ is a polynomial of degree $n-1$, and so on, until its n th difference $\Delta^n f$ is a constant. Each of these differences was stored in a column of wheels as shown in Figure 7.1, where each wheel stored a single digit. The user set the initial values $f(0), \Delta f(0), \dots, \Delta^n f(0)$ by turning these wheels, and the machine then computed $f(x), \Delta f(x), \dots$ for successive values of x by adding these differences to each other. Thus the Difference Engine could be "programmed" to calculate any polynomial whose degree is less than the number of columns, and approximate a wide variety of functions.

The Difference Engine was already a very innovative concept. But before it was built, Babbage turned his attention to a far more ambitious machine—a truly universal computer, which could perform any conceivable calculation. In his autobiography, he wrote:

The whole of arithmetic now appeared within the grasp of mechanism. A vague glimpse even of an Analytical Engine at length opened out, and I pursued with enthusiasm the shadowy vision.

Babbage spent most of the rest of his life turning this "shadowy vision" into a detailed blueprint.



B. H. Babbage, del.

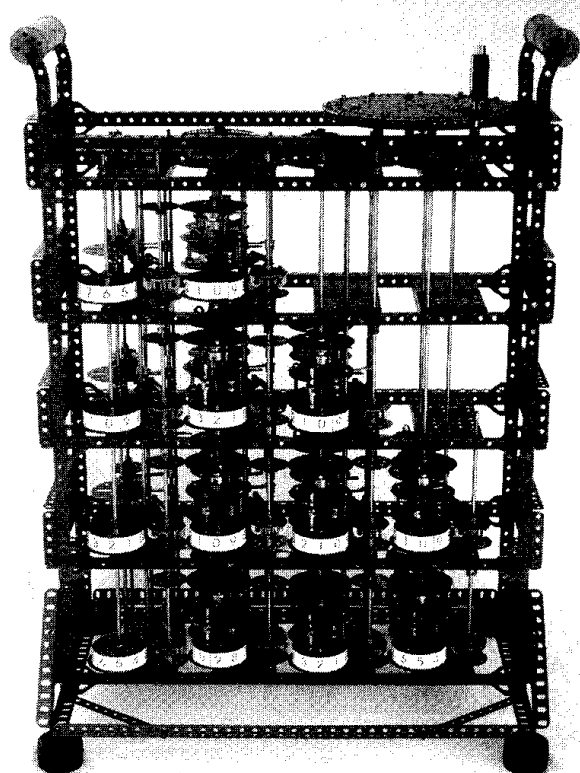


FIGURE 7.1: Charles Babbage's difference engine: 19th century woodcut (left) and 20th century construction from Meccano™ parts (right).

The Analytical Engine was designed with a “store” that could keep a thousand 50-digit numbers, each digit being represented by the position of a cogwheel. The arithmetical heart of the machine—the “mill”—could perform addition, subtraction, multiplication, and division, using clever mechanical assemblies. To control the mill, Babbage proposed a technique that was then being used to direct mechanical looms, and that survived in the programming of electronic computers until the 1970s. In his own words:

It is known as a fact that the Jacquard loom is capable of weaving any design which the imagination of man may conceive. It is also the constant practice for skilled artists to be employed by manufacturers in designing patterns. These patterns are then sent to a peculiar artist, who, by means of a certain machine, punches holes in a set of pasteboard cards in such a manner that when those cards are placed in a Jacquard loom, it will then weave upon its produce the exact pattern designed by the artist.

Babbage's design included “variable” cards that copied numbers from the store into the mill and vice versa, and “operation” cards that instructed the mill to carry out a particular operation on its contents.

Babbage realized that it was not enough to carry out a fixed sequence of operations. He also included “combinatorial” cards that governed the motion of the variable and operation cards, allowing for conditional execution, skipping forwards or backwards in the sequence, or for a cycle of cards to be repeated

a certain number of times. In other words, he invented **if-then** and **goto** statements, and **for** loops. He even proposed that the machine could punch its own “number” cards, recording data from previous calculations, giving it an effectively unlimited memory.

Babbage's goals for the Analytical Engine consisted solely of numerical calculations. It was Augusta Ada, Countess of Lovelace (and daughter of Lord Byron), who realized that the Analytical Engine could do much more than number crunching. She wrote:

The bounds of *arithmetic* were however outstepped the moment the idea of applying the cards had occurred; and the Analytical Engine does not occupy common ground with mere “calculating machines.” In enabling mechanism to combine together *general* symbols in successions of unlimited variety and extent, a uniting link is established between the operations of matter and the abstract mental processes of the most abstract branch of mathematical science...

Just as we do today, Ada thought of computation as a manipulation of *symbols*, not just digits. And, she realized that these symbols can encode many different kinds of information:

Again, it might act upon other things besides number, were objects found whose mutual fundamental relations could be expressed by those of the abstract science of operations... Supposing, for instance, that the fundamental relations of pitched sounds in the science of harmony and of musical composition were susceptible of such expression and adaptations, the engine might compose elaborate and scientific pieces of music of any degree of complexity or extent.

Ada also wrote a program for the Analytical Engine, with two nested **for** loops, to compute a sequence of fractions called the Bernoulli numbers. For this reason, she is often referred to as the first computer programmer.

Sadly, Babbage and Lovelace were almost one hundred years ahead of their time. The first truly programmable computers were built in the 1940s, with electric relays instead of cogwheels. But their basic architecture—a central processing unit, governed by a program, which reads and writes data stored in memory—would have been perfectly recognizable to Babbage. Moreover, the idea of a universal computer played another important role in the early 20th century: as a thought experiment, to understand the ultimate mathematical power of algorithms.

7.2

7.1.2 The Foundations of Mathematics

We hear within us the perpetual call: There is the problem. Seek its solution. You can find it by pure reason, for in mathematics there is no *ignorabimus*.

David Hilbert

Mathematicians from Euclid to Gauss have been thinking about algorithms for millennia. But the idea of algorithms as well-defined mathematical objects, worthy of investigation in and of themselves, didn't emerge until the dawn of the 20th century.

In 1900, David Hilbert delivered an address to the International Congress of Mathematicians. He presented a list of 23 problems which he saw as being the greatest challenges for 20th-century mathematics. These problems became known as Hilbert's problems, and inspired ground-breaking work across the mathematical landscape. His 10th problem was as follows:

Specify a procedure which, in a finite number of operations, enables one to determine whether or not a given Diophantine equation [a polynomial equation with integer coefficients] with an arbitrary number of variables has an integer solution.

He felt sure that there was such an algorithm—the only problem was to find it.

Hilbert showed even more optimism about the power of algorithms in 1928, when he challenged his fellow mathematicians with the *Entscheidungsproblem* (in English, the decision problem):

The Entscheidungsproblem is solved if one knows a procedure that allows one to decide the validity of a given logical expression by a finite number of operations.

In other words, he asked for an algorithm that will take any statement written in a formal language—for instance,

$$\neg \exists x, y, z, n \in \mathbb{N} - \{0\} : (n \geq 3) \wedge (x^n + y^n = z^n),$$

which expresses Fermat's last theorem—and determine, in a finite amount of time, whether it is true or false. Such an algorithm would constitute nothing less than the mechanization of mathematics.

7.3

Let's compare this with our discussion in Section 6.1.3. If $P = NP$, we can tell in $\text{poly}(\ell)$ time whether a given statement has a proof of length ℓ or less. Hilbert wasn't concerned with the distinction between polynomial and exponential time—he wanted a procedure that halts after a finite amount of time, regardless of how long the shortest proof or disproof of the statement is.

Hilbert didn't spell out what he meant by a “procedure.” He didn't have computing devices in mind—he was unaware of Babbage's work, and electronic computers were decades away. He did, however, want this procedure to be mechanical, in the sense that it could be carried out by a human mathematician according to a clear sequence of operations. This gave rise to the question of what functions are “computable” or “effectively calculable.”

At the time, many mathematicians were struggling to build an axiomatic foundation for mathematics. Their goal was to reduce all of mathematics to set theory and logic, creating a formal system powerful enough to prove all the mathematical facts we know and love. But at the turn of the century, several paradoxes shook these foundations, showing that a naive approach to set theory could lead to contradictions.

The best known of these paradoxes is *Russell's paradox*, which deals with sets. Sets can be elements of other sets—for instance, consider the set of all intervals on the real line, each of which is a set of real numbers. So, it seems reasonable to ask which sets are elements of themselves. Once we can do that, we can define the set of sets that are *not* elements of themselves,

$$R = \{S : S \notin S\}.$$

But now we're in trouble. By definition, R is an element of itself if and only if it isn't:

$$R \in R \Leftrightarrow R \notin R.$$

It would be intolerable to have this kind of contradiction at the heart of mathematics. Our only escape is banish R from the mathematical universe—to conclude that, for some reason, R is not a well-defined mathematical object.

The problem with R is the “loopiness” of its definition—it is a loop with a twist in it, like a Möbius strip. We can avoid this kind of loop by insisting on a strictly stratified mathematical universe, where each set can only contain sets from lower levels, until we reach “atoms” like the empty set. In such a universe the idea of a set containing itself is, in Russell's words, “noise without meaning.” Thus we eliminate the paradox by dismissing the definition of R as nonsensical.

This stratified picture of set theory paved the way for an analogous approach to computation. We start with a handful of atomic functions, and build more complicated functions out of them. This lets us build up a universe of computable functions, each of which is inductively defined in terms of simpler functions, defined at an earlier stage of the process.

However, there are many particular ways that we could do this—many notions of what constitutes an atomic operation, and many ways to combine them to make higher-level functions. In the first decades of the 20th century, several definitions of computability along these lines were proposed, but it was not clear whether any of them captured the universe of all conceivable computations.

In 1936, the *annus mirabilis* of computation, a series of results showed that all these definitions are equivalent. This gave rise to the *Church–Turing Thesis*, that these models capture anything that could be reasonably called a computation—in Hilbert's words, any procedure with a finite number of operations. We sketch these models, and their equivalence, in Sections 7.3, 7.4, and 7.5.

At the same time, Turing proposed his Halting Problem, which asks whether a given program will ever halt. Using an argument similar in form to Russell's paradox, we will show in Section 7.2 that if there were a program that solved this problem, we could create a paradox by asking it about itself. Therefore, no such program exists, and the Halting Problem problem is undecidable.

Since the Halting Problem is a well-defined mathematical question, this shows that the Entscheidungsproblem cannot be solved—there is no algorithm that can take an arbitrary mathematical statement as input, and determine whether it is true or false. In 1970, it was shown that telling whether a given Diophantine equation has integer roots is just as hard as the Halting Problem. So, Hilbert's 10th problem is undecidable as well.

Hilbert may have found the phenomenon of undecidability disappointing, but we find it liberating. Universal computation is everywhere, even in seemingly simple mathematical questions. It lends those questions an infinite richness, and proves that there is no mechanical procedure that can solve them once and for all. Mathematics will never end.

Now that the stage is set, let's explore the self-referential nature of computation, and how it leads to undecidable problems and unprovable truths.



7.4



7.5

7.2 Universality and Undecidability

There's no getting round a proof... even if it proves that something is unprovable!

Apostolos Doxiadis and
Christos H. Papadimitriou, *Logicomix*

The most basic fact about modern computers is their universality. Like the Analytical Engine, they can carry out any program we give to them. In particular, *there are programs that run other programs*. A computer's operating system is a program that runs and manages many programs at once. Even web browsers run programs on virtual machines that they simulate inside their own workspace.

In any programming language worth its salt, one can write an *interpreter* or *universal program*—a program that takes the source code of another program as input, and runs it step-by-step, keeping track of its variables and which instruction to perform next. Symbolically, we can define this universal program like this:

$$U(\Pi, x) = \Pi(x). \quad (7.1)$$

That is, U takes a program Π and an input x , and returns the same results that Π would give if we ran Π on x . Note that we are using the symbol Π in two different ways. On the left, we mean Π 's source code, which can be fed to U like any other finite string. On the right, we mean Π in its active form, taking an input x and returning an output.

7.6  The fact that universal programs exist has enormous consequences. We can set up self-referential situations—what Douglas Hofstadter calls “strange loops”—in which programs run themselves, or modified versions of themselves. These loops can lead to paradoxes in which the only way out is to accept that some problems cannot be solved—that they are *undecidable*, and beyond the ability of any algorithm.

Let's start by showing that, in any programming language powerful enough to interpret itself, there are programs that will run forever and never return any output. Then, we will show that there is no program that can tell whether or not a given program, with a given input, will ever halt.

7.2.1 Diagonalization and Halting

Let $U(\Pi, x)$ be a universal program as defined in (7.1). Consider the special case where $x = \Pi$. Then U simulates what Π would do given its own source code:

$$U(\Pi, \Pi) = \Pi(\Pi).$$

Now suppose, for simplicity, that the programs in question returns a Boolean value, true or false. Then we can define a new program V which runs Π on itself, and negates the result:

$$V(\Pi) = \overline{\Pi(\Pi)}.$$

There is nothing mythical about V —it is a concrete program, that really can be written in your favorite programming language. But if we feed V its own source code, an apparent contradiction arises, since

$$V(V) = \overline{V(V)}.$$

The only way to resolve this paradox is if $V(V)$ is undefined. In other words, when given its own source code as input, V runs forever, and never returns any output. This shows that any programming language powerful enough to express a universal program possesses programs that never halt, at least when given certain inputs. In brief,

Universality implies non-halting programs.

Thus any reasonable definition of computable functions includes *partial* functions, which are undefined for some values of their input, in addition to *total* ones, which are always well-defined.

This kind of argument, where we feed things to themselves and add a twist, is known as *diagonalization*. It originated with Georg Cantor, who used it to show that some infinities are bigger than others. Specifically, let $\mathbb{N} = \{0, 1, 2, 3, \dots\}$ denote the set of natural numbers, and let $\wp(\mathbb{N})$ denote its *power set*—that is, the set $\{S : S \subseteq \mathbb{N}\}$ of all subsets of $\mathbb{N}$. If these sets were the same size, it would be possible to put them in one-to-one correspondence. In other words, there would be a way to list all subsets as S_0, S_1, S_2 , and so on, such that for every $S \subseteq \mathbb{N}$ we have $S = S_i$ for some $i \in \mathbb{N}$.

Cantor proved that no such correspondence exists. If one did, we could imagine listing all the subsets in a table, where the i th row contains a series of bits marking which integers are in S_i . For instance, let's say that $S_0 = \emptyset$ is the empty set, $S_1 = \mathbb{N}$, S_2 is the even numbers, S_3 is the odd ones, S_4 is the primes, and so on. Then the beginning of this table would look like this:

	0	1	2	3	4	...
S_0	0	0	0	0	0	
S_1	1	1	1	1	1	
S_2	1	0	1	0	1	
S_3	0	1	0	1	0	
S_4	0	0	1	1	0	
$\vdots$						$\ddots$

Cantor's idea is to take the diagonal of this table, and flip all its bits. This gives us a new set T :

	0	1	2	3	4	...
S_0	0	0	0	0	0	
S_1	1	1	1	1	1	
S_2	1	0	1	0	1	
S_3	0	1	0	1	0	
S_4	0	0	1	1	0	
$\vdots$						
T	1	0	0	0	1	...

Since $i \in T$ if and only if $i \notin S_i$, we have $T \neq S_i$ for all i . Therefore, T was not included in our table of subsets, and our table was not complete. Any correspondence between $\mathbb{N}$ and $\wp(\mathbb{N})$ leaves at least one subset out. So, while both $\mathbb{N}$ and $\wp(\mathbb{N})$ are infinite, $\wp(\mathbb{N})$ is bigger.

We say that $\mathbb{N}$ is *countably* infinite, and we denote its cardinality $\aleph_0$. In contrast, $\wp(\mathbb{N})$ is *uncountably* infinite—its cardinality is a larger infinity. For any set S , the cardinality of its power set is $|\wp(S)| = 2^{|S|}$,

since for each $x \in S$ we can choose whether or not to include x in a subset T . In these terms, the diagonal argument shows that $2^{\aleph_0} > \aleph_0$, and more generally that $2^x > x$ even when x is infinite. The cardinality of $\wp(\wp(\mathbb{N}))$ is an even larger infinity, $2^{2^{\aleph_0}}$, and so on.

7.7

We can phrase our argument about V in diagonal form as follows. Let Π_i denote the i th program in some convenient order. If programs always halt, Π_i gives a well-defined output when given Π_j as input, for all i and j . If we imagine a table of all these outputs, then V consists of taking the diagonal and negating it:

	Π_1	Π_2	Π_3	...
Π_1	$\Pi_1(\Pi_1)$	$\Pi_1(\Pi_2)$	$\Pi_1(\Pi_3)$	
Π_2	$\Pi_2(\Pi_1)$	$\Pi_2(\Pi_2)$	$\Pi_2(\Pi_3)$	
Π_3	$\Pi_3(\Pi_1)$	$\Pi_3(\Pi_2)$	$\Pi_3(\Pi_3)$	
$\vdots$				
V	$\overline{\Pi_1(\Pi_1)}$	$\overline{\Pi_2(\Pi_2)}$	$\overline{\Pi_3(\Pi_3)}$	

Once again, we have a contradiction. The difference is that now V is somewhere in the table—if a universal program exists then so does V , so $V = \Pi_j$ for some j . So, this time around, we escape the contradiction by admitting that $\Pi_i(\Pi_j)$ is not well-defined for all i and j , and that some programs never halt on some inputs. In particular, V cannot differ with itself, so $V(V)$ must not be well-defined. Conversely, if a programming language guarantees, for some reason, that its programs always halt, then it is not powerful enough to interpret itself.

Exercise 7.1 In one of the first diagonal statements, Epimenides the Cretan stated that all Cretans are liars. Show that this is not in fact a contradiction, by giving a consistent interpretation of Epimenides' statement and its truth or falsehood.

Exercise 7.2 Phrase Russell's paradox as a diagonal argument. Hint: imagine a vast table which lists whether or not $S \in T$ for every pair of sets S, T .

7.2.2 The Halting Problem

Since some programs halt and others don't, it would be nice to be able to tell which is which. Consider the following problem:

HALTING

Input: A program Π and an input x

Question: Will Π halt when given x as input? Equivalently, is $\Pi(x)$ well-defined?

We will prove that this problem is *undecidable*—that no program can solve it in any finite amount of time.

In 1936, the fact that undecidable problems exist came as quite a surprise. We have spent much of this book discussing classes of problems that can be solved in polynomial time, exponential time, and so on. Just as we defined complexity classes like P and EXP , we can define a class Decidable as the class of all problems that can be solved in *any finite amount of time*:

```

Fermat
begin
  t := 3;
  repeat
    for n = 3 to t do
      for x = 1 to t do
        for y = 1 to t do
          for z = 1 to t do
            if  $x^n + y^n = z^n$  then return (x, y, z, n);
          end
        end
      end
    end
    t := t + 1;
  until forever;
end

```

FIGURE 7.2: This program does not halt. Andrew Wiles has discovered a truly marvelous proof of this, which this caption is too narrow to contain.

Decidable is the class of problems A for which there is a program Π that halts after a finite amount of time, and which, given any input x , answers whether or not $A(x)$ is true.

Hilbert believed that Decidable includes *all* well-defined mathematical problems. But the Halting Problem shows that it does not—that some problems are beyond the reach of any algorithm.

At first, it seems that we can solve HALTING by running the universal program U , simulating what Π would do given x , and seeing what happens. But if $\Pi(x)$ doesn't halt, neither will U 's simulation. To solve HALTING, we have to be able to see into the future, and predict, in bounded time, whether or not $\Pi(x)$ will ever halt. We will show that no program can do this—since if it could, it could create a contradiction by trying to predict itself.

Before we prove that no program can solve HALTING, let's think about what other problems we could solve if such a program existed—a program with the following behavior:

$$\text{Halts}(\Pi, x) = \begin{cases} \text{true} & \text{if } \Pi \text{ halts on input } x \\ \text{false} & \text{otherwise} \end{cases} \quad (7.2)$$

Such a program would be very handy for debugging software. It could tell us whether a program will hang, and never get back to the user. However, it would have much more profound uses as well. Consider the program Fermat in Figure 7.2. It looks for a counterexample to Fermat's Last Theorem—that is, positive integers x, y, z, n with $n \geq 3$, such that

$$x^n + y^n = z^n, \quad (7.3)$$

It does this by exhaustively checking all quadruples $x, y, z, n \leq t$, with t ranging up to infinity.

Of course, we don't actually want to run `Fermat`—we only want to feed it to `Halts`. It took the mathematical community 358 years to prove that no such quadruples exist, and that `Fermat` will run forever. Obviously having `Halts` would save us a lot of time.

Exercise 7.3 According to a conjecture formulated by Christian Goldbach in 1771, every even integer greater than 2 can be written as the sum of two primes, such as $10 = 3 + 7$ or $12 = 5 + 7$. Write a program that could be fed to `Halts`, if it existed, to prove or disprove Goldbach's Conjecture.

There is a truly marvelous proof that the program `Halts` cannot exist, and the proof fits into a few lines. Once again, we use diagonalization. Suppose that `Halts` existed. Then we could call it as a sub-routine, set $x = \Pi$, and build the following program:

```
Catch22( $\Pi$ )
begin
  if Halts( $\Pi, \Pi$ ) = true then loop forever;
  else return true;
end
```

We have designed `Catch22( $\Pi$ )` so that it halts if and only if $\Pi(\Pi)$ runs forever. But now diagonalization strikes again. If we run `Catch22` on itself, `Catch22(Catch22)` halts if and only if it doesn't. The only escape from the contradiction is to conclude that there is no such program as `Halts`. So, HALTING is undecidable.

7.8

Of course, this does not show that HALTING can never be solved. If we restrict our attention to a particular program Π and define a version of HALTING whose only input is x , then for some choices of Π this problem is clearly decidable—for instance, if Π immediately halts without doing anything with its input. But for some other choices of Π , such as the universal program U , this version of the Halting Problem is just as hard as the general one, and so is undecidable.

Now that we have one undecidable problem, we can prove that other problems are undecidable by reducing HALTING to them. For instance, consider the following problem:

FORTY-TWO

Input: A program Π

Question: Is there an x such that $\Pi(x)$ halts and returns 42?

Given a program Π and input x , we can convert them to a program Π' which ignores its input, runs $\Pi(x)$ instead, and return 42 if it halts:

```
 $\Pi'(x')$ 
begin
  run  $\Pi(x)$ ;
  return 42;
end
```

If $\Pi(x)$ halts, then $\Pi'(x')$ returns 42 for all x' . If $\Pi(x)$ doesn't halt, then neither does Π' , no matter what input x' we give it. Thus if FORTY-TWO were decidable, HALTING would be too. But we know that HALTING is undecidable, so FORTY-TWO must be undecidable as well.

This recipe for Π' is a reduction analogous to the ones we used for NP-completeness—it maps instances of HALTING to instances of FORTY-TWO with the same answer. It shows that FORTY-TWO is at least as hard as HALTING, or

$$\text{HALTING} \leq \text{FORTY-TWO}.$$

Just as we used polynomial-time reductions for exploring P and NP, here we use *computable* reductions. That is, a reduction can be any function from instances of A to instances of B that we can compute in finite time. In that case, $A \leq B$ implies that if B is decidable then A is decidable, and conversely, if A is undecidable then B is undecidable. As Problems 7.3 and 7.4 show, we can prove that virtually any nontrivial question about a program's long-term behavior is undecidable by reducing HALTING to it—typically, with a simple reduction that modifies the program's source code.

It's worth noting that if the only thing we had to worry about was falling into an endless loop, the Halting Problem would be decidable:

Exercise 7.4 Suppose that a program Π has the following property: for any input x , either $\Pi(x)$ halts, or it falls into a periodic loop, where all its variables return again and again to the same values. Show that in that case Π 's Halting Problem is decidable.

Thus the undecidability of the Halting Problem stems from the fact that programs can run forever, but in complicated ways—such as searching for larger and larger counterexamples to Fermat's Last Theorem.

The astute reader may object that we haven't really proved that the Halting Problem is undecidable. All we have proved is that no programming language can answer *its own Halting Problem*, unless it is so weak that it cannot interpret its own programs. To show that the Halting Problem—and therefore, the Entscheidungsproblem, of which the Halting Problem is a special case—is truly undecidable, we need to show that there are particular programming languages, or models of computation, that capture the power of all possible algorithms. This is, to some extent, a philosophical claim, rather than a mathematical one, and we will return to it in Section 7.5.

It's also important to realize that we are making some very important assumptions about how programs can be built out of other programs. For instance, we assume that if a program for $\text{HALTS}(\Pi, x)$ exists, then so does a program that calls HALTS as a subroutine with the special case $x = \Pi$, and then uses an **if-then** statement to do one thing or another depending on its results. Most fundamentally, we assume that each program has a source code that can be read by other programs, or even by itself. Living, as we do, long after the events of 1936, we have the luxury of thinking in terms of high-level programming languages which let us take these things for granted. But at the dawn of computer science, these concepts had to be built from scratch. As we will see later in this chapter, carrying out the diagonal argument took much more work back then than it does now.

But first, let's use the Halting Problem as a springboard to explore the upper reaches of the complexity hierarchy. In the land of polynomial time, we defined classes like P, NP, and the polynomial hierarchy to understand the relationship between a problem's logical structure and its computational complexity. How does this landscape look when we are allowed any finite amount of time to do our computation? And how is the undecidability of the Halting Program related to the other great upheaval of the 1930s—Gödel's Theorem that no formal system can provide a complete foundation for mathematics?

7.2.3 Recursive Enumerability

While the Halting Problem is undecidable, it has a kind of one-sided decidability. If the answer is “yes,” then we can learn that fact in a finite amount of time, by simulating Π until it halts. To put this differently, we can write

$$\text{Halts}(\Pi, x) = \exists t : \text{HaltsInTime}(\Pi, x, t),$$

where $\text{HaltsInTime}(\Pi, x, t)$ is the property that Π , given x as input, halts on its t th step. Since we can tell if this is true by running Π for t steps, HaltsInTime is decidable. Thus Halts is a combination of a decidable problem with a single “there exists.”

In some ways, this is analogous to the relationship between P and NP. Recall that a property A is in NP if it can be written $A(x) = \exists w : B(x, w)$ where B is in P. In other words, x is a yes-instance of A if some witness w exists, and the property $B(x, w)$ that w is a valid witness for x can be checked in polynomial time. Here t is a witness that Π halts, and we can check the validity of this witness in finite time.

There is an interesting alternate way to describe this kind of problem. Typically, we give a program an input x , and ask it to tell whether x is in some set S of yes-instances. If such a program exists, we say that S is decidable. Equivalently, we say that a problem is decidable if its set of yes-instances is decidable.

But we can also hook a program Π up to a printer—with an infinite amount of paper, of course—and ask it to print out a list of all the elements of S . We demand that every $x \in S$ appears at some point in this list, and that no $x \notin S$ ever will. If S is infinite, Π will run forever, but any particular $x \in S$ appears on the printer within a finite amount of time. Note that we allow Π to print the $x \in S$ in any order, and even repeat them, as long as each one appears eventually. If there is such a program, we say that Π *enumerates* S , and that S is *recursively enumerable*.

Now let RE denote the class of problems for which the set of yes-instances is recursively enumerable. We claim that these are exactly the problems of the form $A(x) = \exists w : B(x, w)$ where B is decidable. In one direction, suppose that Π enumerates the yes-instances of a problem A , and let $\text{Prints}(\Pi, x, t)$ denote the property that x is the t th string that Π produces. Then Prints is decidable, and we can express the claim that x is a yes-instance as $\exists t : \text{Prints}(\Pi, x, t)$.

In the other direction, suppose that $A(x) = \exists w : B(x, w)$. For simplicity, we treat x and w as integers. Our first stab at a program that prints out all yes-instances x would look like this:

```

begin
   $x := 0$  ;
  repeat
     $w := 0$  ;
    while  $\neg B(x, w)$  do  $w := w + 1$  ;
    print  $x$  ;
     $x := x + 1$  ;
  until forever;
end
```

The problem with this program, of course, is that if x is a no-instance, the inner **while** loop gets stuck looking for a nonexistent witness w . To avoid this, we need to use an approach similar to Fermat in Figure 7.2. Here is an improved program:

```

begin
   $t := 0$  ;
  repeat
    for  $x = 0$  to  $t$  do
      for  $w = 0$  to  $t$  do
        if  $B(x, w)$  then print  $x$  ;
      end
    end
     $t := t + 1$  ;
  until forever;
end

```

This technique, where an inner loop goes through all pairs (x, w) with $x, w \leq t$ and an outer loop increases t , is called *dovetailing*. Note that this program might print a small yes-instance much later than a large one, if its witness is much larger. But it prints each yes-instance in finite time, so A is in RE.

Exercise 7.5 Show that the Halting Problem is in RE. In other words, describe a program that prints out the set of yes-instances of the Halting Problem, i.e., the set of all pairs (Π, x) such that Π halts when given x as input.

Exercise 7.6 Show that the Halting Problem is RE-complete. That is, show that any problem in RE can be expressed as an instance of the Halting Problem.

Just as we defined coNP as the set of problems for which no-instances have witnesses, we can define coRE as the set of problems for which the set of no-instances is recursively enumerable. Equivalently, these are the problems that can be expressed with a single “for all” quantifier, such as the statement

$$\text{Hangs}(\Pi, x) = \forall t : \overline{\text{HaltsInTime}(\Pi, x, t)},$$

that $\Pi(x)$ will run forever.

Unlike in the polynomial-time world, where the P vs. NP question remains unresolved, we know that RE, coRE, and Decidable are all distinct. Since the Halting Problem is undecidable, Exercise 7.5 tells us that Decidable $\subset$ RE. Then the following exercise shows that RE and coRE are different:

Exercise 7.7 Show that

$$\text{Decidable} = \text{RE} \cap \text{coRE}. \quad (7.4)$$

In other words, show that if both S and $\bar{S}$ are recursively enumerable then S is decidable.

We can conclude from this that

$$\text{RE} \neq \text{coRE},$$

since otherwise (7.4) would imply that Decidable = RE. In contrast, the question of whether $\text{NP} \neq \text{coNP}$ is still open.

7.2.4 A Tower of Gods

You jest with me. So I jest with you. Probably Koshchei jests with all of us. And he, no doubt—even Koshchei who made things as they are—is in turn the butt of some larger jest... and there is merriment overhead, but it is very far away.

James Branch Cabell, *Jurgen*

The classes RE and coRE are just the beginning. Analogous to the polynomial hierarchy, we can define an infinite tower of complexity classes, Σ_k and Π_k , consisting of problems with k quantifiers alternating between $\exists$ and $\forall$. In particular,

$$\text{RE} = \Sigma_1, \quad \text{coRE} = \Pi_1, \quad \text{and} \quad \Sigma_0 = \Pi_0 = \Sigma_1 \cap \Pi_1 = \text{Decidable}.$$

For an example at the second level of this hierarchy, the property that a program Π halts on all inputs is in Π_2 , since it has a $\exists$ nested inside a $\forall$:

$$\forall x : \exists t : \text{HaltsInTime}(\Pi, x, t),$$

The union of all these classes, $\bigcup_{k=0}^{\infty} \Sigma_k = \bigcup_{k=0}^{\infty} \Pi_k$, is known as the *arithmetical hierarchy*. Unlike the polynomial hierarchy, we know that the levels of the arithmetical hierarchy are distinct—that $\Sigma_k \subset \Sigma_{k+1}$ and $\Pi_k \subset \Pi_{k+1}$ for all k . To see why, imagine a “hypercomputer” with access to an oracle that can answer the Halting Problem in finite time. As Problem 7.12 shows, such a computer can solve any problem in $\Sigma_2 \cap \Pi_2$. However, by the same diagonalization argument as before, this hypercomputer cannot solve its own Halting Problem—and this hyper-Halting Problem is complete for Σ_2 .

Thus each time we add another layer of quantification, we get a more powerful class, containing problems at higher and higher orders of undecidability. Climbing this ladder leads into a cosmic realm, populated by gods with increasingly infinite computational power. Let’s return to the world of finite computations, and connect the two great impossibility results of the 1930s: the undecidability of the Halting Problem, and the incompleteness of formal systems.

7.2.5 From Undecidable Problems to Unprovable Truths

God exists since mathematics is consistent, and the Devil exists since we cannot prove it.

André Weil

It cannot be a complete coincidence that several outstanding logicians of the twentieth century found shelter in asylums at some time in their lives.

Gian-Carlo Rota, *Indiscrete Thoughts*

A *formal system* has a finite set of axioms, including rules of inference such as *modus ponens*, “ A and $(A \Rightarrow B)$ implies B .” A *theorem* is a statement that can be proved, with some finite chain of reasoning,

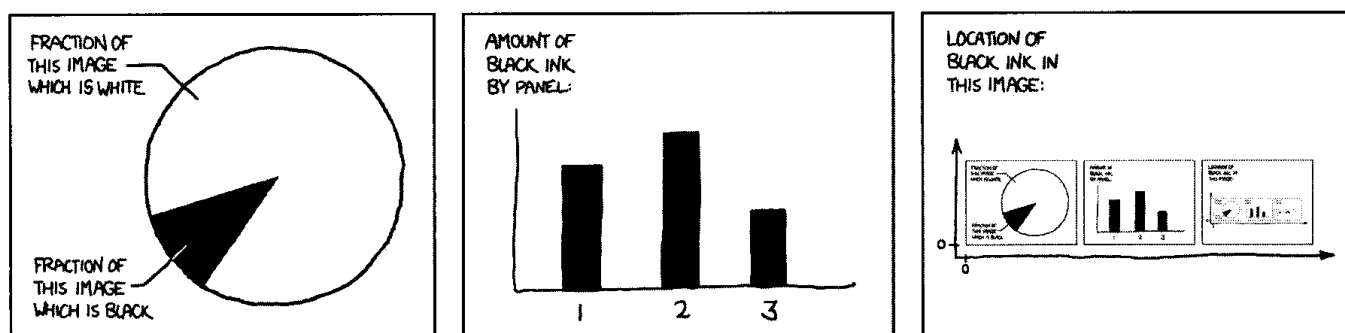


FIGURE 7.3: Above, some self-referential cartoons. Below, this caption.

from the axioms. We say that a formal system is *consistent* if there is no statement T such that both T and $\bar{T}$ are theorems, and *complete* if, for all T , at least one of T or $\bar{T}$ is a theorem.

We can define a statement as true or false by interpreting the symbols of the formal system in some standard way—treating $\exists$ as “there exists,” $\wedge$ as “and,” and so on—and assuming that its variables refer to a specific set of mathematical objects, such as the integers. Then the ideal formal system would be consistent and complete, in that all its theorems are true, and all true statements are theorems. Such a system would fulfill Hilbert’s dream of an axiomatic foundation for mathematics. It would be powerful enough to prove all truths, and yet be free from paradoxes.

In 1931, Kurt Gödel dashed Hilbert’s hopes. He proved the astonishing fact that *no sufficiently powerful formal system is both consistent and complete*. He did this by constructing a self-referential statement which can be interpreted as

This statement cannot be proved.

If this were false, it could be proved—but this would violate consistency by proving something false. Therefore, it must be true, and hence unprovable—and so there are truths that cannot be proved.

If all that Gödel had done was write down this sentence in English (or German), it would be a paradox about language, not mathematics. After all, natural language makes self-reference very easy, as the figure at the top of this page illustrates. He did much more than that. He showed, amazingly, that this kind of self-reference can happen even in formal systems that talk about the integers. In other words, he constructed a sentence that, on one level, claims that an integer with some property does not exist—but that, on another level, can be interpreted as referring to itself, and asserting that no proof of it exists.

Rather than following Gödel’s original proof, we will prove his Incompleteness Theorem using the undecidability of the Halting Problem. In other words, we will trade sentences that talk about themselves for programs that run themselves. Ultimately, these two forms of self-reference are the same—but what Gödel had to build from scratch, universal computation gives us for free.

We start by discussing the property $\text{Theorem}(T)$ that a statement T is provable. We can state it like this:

$$\text{Theorem}(T) = \exists P : \text{Proof}(P, T),$$

where $\text{Proof}(P, T)$ is the property that P is a valid proof of T . As we discussed in Section 6.1.3, Proof is decidable, since we can check P step-by-step and see if each line follows from the previous ones according to the axioms. Thus the set of theorems is recursively enumerable—we can go through all possible proofs, in order of increasing length, and print out all the theorems we find.



7.9

We are interested in formal systems that are powerful enough to talk about computation. First of all, we assume that they include the usual logical operators, including quantifiers like $\exists$ and $\forall$. Secondly, we assume that they can express claims like $\text{Halt}(\Pi, x)$, and that their axioms are strong enough to derive each step of a computation from the previous one.

Now let's ask whether such a system can prove all truths about halting or non-halting programs. If $\Pi(x)$ halts on its t th step, then a transcript of its computation is a proof of this fact, about t lines long. Therefore,

If $\text{Halt}(\Pi, x)$ is true, then it is provable.

But what if Π doesn't halt? Suppose that all true statements of the form $\overline{\text{Halt}(\Pi, x)}$ were provable. Then we could solve the Halting Problem, by doing two things in parallel: running $\Pi(x)$ to see if it halts, and looking for proofs that it won't. Since the Halting Problem is undecidable, there must exist statements of the form $\text{Halt}(\Pi, x)$ that are true, but are not provable. Thus, for some programs and inputs, the fact that they will never halt is an unprovable truth. In that case, neither $\text{Halt}(\Pi, x)$ nor $\overline{\text{Halt}(\Pi, x)}$ is a theorem—it is formally independent of our axioms.

Of course, we can always add $\overline{\text{Halt}(\Pi, x)}$ to our list of axioms. Then the fact that Π doesn't halt on input x becomes, trivially, a theorem of the system. But then there will be another program Π' , and another input x' , for which $\text{Halt}(\Pi', x')$ is true but not provable in this new system, and so on. No finite set of axioms captures all the non-halting programs—for any formal system, there will be some truth that it cannot prove.

7.10

Exercise 7.8 Suppose that a formal system is consistent, and powerful enough to discuss computation. Show that the set of theorems is recursively enumerable but not decidable. Equivalently, the set of non-theorems is not recursively enumerable.

Exercise 7.9 Theorems that are easy to state can be very hard to prove, and the shortest proof can be much longer than the statement of the theorem. Let $\text{ProofLength}(\ell)$ be the maximum, over all theorems T of length ℓ , of the length of the shortest proof of T . Show that ProofLength grows faster than any computable function.

7.3 Building Blocks: Recursive Functions

The fact that we can build a computer out of parts... is due to the recursive dependence of the basic operations between natural numbers.

Rózsa Péter, *Recursive Functions in Computer Theory*

How can we give a clear mathematical definition of computation? Intuitively, a function is computable if it can be defined in terms of simpler functions which are computable too. These simpler functions are defined in turn in terms of even simpler ones, and so on, until we reach a set of primordial functions for which no further explanation is necessary. These primordial functions form the “atoms” of computation. In terms of programming, they are the elementary operations that we can carry out in a single step.

Modern programming languages let us build up these definitions in a number of ways. Functions can call each other as subroutines; they can call themselves, reducing the problem to a simpler instance, until

they reach an easy base case; and they can repeat a set of operations many times by forming **for** and **while** loops. Each of these basic structures can be traced back to the theory of *recursive functions*, which was one of the three main models of computation leading to the unification of 1936. In this section, we take a look at this theory—its atoms and chemistry—and learn what ingredients we really need for universal computation.

7.3.1 Building Functions From Scratch

To build a set of computable functions recursively—in modern terms, to define a programming language—we need to define a set of primordial functions and a set of schemes by which functions can be combined. Aesthetically, we would like both these sets to be as small and simple as possible, while ensuring that we can build any conceivable computation.

As we will see, there is no need to include arithmetic operations like addition and multiplication among our primordial functions. In order to get arithmetic off the ground, all we need is the number zero, i.e., the constant function

$$0(x) = 0,$$

and the *successor* function,

$$S(x) = x + 1.$$

Strictly speaking, in order to use x on the right-hand side we also need to include the identity function $I(x) = x$. Hopefully everyone agrees that it is computable.

Now we need some schemes by which we can construct new functions from old ones. The first one is composition. If f and g are already defined, we can define a new function $h = f \circ g$ by

$$h(\mathbf{x}) = f(g(\mathbf{x})), \quad (7.5)$$

where $\mathbf{x} = (x_1, \dots, x_k)$ represents an arbitrary number of variables. In terms of programming, composition lets us call previously defined functions as subroutines, using the output of one as the input of the other.

More generally, we allow functions to access each of their variables, and to call a subroutine on any subset of them. For instance, if $f(x_1, x_2)$, $g(x_1, x_2)$ and $g'(x_3, x_1)$ are already defined, we can define

$$h(x_1, x_2, x_3) = f(g(x_1, x_2), g'(x_3, x_1)).$$

The second scheme is called *primitive recursion*. It allows us to define a function in terms of itself in a specific way: for one of its variables y , we can define its value at $y + 1$ in terms of its value at y , until we get to the base case $y = 0$. Thus if f and g are defined, we can define h as

$$\begin{aligned} h(\mathbf{x}, 0) &= f(\mathbf{x}) \\ h(\mathbf{x}, y + 1) &= g(\mathbf{x}, y, h(\mathbf{x}, y)). \end{aligned} \quad (7.6)$$

As Figure 7.4 shows, we can turn this into modern-looking pseudocode in two different ways. We can use recursion, as in the definition, or use a **for** loop to iterate through the values of y . Thus primitive recursion adds **for** loops to our programming toolbox.

Note that when we start evaluating a function defined using primitive recursion, we know to what depth it will recurse, since evaluating $h(\mathbf{x}, y)$ takes y levels of recursion. Equivalently, whenever we enter

<pre> h(x, y) begin if y = 0 then return f(x); else return g(x, y, h(x, y - 1)); end </pre>	<pre> h(x, y) begin z := f(x); for y' = 0 to y - 1 do z := g(x, y', z); end return z; end </pre>
---	--

FIGURE 7.4: Two ways to program the primitive recursive function $h(\mathbf{x}, y)$ defined in (7.6). On the left, we use recursion. On the right, we use a **for** loop.

the **for** loop of Figure 7.4, we know that it will run y times and then halt. Of course we disallow nasty tricks like changing the value of y from inside the loop.

Using primitive recursion, we can define addition as

$$\begin{aligned} \text{add}(x, 0) &= x \\ \text{add}(x, y + 1) &= S(\text{add}(x, y)), \end{aligned}$$

or, in plainer language,

$$x + 0 = x \text{ and } x + (y + 1) = (x + y) + 1.$$

Once we have addition, we can define multiplication and then exponentiation:

$$\begin{aligned} \text{mult}(x, 0) &= 0 \\ \text{mult}(x, y + 1) &= \text{add}(\text{mult}(x, y), x), \\ \text{exp}(x, 0) &= 1 \\ \text{exp}(x, y + 1) &= \text{mult}(\text{exp}(x, y), x). \end{aligned}$$

Exercise 7.10 Express the following functions in primitive recursive fashion: predecessor and subtraction,

$$\text{pred}(x) = \begin{cases} 0 & \text{if } x = 0 \\ x - 1 & \text{if } x > 0 \end{cases} \quad \text{sub}(x, y) = \begin{cases} 0 & \text{if } x < y \\ x - y & \text{if } x \geq y \end{cases},$$

positivity and less-or-equal,

$$\text{pos}(x) = \begin{cases} 0 & \text{if } x = 0 \\ 1 & \text{if } x > 0 \end{cases} \quad \text{leq}(x, y) = \begin{cases} 1 & \text{if } x \leq y \\ 0 & \text{if } x > y \end{cases},$$

and parity and division by two,

$$\text{even}(x) = x \bmod 2 = \begin{cases} 0 & \text{if } x \text{ is odd} \\ 1 & \text{if } x \text{ is even} \end{cases} \quad \text{div}_2(x) = \lfloor x/2 \rfloor.$$

Functions that can be generated from the primordial functions 0 and S by using composition and primitive recursion are called *primitive recursive*. In addition to the basic arithmetic functions we just defined, this includes many “higher” functions, such as `prime(x)` which is 1 if x is prime and 0 otherwise. Indeed, most functions we encounter in everyday life are primitive recursive—we can program them using subroutines, **if-then-else**, and **for** loops.

We claim, however, that not all computable functions are primitive recursive. Why? Because we can compute the value of any primitive recursive function, given its definition, with a finite number of steps. In other words, the universal function $U(f, x) = f(x)$ is computable, where by the first f we mean f ’s “source code”—a finite string defining f in terms of simpler functions, and defining those, all the way down to 0 and S.

If U were primitive recursive, the diagonalization argument of Section 7.2.1 would show that some primitive recursive functions are only partially defined—that, for some inputs, the corresponding program never halts. But since the number of times that each **for** loop runs is bounded, primitive recursive programs always halt, and their outputs are always defined. Therefore, they cannot interpret themselves—there is no primitive recursive function whose behavior includes that of all the others.

There is another way to show that the primitive recursive functions are a limited subset of the computable functions. We can define a function that is computable, but whose explosive growth outstrips that of any primitive recursive function. This is what we will do next.

7.3.2 Primitive Programming With BLOOP, and a Not-So-Primitive Function

To explore the power of primitive recursion, it helps to imagine writing programs in a fictional programming language, which we follow Douglas Hofstadter in calling BLOOP. The hallmark of BLOOP is that it has only bounded loops, such as **for** or **repeat y times** loops, where the number of iterations is fixed when we enter the loop.

Figure 7.5 shows BLOOP programs for addition, multiplication, and exponentiation. Instead of calling the earlier functions as subroutines, we write them out in the source code, so you can see how each level of primitive recursion creates another nested loop. The differences in the initial values, namely x for addition, 0 for multiplication, and 1 for exponentiation, are unimportant.

It seems that addition, multiplication, and exponentiation are the first three in a series of operations. What happens if we continued this series? Consider the following definitions:

$$\begin{aligned}
 A_1(x, y) &= x + y = x + \underbrace{1 + \cdots + 1}_{y \text{ times}} \\
 A_2(x, y) &= x \times y = \underbrace{x + x + \cdots + x}_{y \text{ times}} \\
 A_3(x, y) &= x^y = \underbrace{x \times x \times \cdots \times x}_{y \text{ times}} \\
 A_4(x, y) &= x^{x^{\cdot^{\cdot^{\cdot^x}}}} = \underbrace{x \wedge (x \wedge (\cdots \wedge x))}_{y \text{ times}}.
 \end{aligned}$$

You see the pattern: $A_n(x, y)$ is defined by applying x to itself y times using A_{n-1} . For $n \geq 3$, we can write

```

add(x,y)
begin
  z1 := x;
  repeat y times do
    z1 := z1 + 1;
  end
  return z1;
end

mult(x,y)
begin
  z2 := 0;
  repeat y times do
    z1 := x;
    repeat z2 times do
      z1 := z1 + 1;
    end
    z2 := z1;
  end
  return z2;
end

exp(x,y)
begin
  z3 := 1;
  repeat y times do
    z2 := 0;
    repeat z3 times do
      z1 := x;
      repeat z2 times do
        z1 := z1 + 1;
      end
      z2 := z1;
    end
    z3 := z2;
  end
  return z3;
end

```

FIGURE 7.5: BLOOP programs for addition, multiplication, and exponentiation, with one, two, and three levels of nested loops.

this recursively as

$$A_n(x, y) = \begin{cases} 1 & \text{if } y = 0 \\ A_{n-1}(x, A_n(x, y-1)) & \text{if } y > 0. \end{cases} \quad (7.7)$$

As n increases, $A_n(x, y)$ grows astronomically, as the following exercise will help you appreciate.

Exercise 7.11 List all pairs (x, y) for which $A_4(x, y)$ does not exceed 2^{64} . Compare $A_4(3, 4)$ to the number of atoms in the visible universe, which is roughly 10^{80} . Which is larger, $A_5(4, 2)$ or $A_4(5, 2)$?

The function $A_n(x, y)$ is called the *Ackermann function*. For each fixed n , it is a primitive recursive function of x and y , since, as Figure 7.6 shows, we can calculate it with a BLOOP program that has n levels of nested **for** loops. But what if n is a variable rather than a constant, giving a three-variable function $A(n, x, y) = A_n(x, y)$? Any particular BLOOP program has a fixed number of nested loops, so no such program can suffice for all n . This suggests that A is not primitive recursive.

In Problem 7.17, we lead you through a proof of this intuition. Specifically, we ask you to show that, for any primitive recursive function $f(y)$, there is an n such that A_n *majorizes* f in the following sense: for all $y \geq 3$,

$$f(y) < A_n(2, y). \quad (7.8)$$

```

 $A_n(x, y)$ 
begin
   $z_n := 1$ ;
  repeat  $y$  times do
     $z_{n-1} := 1$ ;
    repeat  $z_n$  times do
       $\vdots$       ( $n$  nested loops)
       $z_1 := x$ ;
      repeat  $z_2$  times do
         $z_1 := z_1 + 1$ ;
      end
       $z_2 := z_1$ ;
       $\vdots$ 
    end
     $z_n := z_{n-1}$ ;
  end
  return  $z_n$ ;
end

```

FIGURE 7.6: BLOOP program for the Ackermann function $A_n(x, y)$ for fixed n .

In essence, n is the number of nested **for** loops in the BLOOP program for f , or the number of levels of primitive recursion it takes to define f from S and 0 . Now, if $A(n, x, y)$ were primitive recursive, so would be the special case $f(y) = A_y(2, y)$. But then for some n we would have

$$A_y(2, y) < A_n(2, y).$$

If we set $y = n$, which is yet another form of diagonalization, we get a contradiction.

We can certainly compute $A(n, x, y)$ in finite time, since applying the recursive equation (7.7) repeatedly will eventually reach a base case. However, unless n is constant, there is no way to express (7.7) in primitive recursive form, i.e., with a constant number of equations of the form (7.6)—it is a more complicated type of recursion.

Thus $A(n, x, y)$ is a concrete example of a function that is computable but not primitive recursive. What kind of programming language do we need to compute it? What is missing from BLOOP?



7.12

7.3.3 Run Until You're Done: *while* Loops and μ -Recursion

But what has been said once can always be repeated.

Zeno of Elea

We can always program the Ackermann function by translating (7.7) directly into pseudocode. This gives a recursive algorithm, as shown on the left side of Figure 7.7, in which $A(n, x, y)$ calls itself on smaller

```

A(n, x, y)
begin
  if n = 2 then
    return mult(x, y);
  else if y = 0 then
    return 1;
  else
    return A(n - 1, x, A(n, x, y - 1));
end

A(n, x, y)
begin
  push(n);
  while stack not empty do
    pop(n);
    if n = 2 then
      y := mult(x, y);
    else if y = 0 then
      y := 1;
    else
      y := y - 1;
      push(n - 1);
      push(n);
    end
  end
  return y;
end

```

FIGURE 7.7: We can implement the Ackermann function recursively, or iteratively using a **while** loop and a stack. For convenience we assume that $n \geq 2$ and use $A(2, x, y) = \text{mult}(x, y)$ as our base case.

values of n . But what if we want to “unfold” this recursion as in Figures 7.5 and 7.6? What kind of loop do we need to calculate $A(n, x, y)$ iteratively instead of recursively?

We can carry out a recursive algorithm by using a *stack* to keep track of the tasks and subtasks along the way. We kick the process off by pushing the number n on the stack. Then, at each step, we pop the top number off the stack to see what value of n we are working on. Recall from (7.7) that

$$A(n, x, y) = A(n - 1, x, A(n, x, y - 1)).$$

This tells us that, in order to calculate $A(n, x, y)$, we have to calculate $A(n, x, y - 1)$ first. To do this, we decrement y , and push another copy of n on the stack. But first we push $n - 1$ on the stack, in order to remember that after we calculate $y' = A(n, x, y - 1)$, we will feed it into $A(n - 1, x, y')$.

We continue in this way, pushing subtasks onto the stack, until we reach a base case where we can compute $A(n, x, y)$ directly. To keep things simple, in Figure 7.7 we define our base case as $n = 2$, at which point we call the `mult` function on x and y . Once we have computed $A(n, x, y)$, we use the variable y to pass its value back to the subtask below it on the stack. Thus we push and pop subtasks until the stack is empty, our task is done, and we return the answer.

It is a simple matter to encode an entire stack as a single integer, and to write primitive recursive functions `push`, `pop`, and `isempty` to manipulate it. For one method, see Problem 7.18. So each step of this process is primitive recursive. But how many times do we need to repeat it? If we knew the running time in advance, we could put the entire process inside a **for** loop. Unfortunately, the running time is essentially $A(n, x, y)$ itself. So we can't use a bounded loop, since the bound is the function we're trying to calculate in the first place.

What we need is a **while** loop, which runs as long as some condition is satisfied—in this case, as long as there are still tasks left to be done on the stack. In the theory of recursive functions, we represent these

loops with an operator called μ -recursion. Given a function $f(\mathbf{x}, y)$ which is already defined, μ -recursion lets us define a new function $h(\mathbf{x})$ as follows:

$$h(\mathbf{x}) = \mu_y f(\mathbf{x}, y) = \min\{y : f(\mathbf{x}, y) = 0\}. \quad (7.9)$$

Thus $h(\mathbf{x})$ returns the smallest solution y to the equation $f(\mathbf{x}, y) = 0$. This corresponds to the following **while** loop:

```

h(x)
begin
  y := 0;
  while f(x, y) ≠ 0 do y := y + 1;
  return y;
end

```

Of course, there may be no such y , and this **while** loop might never halt. In that case, $h(\mathbf{x})$ is undefined. Thus when we add μ -recursion and **while** loops to our toolbox, we gain a great deal of computational power, but at a cost—we must now include partial functions, which might be undefined for some values of their input. Indeed, when we apply μ to partial functions, we require that $f(\mathbf{x}, y')$ is defined for all the y' we need to check:

$$\mu_y f(\mathbf{x}, y) = \min\{y : f(\mathbf{x}, y) = 0 \text{ and } f(\mathbf{x}, y') \text{ is defined for all } y' \leq y\}. \quad (7.10)$$

Functions that can be generated from the primordial functions 0 and S using composition, primitive recursion, and μ -recursion are called *partial recursive*. Since the corresponding programming language allows *free* loops in addition to bounded ones, we follow Hofstadter in calling it FLOOP. Unlike BLOOP, FLOOP contains all the basic constructs that appear in modern programming languages. So, we can just as well define the partial recursive functions as those you can compute in C, FORTRAN, or whatever your favorite programming language is.

We can consider the subclass of *total* recursive functions, i.e., those which are defined for all values of their input. Since there exist functions, such as the Ackermann function, that are total but not primitive recursive, the primitive recursive functions form a proper subclass of the total recursive ones:

$$\text{primitive recursive} \subset \text{total recursive} \subset \text{partial recursive}.$$

In terms of programming,

$$\text{BLOOP programs} \subset \text{FLOOP programs that always halt} \subset \text{FLOOP programs}.$$

7.3.4 A Universal Function

At the end of Section 7.3.1, we pointed out that the universal function $U(f, x) = f(x)$ is not primitive recursive, even when f is restricted to the primitive recursive functions. In other words, BLOOP is not strong enough to interpret itself—it is not flexible enough to bite its own tail.

FLOOP, on the other hand, does have this self-referential power. In other words, there is a partial recursive function U , such that $U(f, x) = f(x)$ for any partial recursive function f and any x , with the understanding that $U(f, x)$ is well-defined if and only if $f(x)$ is.

We can justify this claim by writing U in FLOOP. Since we don't know f 's running time, we use a **while** loop to run it until it halts:

```

     $U(f, x)$ 
    begin
        initialize  $f$  with input  $x$  ;
        while  $f$  hasn't halted yet do
            run a step of  $f$  ;
        return  $f$ 's output ;
    end

```

For now, we beg the reader's indulgence on the line "run a step of f ," since there is a more serious issue to deal with first. By definition, partial recursive functions act on integers, not on functions. How can we feed one function to another as input?

In the modern age, the answer is simple. Each partial recursive function f has a source code—a FLOOP program, or a series of definitions using composition, primitive recursion, and μ -recursion. In either case, this source code is a finite string. If it is written in an alphabet of, say, 50 symbols, we can write it out and treat it as an integer written in base 50. Thus every program is an integer that describes the function it computes, and we can feed this integer to U .

In the early days of computation, however, this question threw people for a loop. The fact that a function can be encoded as an integer, which can in turn be acted on by other functions—in other words, that functions can act both as programs and as data—came as a revelation.

This idea first appeared in the work of Gödel. In order to feed them to other functions, he devised a way to give each function an integer description—what we now call a *Gödel number*. Let $\bar{h}$ denote the Gödel number of a partial recursive function h . We can start by defining $\bar{h}$ for the zero, successor, and identity functions—the atoms of our language—and then work upward from there. One possible scheme might look like this:

$$\bar{h} = \begin{cases} 7 & \text{if } h = 0 \\ 11 & \text{if } h = S \\ 13 & \text{if } h = I \\ 2 \cdot 3^{\bar{f}} \cdot 5^{\bar{g}} & \text{if } h = f \circ g \\ 2^2 \cdot 3^{\bar{f}} \cdot 5^{\bar{g}} & \text{if } h \text{ is defined from } f \text{ and } g \text{ using primitive recursion} \\ 2^3 \cdot 3^{\bar{f}} & \text{if } h = \mu_y f. \end{cases} \quad (7.11)$$

Note that these descriptions are unambiguous, since the prime factorization of an integer is unique. The Gödel number of our addition function would then be

$$\overline{\text{add}} = 2^2 \cdot 3^{13} \cdot 5^{11} = 311391210937500.$$

We need to add some bells and whistles to deal with functions of several variables, but you get the idea. The Gödel number of h is defined inductively from the Gödel numbers of simpler functions, until we get down to elementary functions that don't have to be defined. Thus it is much like a program in a modern programming language, albeit a little less readable.

Now we claim that, using this Gödel numbering or any other reasonable definition of a function's source code, the universal function $U(f, x) = f(x)$ is partial recursive. The proof of this claim is rather technical, and with a thankful nod to the pioneers of computation, we relegate its details to history. But the idea is simple. There are primitive recursive functions that “read” integers, either in a format like (7.11) or as a string of symbols, and parse it as a string of instructions. Then there is a primitive recursive function `step` that executes a single instruction, updates the variables that keep track of intermediate results, and keeps track of what instruction it should execute next. Thus the line “run a step of f ” in our pseudocode for U is primitive recursive, and U is partial recursive.

Exercise 7.12 Kleene's Normal Form Theorem. *Sketch a proof that there exist primitive recursive functions f and g such that every partial recursive function h can be written in the form*

$$h(\mathbf{x}) = g(\mathbf{x}, \mu_y f(p, \mathbf{x}, y))$$

*for some integer p . Therefore, any partial recursive function can be written with a single use of μ -recursion. Equivalently, any partial recursive function can be computed by a FLOOP program with a single **while** loop.*

We have seen that the partial recursive functions are very rich, and that they can simulate and interpret themselves just as today's computers can. Nevertheless, we can still ask whether the set of partial recursive functions is synonymous with the set of computable ones. Do composition, primitive recursion, and μ -recursion cover every reasonable way in which functions can be defined inductively in terms of simpler ones? Equivalently, can all possible computations be carried out with basic arithmetic, along with **for** and **while** loops?

With the benefit of hindsight, the answer is yes—otherwise, there is something missing from all the programming languages we use today. However, if we put ourselves in the shoes of our counterparts in 1936, we have no reason to be convinced of this. Let's look at another approach to computation, which at first seems totally different from the partial recursive functions, but which turns out to be exactly equivalent to them.

7.4 Form is Function: the λ -Calculus

You say that I pay too much attention to form.
Alas! it is like body and soul: form and content
to me are one; I don't know what either is
without the other.

Gustave Flaubert

Traditional mathematics is a rigidly stratified society. At the bottom of the ladder we have numbers. Above them are functions, which act on numbers and return numbers. On the level above that are more abstract operators, such as composition, primitive recursion, or the derivative, which take one or more functions as input and return functions as output.

This strict hierarchy is perfectly fine for almost all of mathematics, and as we discussed in Section 7.1.2, it banishes a host of paradoxes based on self-reference. But for computer science, it is too confining. We need to be able to make statements such as $U(f, x) = f(x)$ where objects live on two levels at once—where

f plays two roles, as both a function and a piece of data that can be fed to other functions. Level-crossing statements like these are our bread and butter, including “loopy” ones like $U(f, f) = f(f)$ where we feed a function to itself.

If we have to, we can close these loops using Gödel numbers, where each function f has a corresponding integer $\bar{f}$, but this involves a lot of technical work. In the modern world, programs and data are both simply finite strings, and we can feed any string to any program without breaking any laws. Is there an elegant mathematical model which is similarly democratic?

The logician Alonzo Church invented a system, called the λ -calculus, in which functions at all levels of abstraction can interact with each other as equals. At first, the notation used in this system is quite strange, and even forbidding. However, it has its own charms which will reward the persistent reader. It represents an attitude towards computation that is very different from other models, and which still survives today in functional programming languages such as LISP and SCHEME. Because it makes no distinction between functions and data, it makes diagonal arguments, like the undecidability of the Halting Problem, elegant and immediate.

7.4.1 Functions are Forms

The format of the λ -calculus takes a little getting used to. First, rather than writing

$$f(x) = x^2,$$

we will write

$$f = \lambda x . x^2.$$

In other words, $\lambda x . x^2$ is the function that takes a variable x and returns x^2 . If we place a number to the right of this function, it substitutes that number for the variable x :

$$(\lambda x . x^2)7 = 49.$$

So far, this is just a strange shift in notation. But the way the λ -calculus treats the *process* of computation is also very different. Consider the addition function, which we write

$$\lambda xy . x + y.$$

Let’s apply this to 3 and 5. Rather than writing

$$(\lambda xy . x + y)(3, 5),$$

we write

$$((\lambda xy . x + y)3)5 = (\lambda y . 3 + y)5 = 8.$$

Halfway through this computation, we have the object $\lambda y . 3 + y$. In other words, having fixed $x = 3$, we have a function on the other variable y , which adds 3 to it.

This is a subtle but powerful shift in how we think about performing a computation. Rather than modifying variables and passing their values up to functions, we evaluate functions from the top down, modifying what they will do with the rest of their data when they get it. The notation λxy is really a shorthand, which works like this:

$$\lambda xy . x + y = \lambda x . (\lambda y . x + y).$$

Thus addition is a function on *one* variable x , which, when given x , returns the function that adds x to its input y .

The purest form of the λ -calculus doesn't have arithmetic operations like addition, or even the concept of the integers. The basic objects in the λ -calculus are strings, or *combinators*, which take the next string to their right and substitute it for each appearance of the first variable marked with λ . In other words, if x is a variable and U and V are strings, then

$$(\lambda x.U)V = U[x \rightarrow V], \quad (7.12)$$

where $U[x \rightarrow V]$ denotes the string we get from U by replacing each appearance of x with V . Each such step is called a *reduction*. A computation then consists of a chain of reductions, such as

$$(\lambda xy . yxy)ab = (\lambda y . y ay)b = bab.$$

Note that we evaluate strings from left to right, and that in the absence of parentheses, we interpret the string zab as $(za)b$. The operation of reduction is nonassociative, meaning that $z(ab) \neq (za)b$ in general. For instance, grouping a and b together in this example gives

$$(\lambda xy . yxy)(ab) = \lambda y . y(ab)y.$$

This process of substituting strings for symbols is all the λ -calculus ever does. Each combinator simply provides a form for strings to plug into. And yet, these forms give us all the function we need. As we will prove below, the λ -calculus is just as powerful as our other models of computation.

To see how things might get interesting, let's look at some examples where we feed one combinator to another. Consider the following three combinators:

$$\begin{aligned} \mathbf{T} &= \lambda xy . x \\ \mathbf{F} &= \lambda xy . y \\ \mathbf{I} &= \lambda x . x \end{aligned} \quad (7.13)$$

The combinators $\mathbf{T}$ and $\mathbf{F}$ take two inputs and return the first or second one. Since $\mathbf{I}$ is the identity, returning its input unchanged, we have $\mathbf{IT} = \mathbf{T}$ and $\mathbf{IF} = \mathbf{F}$. But if we combine these in the opposite order, something funny happens:

$$\begin{aligned} \mathbf{TI} &= (\lambda xy . x)(\lambda x . x) \\ &= (\lambda xy . x)(\lambda z . z) \\ &= \lambda y . (\lambda z . z) \\ &= \lambda yz . z \\ &= \mathbf{F}. \end{aligned}$$

In the second line, we changed the variable in $\mathbf{I}$ from x to z to avoid ambiguity, and we renamed variables again in the last line to identify $\lambda yz . z$ with $\mathbf{F}$. These renamings are allowed because $\lambda x . x$ and $\lambda z . z$ are both the combinator $\mathbf{I}$, and $\lambda xy . y$ and $\lambda yz . z$ are both the combinator $\mathbf{F}$, just as $f(x) = x^2$ and $f(y) = y^2$ are both the function that returns the square of its input.

Some products, such as **TI**, reach a final state, or *normal form*, in which no more reductions are possible. Either there are no λ s left, or there are no strings to the right of the first combinator to plug into it. However, this is not true of all expressions. Consider the combinator $\mathbf{Q} = \lambda x . xx$, which takes x as input and returns xx . If we feed $\mathbf{Q}$ to itself, we get

$$\begin{aligned}\mathbf{Q}\mathbf{Q} &= (\lambda x . xx)(\lambda x . xx) = (\lambda x . xx)(\lambda x . xx) \\ &= (\lambda x . xx)(\lambda x . xx) \\ &= \dots\end{aligned}$$

The chain of reductions never halts. We keep plugging the second $\lambda x . xx$ into the λx of the first, and the expression never reaches a normal form. Even worse is the following expression, which grows without bound as we try to reduce it:

$$\begin{aligned}(\lambda x . xx)(\lambda y . yyy) &= (\lambda y . yyy)(\lambda y . yyy) \\ &= (\lambda y . yyy)(\lambda y . yyy)(\lambda y . yyy) \\ &= (\lambda y . yyy)(\lambda y . yyy)(\lambda y . yyy)(\lambda y . yyy) \\ &= \dots\end{aligned}$$

These are examples of infinite loops in the λ -calculus. Like partial recursive functions, λ -expressions may never return an answer, and the functions they compute may be undefined for some values of their input. Telling which expressions reach a normal form and which ones don't is λ -calculus's version of the Halting Problem, and we will prove below that it is undecidable.

As Problem 7.21 shows, we can also think of $\mathbf{Q}\mathbf{Q} = (\lambda x . xx)(\lambda x . xx)$ as a “quine”—a program that prints out its own source code. We can even use it as a recipe to create quines in other programming languages.



7.13

7.4.2 Numbers Are Functions Too

So far, so good. But how does all this string manipulation actually compute anything? For an example of how λ -calculus can perform simple logical operations, take a look at the following exercise:

Exercise 7.13 Consider the combinators **T** and **F** defined in (7.13). As their names imply, let's think of these as representing *true* and *false* respectively. Show that we can then represent Boolean functions as follows:

$$\begin{aligned}\mathbf{not} &= \lambda pxy . pyx \\ \mathbf{and} &= \lambda pq . ppq \\ \mathbf{or} &= \lambda pq . ppq.\end{aligned}$$

For instance, show that

$$\mathbf{and}\ \mathbf{TF} = (\mathbf{and}\ \mathbf{T})\mathbf{F} = \mathbf{IF} = \mathbf{F}.$$

Of course, there is no inherent connection between **T** and **F** and the truth values *true* and *false*. Indeed, λ -calculus allows us to manipulate strings in such general ways that nearly any two strings could play these roles.

How about arithmetic? There are many ways to represent the natural numbers $0, 1, 2, 3 \dots$ in λ -calculus. We will use the *Church numerals*, defined as follows:

$$\begin{aligned}\bar{0} &= \lambda f x . x , \\ \bar{1} &= \lambda f x . f x , \\ \bar{2} &= \lambda f x . f(f x) , \\ \bar{3} &= \lambda f x . f(f(f x)) ,\end{aligned}$$

and so on. Given a function f , the Church numeral $\bar{n}$ returns the function f^n , i.e., f iterated n times:

$$\bar{n} f = \lambda x . f^n x . \quad (7.14)$$

To flex our muscles, let's define combinators that carry out arithmetic operations on these numerals. First, consider

$$\mathbf{succ} = \lambda m f x . f(m f x) . \quad (7.15)$$

If $\bar{m}$ is a numeral, $\mathbf{succ} \bar{m}$ applies f one more time than $\bar{m}$ would. Thus $\mathbf{succ}$ acts as the successor function. Let's check this, reducing one step at a time:

$$\begin{aligned}\mathbf{succ} \bar{n} &= (\lambda m f x . f(m f x))(\lambda g y . g^n y) \\ &= \lambda f x . f((\lambda g y . g^n y) f x) \\ &= \lambda f x . f((\lambda y . f^n y) x) \\ &= \lambda f x . f(f^n x) \\ &= \lambda f x . f^{n+1} x \\ &= \overline{n+1} .\end{aligned}$$

In contrast to traditional numbers, Church numerals are not simply passive data. They are active functions, which create iterated versions of other functions. This lets us write inductive definitions of functions very compactly. For instance, suppose we want a combinator $\mathbf{add}$ that adds two numerals together. One way to do this is

$$\mathbf{add} = \lambda n m f x . n f(m f x) . \quad (7.16)$$

Since $f^m(f^n(x)) = f^{m+n}(x)$, we have

$$\mathbf{add} \bar{m} \bar{n} = \overline{m+n} .$$

But we can also describe adding n as iterating the successor function n times. So alternately, we could write

$$\mathbf{add}' = \lambda n . n \mathbf{succ} . \quad (7.17)$$

Problem 7.22 shows that $\mathbf{add}$ and $\mathbf{add}'$ are equivalent, at least when applied to Church numerals.

For multiplication, we use the fact that if we iterate f^n m times we get $(f^n)^m = f^{nm}$. Thus if

$$\mathbf{mult} = \lambda n m f . n(m f) ,$$

we have

$$\mathbf{mult} \bar{m} \bar{n} = \overline{m n} .$$

Note that all these combinators are happy to take inputs n and m that are not Church numerals, but in that case their behavior may have nothing to do with arithmetic.

Finally, what happens if we apply one numeral to another? Applying $\bar{n}$ to $\bar{m}$ means iterating, n times, the operator that changes f to f^m . This may seem a bit abstract, but this is the sort of thing that λ -calculus is good at. The answer is given by the following exercise.

Exercise 7.14 Show that if

$$\mathbf{exp} = \lambda n m . m n$$

then

$$\mathbf{exp} \bar{n} \bar{m} = \overline{m^n}.$$

Writing the successor function for Church numerals was easy, and this let us construct combinators for addition, multiplication, and exponentiation. But what about the predecessor? Is there a combinator **pred** such that **pred** $\bar{n} = \overline{n-1}$ if $n > 0$, and **pred** $\bar{0} = \bar{0}$? Constructing **pred** is a little tricky, and Church himself believed for a while that it was impossible. His student Stephen Kleene found a clever solution, which we invite you to work out in Problem 7.23.

7.4.3 The Power of λ

When one first works with the λ -calculus it seems artificial and its methods devious; but later one appreciates its simplicity and power.

Robin Gandy

As we have seen, λ -calculus is strong enough to do basic arithmetic. Can it do more? Let us say that a function $f : \mathbb{N} \rightarrow \mathbb{N}$ is λ -definable if there is a combinator ϕ such that

$$\phi \bar{n} = \overline{f(n)}$$

for any Church numeral $\bar{n}$, as long as $f(n)$ is defined. How expressive is the λ -calculus compared to the recursive functions? Are all partial recursive functions λ -definable, or vice versa?

As described in Section 7.3, the partial recursive functions are defined inductively from zero and the successor function using composition, primitive recursion, and μ -recursion. We already have zero and the successor. We can compose two functions with the following combinator,

$$\mathbf{comp} = \lambda f g x . f(gx), \tag{7.18}$$

which takes two functions f, g as input and returns $f \circ g$. In fact, this is the same combinator **mult** that we used to multiply Church numerals above. If we can also perform primitive recursion and μ -recursion in λ -calculus, then by induction all partial recursive functions are λ -definable.

One way to handle primitive recursion is to treat it as a **for** loop. We can easily write a **for** loop that runs n times using the Church numeral $\bar{n}$. We ask you to do this for the factorial function in Problem 7.24, and for primitive recursion in general in Problem 7.25. This shows that any primitive recursive function is λ -definable.

However, a **for** loop is often not the most elegant way to define a function. We would like to be able to use recursive definitions like this,

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n(n-1)! & \text{if } n > 0. \end{cases} \quad (7.19)$$

This requires us to know when we have reached the base case $n = 0$, and to act accordingly. We can do this using the following exercise:

Exercise 7.15 Show that if

$$\mathbf{iszero} = \lambda n . n(\lambda x . \mathbf{F})\mathbf{T}$$

where $\mathbf{F}$ and $\mathbf{T}$ are defined as in (7.13), then

$$\mathbf{iszero} \bar{n} x y = \begin{cases} x & \text{if } n = 0 \\ y & \text{if } n > 0. \end{cases}$$

Using **iszero**, **mult**, and the predecessor **pred**, it seems that we can write the factorial as

$$\mathbf{fac} = \lambda n . \mathbf{iszero} n \bar{1}(\mathbf{mult} n(\mathbf{fac}(\mathbf{pred} n))).$$

However, this definition doesn't quite work. Like any other combinator, **fac** must be a finite string of symbols. We are free to use previously-defined functions as shorthand, but defining it directly in terms of itself would lead to an infinite regress.

This makes it seem that functions in λ -calculus cannot be defined recursively in terms of themselves. Yet we don't want to lose the expressive elegance of recursion. How can we arrange for λ -expressions like **fac** to refer to themselves indirectly, rather than directly?

The idea is to treat recursive definitions like (7.19) as equations that functions must satisfy. Let's define an operator **R** as follows:

$$(\mathbf{R}f)(n) = \begin{cases} 1 & \text{if } n = 0 \\ n f(n-1) & \text{if } n > 0, \end{cases} \quad (7.20)$$

or, in λ -style,

$$\mathbf{R} = \lambda f n . \mathbf{iszero} n \bar{1}(\mathbf{mult} n(f(\mathbf{pred} n))). \quad (7.21)$$

The effect of **R** is to modify a function f by applying a single step of the recursion (7.19). Then $f(n) = n!$ is the unique function such that

$$f = \mathbf{R}f.$$

In other words, f should be a *fixed point* of **R**.

But how can we know that **R** has a fixed point? And if it exists, how can we find it? In traditional mathematics, functions may or may not have fixed points, and for those that do, it is generally impossible to calculate them explicitly. One of the most beautiful results of λ -calculus is that such fixed points always exist, and that we can actually construct them.

Theorem 7.1 (Fixed Point Theorem) For any λ -expression **R**, there is at least one f such that

$$\mathbf{R}f = f.$$

Moreover, one such f can be written

$$f = \mathbf{Y}\mathbf{R},$$

where

$$\mathbf{Y} = \lambda R. (\lambda x. R(xx)) (\lambda x. R(xx)). \quad (7.22)$$

The combinator $\mathbf{Y}$ is called a *fixed point combinator*. It seems quite magical—given any combinator $\mathbf{R}$, it returns a fixed point of $\mathbf{R}$. Skeptical? Here's the proof:

$$\begin{aligned} \mathbf{Y}\mathbf{R} &= \lambda R. (\lambda x. R(xx)) (\lambda x. R(xx)) \mathbf{R} \\ &= (\lambda x. \mathbf{R}(xx)) (\lambda x. \mathbf{R}(xx)) \\ &= \mathbf{R}((\lambda x. \mathbf{R}(xx)) (\lambda x. \mathbf{R}(xx))) \\ &= \mathbf{R}(\mathbf{Y}\mathbf{R}). \end{aligned}$$

Nor is $\mathbf{Y}$ the only combinator with this property—there are an infinite number of fixed point combinators. We give two other examples in Problem 7.26.

With $\mathbf{Y}$'s help, we can define the factorial as $\mathbf{Y}\mathbf{R}$ where $\mathbf{R}$ is defined as in (7.21). At first, this definition seems like abstract nonsense. But it is perfectly concrete, and $\mathbf{fac} = \mathbf{Y}\mathbf{R}$ really does compute $n!$ in a finite number of steps. As we reduce it, it repeatedly applies the recursion $\mathbf{R}$,

$$\mathbf{fac} \bar{n} = \mathbf{R}(\mathbf{fac}) = \mathbf{R}(\mathbf{R}(\mathbf{fac})) \bar{n} = \dots,$$

or, in more familiar notation,

$$n! = n(n-1)! = n(n-1)(n-2)! = \dots,$$

until it reaches the base case $n = 0$ and returns the answer.

As we show in Problem 7.27, this ability to construct fixed points is not limited to the λ -calculus. In any programming language capable of universal computation, if $\mathbf{R}$ is a program that modifies other programs, there is a program f such that $\mathbf{R}(f)$ computes the same function that f does—and there is a program $\mathbf{Y}$ such that $\mathbf{Y}(\mathbf{R})$ is one such f . But without the loopy power of the λ -calculus, in which functions can act on functions that act on other functions, it would be difficult to juggle the different levels of abstraction at play here.

Our proof that all partial recursive functions are λ -definable will be complete if we can also handle μ -recursion. First, recall its definition. If $f(x, y)$ is partial recursive, we can define a function $h(x)$ as the smallest y such that $f(x, y) = 0$:

$$h(x) = \mu_y f(x, y) = \min\{y : f(x, y) = 0\}, \quad (7.23)$$

assuming that $f(x, y)$ is defined for all $y \leq h(x)$. As we discussed in Section 7.3.3, μ -recursion is like a **while** loop. We don't know in advance how many times it will repeat, or if it will ever halt at all. This is similar in spirit to iterating a function until we reach a fixed point, and we can express $h(x)$ in this style.

First let ϕ be a function which takes a pair (x, y) as input. It explores larger and larger values of y until it finds one such that $f(x, y) = 0$, which it returns:

$$\phi(x, y) = \begin{cases} y & \text{if } f(x, y) = 0 \\ \phi(x, y + 1) & \text{otherwise.} \end{cases}$$

Then we can write

$$h(x) = \phi(x, 0).$$

If f is λ -definable, we can write the recursive definition

$$\phi = \lambda x y . \mathbf{iszero}(fxy)y(\phi x(\mathbf{succ}y)).$$

Thus ϕ is the fixed point of the function $\mathbf{H}f$, where

$$\mathbf{H} = \lambda \phi x y . \mathbf{iszero}(fxy)y(\phi x(\mathbf{succ}y)),$$

and we can write

$$h = \lambda x . \phi x \bar{0} \text{ where } \phi = \mathbf{YH}.$$

In practice, this function works like this:

$$\begin{aligned} hx &= \phi x \bar{0} = \bar{0} \text{ if } f(x, 0) = 0, \text{ or } \dots \\ &= \mathbf{H}(\phi x \bar{0}) = \phi x \bar{1} = \bar{1} \text{ if } f(x, 1) = 0, \text{ or } \dots \\ &= \mathbf{H}(\mathbf{H}(\phi x \bar{0})) = \phi x \bar{2} = \bar{2} \text{ if } f(x, 2) = 0, \text{ or } \dots \end{aligned}$$

If $f(x, y) \neq 0$ for all y , this chain of reductions goes on forever, exactly as the corresponding **while** loop would. On the other hand, if $f(x, y) = 0$ for some y , and if $f(x, y')$ is defined for all $y' < y$, then we arrive at a normal form after a finite chain of reductions, and $h(x)$ is defined.

Thus if f is λ -definable, so is h . By stepping up one more level of abstraction, we can even write μ -recursion itself as a combinator in the λ -calculus, which takes f and returns h :

$$\mu = \lambda f x . \mathbf{Y}(\mathbf{H}'f)x\bar{0} \text{ where } \mathbf{H}' = \lambda f \phi x y . \mathbf{iszero}(fxy)y\phi(x\mathbf{succ}y).$$

This completes the proof that all partial recursive functions are λ -definable.

It is not too hard to see, at least intuitively, that the converse is true as well—all λ -definable functions are partial recursive. Like any other finite string of symbols, we can think of a λ -expression as an integer. Then there is a primitive recursive function that carries out a single reduction step, by searching for strings of digits in an integer and substituting them according to (7.12). We can iterate this function, and find its fixed point if one exists, using μ -recursion. Therefore, the partial function **eval** which maps each expression to its normal form, and which is undefined if the reduction process never halts, is partial recursive. This was proved by Kleene in 1936, showing that the λ -calculus and the partial recursive functions are exactly equivalent in their computational power.

7.4.4 Universality and Undecidability

In Section 7.3, we saw that in order to define a universal function $U(f, x) = f(x)$ in the partial recursive functions, we had to go through a somewhat complicated process of assigning a Gödel number to each function. In the λ -calculus, in contrast, functions can already take other functions as input, so no such translation is needed. In a sense, the universal function is simply the identity:

$$\mathbf{U} = \lambda f x . fx.$$

This same ability to handle abstraction makes it very easy to prove in the λ -calculus that the Halting Problem is undecidable. That is, we can prove that there is no combinator **halts** that tells whether a given λ -expression $\alpha\beta$ reaches a normal form after a finite series of reductions:

$$\mathbf{halts}\,\alpha\beta = \begin{cases} \mathbf{T} & \text{if } \alpha\beta \text{ reduces to a normal form} \\ \mathbf{F} & \text{otherwise,} \end{cases}$$

where $\mathbf{T} = \lambda xy . x$ and $\mathbf{F} = \lambda xy . y$ as before.

The proof exactly mirrors the one we gave in Section 7.2.2, but in the λ -calculus it is even more compact. Suppose that **halts** exists. Then let **normal** be some λ -expression, such as $\lambda x . x$, which is already in normal form—and let **never** be an expression such as $(\lambda x . xx)(\lambda x . xx)$ which never halts.

Now define the combinator

$$\mathbf{C}_{22} = \lambda\alpha . (\mathbf{halts}\,\alpha\alpha) \mathbf{never} \mathbf{normal},$$

where **C** stands for “catch.” Then $\mathbf{C}_{22}\,\alpha$ evaluates to normal form if and only if $\alpha\alpha$ never does. Feeding this beast to itself gives a contradiction, since the λ -expression

$$\mathbf{C}_{22}\,\mathbf{C}_{22}$$

reaches normal form if and only if it doesn’t. We are forced to conclude that the combinator **halts** does not exist. Thus the λ -calculus cannot solve its own Halting Problem. Equivalently, it cannot tell whether a partial recursive function α is defined on a given input β .

7.14

7.5 Turing’s Applied Philosophy

What does a Turing machine look like? You can certainly imagine some crazy looking machine, but a better approach is to look in a mirror.

Charles Petzold, *The Annotated Turing*

The partial recursive functions and the λ -calculus are clearly very powerful, and we have sketched a proof that they are equivalent. But why should we believe that they encompass the power of all possible computations? Are the particular forms of recursion and substitution that they offer general enough to carry out anything that Hilbert would have called a finite procedure?

Before the work of Alan Turing, this was genuinely unclear. Several other ways to define recursive functions had been proposed, and Gödel himself found the λ -calculus “thoroughly unsatisfactory.” It was not until Turing’s proposal of a simple model of computation, and the proof that all three of these models are equivalent, that a consensus was established about the definition of computability.

7.5.1 Meet the Computer

Turing’s machine begins as a model of a *human* computer like those shown in Figure 7.8, not an electronic one—a human carrying out a mathematical procedure according to a fixed set of rules. He distills this process down to its bare essentials, until it consists of elementary steps in which we read and write single symbols. In his own words:



FIGURE 7.8: Computers in the Hamburg observatory in the 1920s.

Computing is usually done by writing certain symbols on paper. We may suppose this paper is divided into squares like a child's arithmetic book... The behavior of the computer at any moment is determined by the symbols which he is observing, and of his "state of mind" at that moment.

Let us imagine the operations performed by the computer to be split up into "simple operations" which are so elementary that it is not easy to imagine them further divided. Every such operation consists of some change of the physical system consisting of the computer and his tape... We may suppose that in a simple operation not more than one symbol is altered. Any other changes can be split up into simple changes of this kind.

While paper is two-dimensional, Turing points out that a one-dimensional paper tape will do, with one symbol written in each square on the tape. Since we can only distinguish a finite number of different symbols from each other, he requires that the "alphabet" of symbols that can be written on the tape be finite. He makes the same argument about "states of mind," saying that these form a finite set as well. One argument for this is the finiteness of our brains. But he gives another argument as well:

It is always possible for the computer to break off from his work, to go away and forget all about it, and later to come back and go on with it. If he does this he must leave a note of instructions (written in some standard form) explaining how the work is to be continued. This note is the counterpart of the "state of mind." We will suppose that the computer works

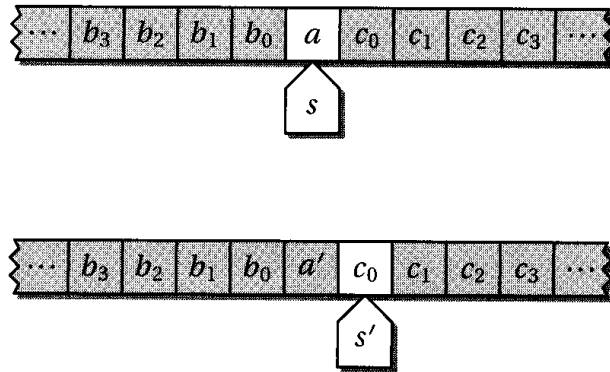


FIGURE 7.9: One step of a Turing machine. It changes the tape symbol from a to a' , changes its internal state from s to s' , and moves one step to the right.

in such a desultory manner that he never does more than one step at a sitting. The note of instructions must enable him to carry out one step and write the next note. Thus the state of progress of the computation at any stage is completely determined by the note of instructions and the symbols on the tape.

In other words, a finite procedure is, by definition, something where we can record our current state, and our progress so far, with a finite string of symbols. Note that Turing doesn't claim here that his machine is an adequate model for all of human cognition, let alone consciousness—that is a separate philosophical question. He simply claims that it can model the type of cognition we engage in when we carry out a computation according to a fixed procedure.

At the end of this distillation, we have the following mathematical model. A *Turing machine* has a “head” which can be in a finite number of internal states, and a “tape” where each square contains a symbol drawn from a finite alphabet. In each step of computation, it observes the symbol at its current location on the tape. Based on this symbol and its internal state, it then

1. updates the symbol at its current location on the tape,
2. updates its internal state, and
3. moves one step left or right on the tape.

If we let A denote the alphabet of symbols, and let S denote the set of internal states, we can write the machine's behavior as a transition function,

$$F : A \times S \rightarrow A \times S \times \{\pm 1\}.$$

For instance, $F(a, s) = (a', s', +1)$ would mean that if the machine sees the symbol a on the tape when it is in state s , then it changes the tape symbol to a' , changes its state to s' , and moves to the right. We show this in Figure 7.9.

We start the machine by writing the input string x on the tape, with a special blank symbol $_$ written on the rest of the tape stretching left and right to infinity. We place the head at one end of the input, and start it in some specific initial state. The machine uses the tape as its workspace, reading and writing symbols representing its intermediate results until the output is all that remains. At that point, it enters a special HALT state to announce that it's done.

Exercise 7.16 Write a Turing machine that computes the successor function $S(x) = x + 1$. In other words, let $A = \{0, 1, \sqcup\}$, and design a Turing machine that starts at the least significant bit of x , changes x to $x + 1$, ends up back at the least significant bit, and halts. Hint: let the set of states be $S = \{\text{CARRY}, \text{RETURN}, \text{HALT}\}$, and start it in the CARRY state.

We can think of a Turing machine's transition function as its “source code.” Moreover, we can write out a table of its values as a finite string of symbols, and thus feed one Turing machine to another. We can then ask whether there is a universal machine U , which takes a pair (M, x) as input—where M is the source code of a Turing machine, and x is an input string, written next to each other on U 's tape—and simulates M 's action on the input x .

Turing gave an explicit construction of just such a machine. It uses marker symbols to keep track of M 's simulated state and location, and runs over to M 's source code after each step to look up what transition it should perform next. If its simulation of M halts, then U returns $U(M, x) = M(x)$. Note that U can simulate Turing machines with an arbitrary number of states and tape symbols, even though it only has a fixed number of them itself. His original construction was fairly large, but since then clever people have found universal Turing machines with just a handful of states and symbols.

Turing then applied the diagonal argument to show that the Halting Problem is undecidable. Suppose there were a machine $H(M, x)$ that could tell, in finite time, whether another machine M , given input x , will ever halt. It would be easy to modify H , and make a machine $C_{22}(M)$ that halts if $M(M)$ doesn't halt, and enters an infinite loop if it does. Then if we fed C_{22} to itself, $C_{22}(C_{22})$ would halt if and only if it wouldn't, and we are forced to conclude that H does not exist. Analogous to Problem 7.4, virtually any question about the long-term behavior of a Turing machine is undecidable.

In the minds of most computer scientists, in 1936 and today, Turing succeeded in finding the definitive model of computation. One reason that his work is so convincing is that he did not proceed by inventing a particular formalism, such as the partial recursive functions or the λ -calculus, and arguing that certain kinds of recursion or string substitution are sufficient. Instead, he arrived at his machine by contemplating the process of computation in general, with what Emil Post—who independently invented a very similar model—called “psychological fidelity.” Once we accept that a computation is something that we carry out with pencil and paper, by reading and writing symbols until we arrive at the result, it becomes clear that the Turing machine leaves nothing out. As Gödel said in 1946,

Tarski has stressed in his lecture (and I think justly) the great importance of the concept of general recursiveness (or Turing's computability). It seems to me that this importance is largely due to the fact that with this concept one has for the first time succeeded in giving an absolute definition of an interesting epistemological notion, i.e., one not depending on the formalism chosen.

Turing's model was also the first in which a universal function can be defined concretely and easily, without the technical hurdle of Gödel numbers or the abstraction of the λ -calculus. Gödel referred to the ease with which Turing machines can simulate each other—and, therefore, the ease with which the diagonal argument can be applied—as “a kind of miracle.”

The Turing machine is also extremely robust to changes in its definition. If we try to make it more elaborate and powerful, we find that it can always be simulated by the original model. Multiple heads? Represent their positions with additional symbols on the tape, and have a single head shuttle back and forth between them. Multiple tapes? Interleave them, or combine their symbols in a larger alphabet, and



7.15

write them all on one tape. A two-dimensional grid, where the head can move north, south, east, or west? Draw a spiral on the grid and unroll it into a one-dimensional tape, and so on. There are certainly things we can do to the Turing machine to cripple it, such as forcing it to move to the right on every step. But there is nothing we can do to make it stronger.

Finally, as we will see next, Turing's model of computation is equivalent to the partial recursive functions and the λ -calculus. Thus both psychological fidelity and mathematical unity suggest that it is the right definition.



7.16

Exercise 7.17 Define a version of the Turing machine that, in addition to moving left and right, can insert and delete squares on the tape. Sketch a proof that such a machine can be simulated by a standard Turing machine.

Exercise 7.18 A one-sided Turing machine is one that can never move to the left of its initial position, so that its tape stretches from 0 to $+\infty$ rather than from $-\infty$ to $+\infty$. Sketch a proof that such a machine can simulate a standard Turing machine. Hint: fold the tape in half.

Exercise 7.19 Sketch a proof that two tape symbols suffice—in other words, that any Turing machine can be simulated by one where $A = \{0, 1\}$, possibly with a larger number of states.

Exercise 7.20 Show that BLANK TAPE HALTING, the problem of whether a given Turing machine will halt if started on a blank tape, is undecidable.

Exercise 7.21 The busy beaver function $BB(n, m)$ is the maximum, over all Turing machines with n states and m tape symbols that halt when given a blank tape as input, of the number of steps it takes them to do so. Show that $BB(n, m)$ grows faster, as a function of n and m , than any computable function. Hint: compare to Exercise 7.9.



7.17

7.5.2 The Grand Unification

All the prophets of God proclaim the same faith.

Baha'u'llah

As we discussed in Sections 7.3 and 7.4, the partial recursive functions and the λ -calculus are equivalent in their computational power—a function is partial recursive if and only if it is λ -definable. Let's complete the triumvirate by showing that they are equivalent to Turing machines as well. In what follows, we say that a partial function $f(x)$ is *Turing-computable* if there is a Turing machine that, given input x , halts with $f(x)$ on its tape if $f(x)$ is defined, and runs forever if $f(x)$ is undefined.

First, we sketch a proof that Turing-computable functions are partial recursive. There is a simple way to “arithmetize” the tape—namely, break it into two halves and convert each half to an integer, with the least significant digit closest to the head. Using the notation of Figure 7.9, this gives

$$B = \sum_{i=0}^{\infty} 2^i b_i \quad \text{and} \quad C = \sum_{i=0}^{\infty} 2^i c_i, \quad (7.24)$$

where we assume for simplicity that the tape alphabet is $A = \{0, 1\}$, and that 0 also acts as the blank symbol. Note that B and C are finite, since all but a finite number of squares on the tape are blank. We represent the tape symbol at the head's current position as an additional integer a .

With this representation, it's easy to carry out a step of the Turing machine's computation. For instance, to change the tape symbol from a to a' and move one step to the right as in Figure 7.9, we update B , C , and a as follows:

$$\begin{aligned} B &\rightarrow 2B + a' \\ C &\rightarrow \lfloor C/2 \rfloor \\ a &\rightarrow C \bmod 2. \end{aligned} \tag{7.25}$$

We showed in Exercise 7.10 that all of these functions are primitive recursive. The internal state s is just another integer, and we can update it—as well as determine a' and what direction to move the head—by checking a finite number of cases.

Thus we claim that, for any given Turing machine M , there is a primitive recursive function `step` which performs a single step of M 's computation, updating the vector of variables (B, C, a, s) . We give M an input x by setting, say, $B = x$ and $C = a = s = 0$. Then, since primitive recursion allows us to iterate the `step` function t times, the state (B, C, a, s) of the machine after t steps is a primitive recursive function of x and t . Finally, we can use μ -recursion to find the t , if there is one, such that M enters the `HALT` state on its t th step, and return its state at that point. This shows that the partial function computed by M is partial recursive.

Conversely, recall that the partial recursive functions are those that can be defined from zero and the successor function $S(x) = x + 1$ using composition, primitive recursion, and μ -recursion. Turing machines can certainly compute zero, and Exercise 7.16 shows that they can compute $S(x)$. We just need to show that the Turing-computable functions are closed under these three ways of defining new functions from old ones. For composition, for instance, we need to show that if $f(x)$ and $g(x)$ are Turing-computable, then so is $f(g(x))$. But this is easy—simply combine the two machines into one by taking the union of their sets of states, and identify the `HALT` state of g 's machine with the initial state of f 's.

Similarly, there are Turing machines that compute functions formed from f and g by primitive recursion or μ -recursion. The details are tedious, but the ideas are simple—the machine applies these functions repeatedly, while incrementing or decrementing counters to keep track of its work. If a function $h(\mathbf{x})$ defined by μ -recursion is undefined because $f(\mathbf{x}, y) \neq 0$ for all y , then the machine runs forever, trying larger and larger values of y , just as the corresponding `FLOOP` program would. Thus all partial recursive functions are Turing-computable.

As an alternate proof, it is not hard to see that Turing machines can scan through a string for some symbol, and substitute another string at each place where that symbol appears. Thus they can simulate reductions in the λ -calculus, halting if they reach normal form. Conversely, the λ -calculus can represent the two halves of a Turing machine's tape as strings. With a little work we can represent the `step` function as a combinator, which reaches a normal form if the machine halts. Thus a function is Turing-computable if and only if it is λ -definable.

7.5.3 The Church–Turing Thesis

We have seen that all our definitions of computability are equivalent:

$$\text{partial recursive} = \lambda\text{-definable} = \text{Turing-computable}.$$

For good measure, we can add

= programmable in FLOOP, or your favorite programming language.

This brings us to the philosophical stance that has structured computer science ever since 1936. The *Church–Turing Thesis* states that Turing computability, in any of its equivalent forms, is the right definition of computability—that it captures anything that could reasonably be called a procedure, or a computation, or an algorithm. The universal Turing machine is capable of simulating, not just any Turing machine, but any computing device that can be finitely described, where each step accesses and modifies a finite amount of information. In the words of John von Neumann:

[A Turing machine] is able to imitate any automaton, even a much more complicated one... if it has reached a certain minimum level of complexity. In other words, a simpler thing will never perform certain operations, no matter what instructions you give it; but there is a very definite finite point where an automaton of this complexity can, when given suitable instructions, do anything that can be done by automata at all.

The Church–Turing Thesis allows us to think intuitively about computation, without worrying unduly about the details of our model. In a world of bounded resources, such as polynomial time, there seem to be differences between what, say, classical and quantum computers can do. But in the unbounded world of computability and decidability, these differences disappear completely.



7.18

The Thesis has another consequence as well. If Turing machines can carry out any computation, so can any system that can simulate a Turing machine. This suggests that there is a large class of systems that are *computationally universal*, and that each one can simulate all the others—just as any NP-complete problem can simulate, or express, every other one. In the next section, we will see just how widespread this phenomenon is.

7.6 Computation Everywhere

Computers made of water, wind and wood... may be bubbling, sighing or quietly growing without our suspicion that such activities are tantamount to a turmoil of computation whose best description is itself.

A. K. Dewdney *Computer Recreations*

If computation is software, it can run on many different kinds of hardware—any physical or mathematical system whose interactions allow it to store and process information in sufficiently complicated ways. In this section, we will see that many surprisingly simple systems are computationally universal, and therefore that even simple questions about these systems are undecidable.

If anything, computational universality, and undecidability, are the rule rather than the exception. Once we start looking for them, we see them everywhere.

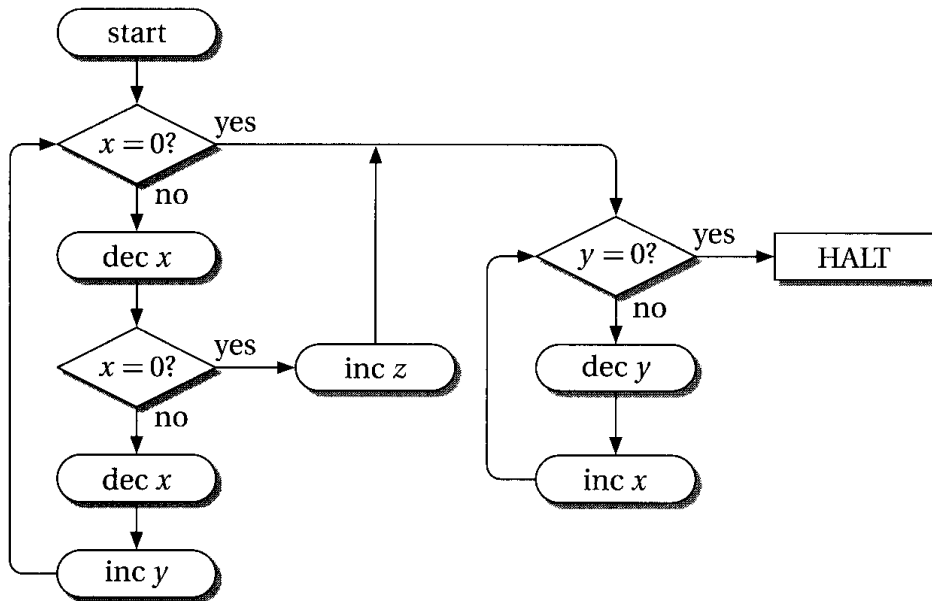


FIGURE 7.10: A counter machine with three counters. Given an initial value of x , where $y = z = 0$, it sets $x \rightarrow \lfloor x/2 \rfloor$ and $z \rightarrow x \bmod 2$. It uses y as temporary storage for $\lfloor x/2 \rfloor$. Each internal state of the machine, or line of the program, corresponds to a node in the flowchart.

7.6.1 Computing With Counters

“Can you do Addition?” the White Queen asked. “What’s one and one and one and one and one and one and one and one and one and one and one?”

“I don’t know,” said Alice. “I lost count.”

Lewis Carroll, *Through The Looking-Glass*

If a machine only has a finite number of states, with no access to anything but its own thoughts, then it is doomed to halt or fall into a periodic loop after a finite number of steps. But if we give it access to a memory with an infinite number of possible states, then it is capable of universal computation. Turing showed that this is the case even if this memory has a simple structure, and the machine can only observe and modify it through a very narrow window—namely, the symbol at its current location on the tape. Can we make the memory even simpler, and this window even narrower?

A *counter machine* has a finite number of internal states, and access to a finite number of integer counters. The only thing it can do to these counters is increment or decrement them, and the only question it can ask about them is whether or not they are zero. The machine receives its input through the initial state of one of these counters, and like the Turing machine, it enters a **HALT** state when it is done.

We can think of a counter machine as a flowchart or a program in a miniature programming language. Each internal state corresponds to a node in the flowchart or a line of the program, and the only allowed instructions are **inc** (increment), **dec** (decrement), and conditionals like **if $x = 0$** . For instance, the flowchart in Figure 7.10 uses three counters, x , y , and z . Assuming that their initial values are $(x, 0, 0)$,

it transforms them to $(\lfloor x/2 \rfloor, 0, x \bmod 2)$. It uses y as temporary storage for $\lfloor x/2 \rfloor$, first incrementing y for every two times it decrements x , and then incrementing x and decrementing y at the same rate. Similarly, we can compute $2x$ by incrementing y twice each time we decrement x , and then feeding y back into x .

But if we can compute $\lfloor x/2 \rfloor$, $2x$, and $x \bmod 2$, we can simulate a Turing machine arithmetically as in Section 7.5.2. Specifically, we can transform any Turing machine into a counter machine with just three counters: one for each half of the tape, B and C as in (7.25), and a temporary one in which we store, say, $2B$ or $\lfloor C/2 \rfloor$ before feeding it back into B and C . We include the current tape symbol, and the head's internal state, as part of the internal state of the counter machine.

Thus the Halting Problem for three-counter machines is undecidable. Moreover, three-counter machines can compute any partial recursive function f , by starting with x in one counter and ending with $f(x)$ in that counter when they halt.

Exercise 7.22 Design a counter machine with two different HALT states, called YES and NO, such that it ends in one or the other depending on whether x is a power of 2, and such that the final value of another counter is $\lfloor \log_2 x \rfloor$.

We can reduce the number of counters even further. First, let's consider *multiplicative* counter machines, where each step can multiply or divide a counter by a constant, or check to see whether a counter is a multiple of some constant. Now even a *single* counter suffices. This is because we can encode three additive counters as a single multiplicative one:

$$w = 2^x 3^y 5^z.$$

We can increment or decrement y , say, by multiplying or dividing w by 3. Similarly, we can test whether $y \neq 0$ by asking whether w is a multiple of 3.

But a one-counter multiplicative machine, in turn, can be simulated by a two-counter additive one. If we have an additional counter u , we can multiply or divide w by 2, 3, or 5, or check to see whether w is a multiple of any of these, using loops similar to that in Figure 7.10. To divide w by 3, for instance, we decrement w three times for each time we increment u . If w becomes zero partway through a group of three decrements, we know that w is not a multiple of 3.

Thus even two counters suffice for universal computation. Specifically, for any partial recursive function f , there is a two-counter machine which, given the initial value $w = 2^x$, ends with $w = 3^{f(x)}$, or runs forever if $f(x)$ is undefined.

For yet another simple but universal automaton, consider a *two-dimensional finite automaton*, which wanders on the northeast quadrant of the infinite plane (Figure 7.11). It has a finite number of states, and at each step it moves north, south, east, or west. It cannot write anything on the squares it touches, and it cannot see. All it knows about its position is whether it is touching the southern or western edge. But this is a two-counter machine in disguise, whose counters are its x and y coordinates. Therefore, it is undecidable to tell whether such an automaton will ever halt—or ever touch the origin—or just wander off to infinity.

It's worth noting that these counter machines are extremely inefficient. If there are n symbols on the tape, it takes the three-counter machine about 2^n steps to move the head left or right. The two-counter machine is even worse, taking as many as 5^{2^n} steps. Thus an algorithm that a Turing machine can run in polynomial time could take doubly-exponential time. But where computational universality

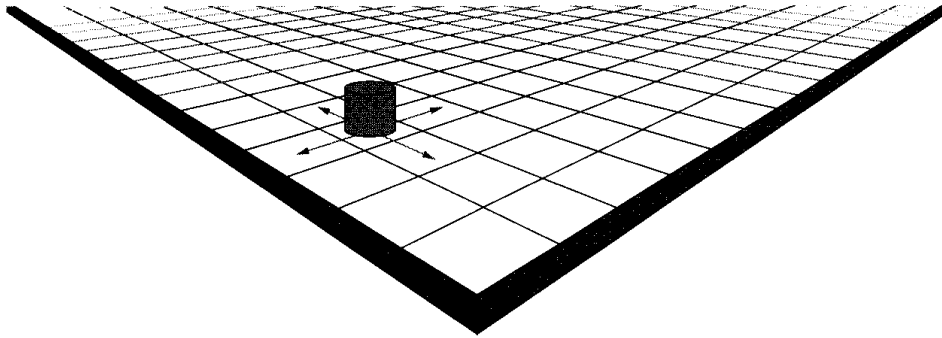


FIGURE 7.11: This automaton moves blindly on a quadrant of the infinite plane. It can only sense whether it is touching one of the boundaries, and it only has a finite number of internal states—and yet it is computationally universal.

is concerned, all that matters is that these machines can carry out any computation if we give them long enough. What's exponential time when you have eternity?

Exercise 7.23 Argue that any counter machine with a constant number of counters needs exponential time to simulate a Turing machine. Hint: how long does it take to read or write n bits of information from or to an integer counter?

7.6.2 Fantastic Fractions

If computers can come in many forms, then so can programs. Here is a program written in a rather odd programming language called `FRACTRAN`, consisting of nothing more than a list of fractions:

$$\frac{17}{91} \quad \frac{78}{85} \quad \frac{19}{51} \quad \frac{23}{38} \quad \frac{29}{33} \quad \frac{77}{29} \quad \frac{95}{23} \quad \frac{77}{19} \quad \frac{1}{17} \quad \frac{11}{13} \quad \frac{13}{11} \quad \frac{15}{14} \quad \frac{15}{2} \quad \frac{55}{1}$$

The state of the computer consists of an integer n . At each step, we run through the list until we find the first fraction p/q such that $(p/q)n$ is an integer—that is, the first one whose denominator q divides n . Then we multiply n by p/q , and start again at the beginning of the list. If there is no such p/q in the list, the program halts.

This particular program never halts, but it does something rather surprising. If we start it with the initial value $n = 2$, it produces the sequence

$$2, \frac{15}{2} \times 2 = 15, \frac{55}{1} \times 15 = 825, \frac{29}{33} \times 825 = 725 \dots,$$

until, after 19 steps, $n = 4 = 2^2$. After 50 more steps, $n = 2^3$, and 211 steps after that, $n = 2^5$. The steps where n is a power of 2 are shown in Table 7.1. As you can see, this program produces the prime numbers x , in increasing order, in the form 2^x .

step	19	69	280	707	2363	3876	8068	11319	...
n	2^2	2^3	2^5	2^7	2^{11}	2^{13}	2^{17}	2^{19}	...

TABLE 7.1: Prime powers of two, generated by a FRACTRAN program.

At first glance, this is extremely mysterious. But given the previous section, we know that this program is a kind of multiplicative counter machine. In this case, we can write the state in the form

$$n = 2^a 3^b 5^c 7^d 11^e 13^f 17^g 19^h 23^i 29^j,$$

since these are the primes appearing in the fractions' denominators. Thus we have 10 counters, which we can increment and decrement by multiplying n by fractions. For instance, the first fraction in the program is $17/91 = 17/(7 \times 13)$. So, if both d and f are nonzero, we decrement them and increment g . If either $d = 0$ or $f = 0$, then we move on to $78/85 = 78/(5 \times 17)$ and check to see if both c and g are nonzero, and so on.

We already know that additive counter machines can simulate Turing machines. To convert such a machine into a FRACTRAN program, we just need to do a little extra work. First, we modify the machine so that whenever it checks to see if a counter is zero, it decrements it if it is nonzero. We can always increment the counter afterward with an additional state.

We need to express the machine's internal state as part of the integer n . We can do this by associating each state s with an additional prime p_s . Then, for each kind of step that we could take from the state s , we include p_s in the denominator of the corresponding fraction, and include $p_{s'}$ for the new state s' in the numerator. Since the numerator and denominator must be relatively prime, we need to ensure that $s' \neq s$. We can do this by adding more states, if necessary replacing a self-loop with a pair of states transitioning to each other. We ask the reader to work out some examples in Problem 7.32 and 7.33.

Thus, like its namesake FORTRAN, FRACTRAN is computationally universal. There are FRACTRAN problems that simulate other FRACTRAN programs, or counter machines, or Turing machines. Its Halting Problem is undecidable, and for any computable function $f(x)$ there is a program that, if given 2^x as input, returns $3^{f(x)}$ as output whenever $f(x)$ is well-defined.

Exercise 7.24 Use FRACTRAN to prove that any partial computable function $f(x)$ can be written in the form

$$f(x) = g(\mu_y f(x, y)),$$

where f and g are primitive recursive. See also Exercise 7.12 on page 249.

FRACTRAN might seem like a gimmick, but it is related to deep problems in number theory. Consider the function

$$g(x) = \begin{cases} x/2 & \text{if } x \text{ is even} \\ 3x+1 & \text{if } x \text{ is odd.} \end{cases} \quad (7.26)$$

The *Collatz conjecture*, also known as the $3x+1$ problem, states that the sequence

$$x, g(x), g^2(x) = g(g(x)), \dots$$

ends in the cycle $4, 2, 1, 4, 2, 1, \dots$, no matter what the initial value of x is. This conjecture has been verified for all x up to 10^{18} , and there are convincing heuristic arguments for it. However, there is still no proof,

and resolving it seems quite difficult. According to the great mathematician Paul Erdős, “mathematics is not yet ready for such problems.”

The $3x + 1$ function belongs to a family of functions whose behavior depends on $x \bmod q$ for some q :

$$g(x) = \frac{a_i}{q}(x - i) + b_i \quad \text{where} \quad x \equiv_q i. \quad (7.27)$$

Here a_i and b_i are integer coefficients for $0 \leq i < q$. Note that $g(x)$ is an integer for any integer x . For any function of this form, we can ask whether a given initial x will reach some y after some number of iterations:

COLLATZ

Input: Integers x , y , and q , and integer coefficients a_i, b_i for $0 \leq i < q$

Question: Is there an integer t such that $g^t(x) = y$?

Given the current state n of a FRACTRAN program, let $g(n)$ denote its state after the next step. If Q is the lowest common multiple of the program's denominators, it is easy to see that $g(n) = an$ where a only depends on $n \bmod Q$. Thus we can express a single step of a FRACTRAN program as a Collatz function—indeed one where $b_i = 0$ for all i .

Since we can arrange for the FRACTRAN program to arrive a particular value y if it halts, we can convert its Halting Program to an instance of COLLATZ. This gives us the reduction

$$\text{HALTING} \leq \text{COLLATZ},$$

and proves that COLLATZ is undecidable.

This does not show that the original $3x + 1$ problem is undecidable. But it does show that there is no way of addressing it that is general enough to solve all problems of this form. It is tempting to believe that this is part of the reason why this particular one is so difficult.



7.19

7.6.3 The Game of Tag

In 1921, the young logician Emil Post was studying the axioms and rules of inference in *Principia Mathematica*, the system of mathematical logic proposed by Bertrand Russell and Alfred North Whitehead. His idea was to approach formal systems abstractly as string-rewriting systems, not unlike the λ -calculus that Church would invent a decade later. Post was one of the first to think of formal systems in a combinatorial sense, as sets of strings produced by formal rules, regardless of the meaning that we assign to them.

Post showed, remarkably, that the axioms of the *Principia* can be reduced to a single initial string, and that its rules of inference can be replaced with rules of the following *normal form*:

$$gs \Rightarrow sh,$$

where g and h are fixed strings and s is arbitrary. In other words, for any string that begins with g , we can remove g from its head and append h to its tail. This is a beautiful early example of a proof that a simple system is universal, and capable of simulating much more general systems—even though Post thought of these as formal systems, rather than models of computation.

At first, Post hoped to solve the *Entscheidungsproblem* by finding an algorithm to determine whether a given string can be produced from an initial string, representing the system's axioms, using a given set of rules $\{(g_i, h_i)\}$ in normal form. We can think of the production process as a computation, where each step consists of applying one of the rules. In general, this computation might be nondeterministic, since we might have a choice of which rule to apply to a given string. For instance, with the rules $\{(a, b), (ab, c)\}$, we could change abc to either bcb or cc .

As a warm-up, Post considered the case where all the g_i have the same length ν for some constant ν , and where each h_i is determined by g_i 's first symbol a . Thus, at each step, if a string begins with the symbol a , we remove its first ν symbols including a , and append the string $h(a)$. This process is deterministic, so each initial string leads to a fixed sequence. If this sequence reaches a string whose length is less than ν , it halts.

These are called *tag systems*, and Post initially expected them to be an easy special case. However, he found that even tag systems possess, in his words, "bewildering complexity." To get a sense of how complicated they can be, consider the following tag system, where $\nu = 2$ and the alphabet is $\{a, b, c\}$:

$$\begin{aligned} h(a) &= bc \\ h(b) &= a \\ h(c) &= aaaa. \end{aligned}$$

Let's start with the initial string $a^3 = aaa$ and see what happens.

$$\begin{aligned} &aaa \\ &\quad abc \\ &\quad\quad cbc \\ &\quad\quad\quad caaaa \\ &\quad\quad\quad\quad aaaaaa = a^5. \end{aligned}$$

The string seems to be growing, and six iterations later, it becomes a^8 . But then it becomes, in short order, a^4 , a^2 , and then a . At that point its length is less than $\nu = 2$, so it halts.

After exploring a few examples, the reader might ask whether the initial string a^x eventually halts for all $x \geq 1$. We agree that this is an excellent question, especially considering the following exercise.

Exercise 7.25 Show that this tag system maps a^x to $a^{g(x)}$, where

$$g(x) = \begin{cases} x/2 & \text{if } x \text{ is even} \\ (3x+1)/2 & \text{if } x \text{ is odd.} \end{cases} \quad (7.28)$$

This is the $3x+1$ function from (7.26), except that it performs an extra iteration if x is odd. Thus the question of whether a^x will halt is exactly the question of whether x will ever reach the cycle $\{4, 2, 1\}$ in the $3x+1$ problem.

As Problem 7.34 shows, it is not too hard to convert any Collatz function of the form (7.27) into a tag system, and thus reduce COLLATZ to the problem of telling whether one string in a tag system will lead to another. We can construct a similar reduction for the problem of whether the string will shrink to a length less than ν and halt. Thus the following problem is undecidable:

TAG SYSTEM HALTING

Input: An integer ν , a finite alphabet A , a string $h(a)$ for each $a \in A$, and an initial string x

Question: Does x lead to a string of length less than ν ?

Since tag systems are a special case of normal form, the Halting Problem for systems in normal form is undecidable as well:

NORMAL FORM HALTING

Input: A list of pairs of strings $\{(g, h)\}$ and an initial string x

Question: By starting with x and applying the rules $gs \Rightarrow sh$, is it possible to derive a string y to which no rule can be applied, i.e., which does not begin with any g in the list?

We have shown that this is undecidable even for systems which are deterministic, or as Post called them, *monogenic*—that is, where only one sequence of strings can be derived from the initial string.

Exercise 7.26 Show that, given a system in normal form, an initial string x , and a string y , it is undecidable tell whether or not y can be derived from x .

We can use tag systems to prove that another seemingly simple problem is undecidable. In Post's *correspondence problem*, we have a list of pairs of strings,

$$\left\{ \begin{pmatrix} s_1 \\ t_1 \end{pmatrix}, \begin{pmatrix} s_2 \\ t_2 \end{pmatrix}, \dots, \begin{pmatrix} s_k \\ t_k \end{pmatrix} \right\}.$$

We then ask whether there is any finite, nonempty string that can be written both as the concatenation of some sequence of s_i and that of the corresponding t_i , where a given i can appear any number of times in this sequence. For instance, if we have

$$\begin{pmatrix} s_1 \\ t_1 \end{pmatrix} = \begin{pmatrix} a \\ abab \end{pmatrix}, \begin{pmatrix} s_2 \\ t_2 \end{pmatrix} = \begin{pmatrix} bab \\ ba \end{pmatrix}, \begin{pmatrix} s_3 \\ t_3 \end{pmatrix} = \begin{pmatrix} aa \\ a \end{pmatrix},$$

then the answer is “yes,” since

$$s_1 s_2 s_2 s_3 = t_1 t_2 t_2 t_3 = ababbabaa.$$

Formally, we state this problem as follows.

CORRESPONDENCE

Input: A list of pairs of strings $\{\begin{pmatrix} s_1 \\ t_1 \end{pmatrix}, \begin{pmatrix} s_2 \\ t_2 \end{pmatrix}, \dots, \begin{pmatrix} s_k \\ t_k \end{pmatrix}\}$

Question: Is there a finite sequence $i_1, i_2, \dots, i_\ell$ with $\ell \geq 1$ such that $s_{i_1} s_{i_2} \dots s_{i_\ell} = t_{i_1} t_{i_2} \dots t_{i_\ell}$?

We will prove that CORRESPONDENCE is undecidable by reducing TAG SYSTEM HALTING to it. Given a tag system, we add three symbols to its alphabet, $\langle$, $|$, and $\rangle$. If the initial string is $x = x_1 x_2 \cdots x_m$, we include the pair

$$\begin{pmatrix} s_x \\ t_x \end{pmatrix} = \begin{pmatrix} \langle \\ \langle x_1 | x_2 | \cdots | x_m \rangle \end{pmatrix}.$$

Then, for each string $g = g_1 g_2 \cdots g_\nu$ of length ν , if $h(g_1) = h_1 h_2 \cdots h_n$, we include the pair

$$\begin{pmatrix} s_g \\ t_g \end{pmatrix} = \begin{pmatrix} g_1 | g_2 | \cdots | g_\nu | \\ | h_1 | h_2 | \cdots | h_n \end{pmatrix}.$$

Finally, for each string $y = y_1 y_2 \cdots y_t$ of length $t < \nu$, we include the pair

$$\begin{pmatrix} s_y \\ t_y \end{pmatrix} = \begin{pmatrix} y_1 | y_2 | \cdots | y_t \rangle \\ \rangle \end{pmatrix}.$$

The pairs $\begin{pmatrix} s_x \\ t_x \end{pmatrix}$ and $\begin{pmatrix} s_y \\ t_y \end{pmatrix}$ force us to start with x and end with y , and each pair $\begin{pmatrix} s_g \\ t_g \end{pmatrix}$ forces us to carry out one step of the tag system. For instance, if the initial string is $x = aaaa$, the $3x + 1$ tag system described above gives the pairs

$$\begin{pmatrix} \langle \\ \langle a | a | a | a \rangle \end{pmatrix}, \begin{pmatrix} a | a | \\ | b | c \end{pmatrix}, \begin{pmatrix} a | b | \\ | b | c \end{pmatrix}, \begin{pmatrix} b | c | \\ | a \end{pmatrix}, \begin{pmatrix} c | a | \\ | a | a | a \end{pmatrix}, \begin{pmatrix} c | b | \\ | a | a | a \end{pmatrix}, \begin{pmatrix} a \rangle \\ \rangle \end{pmatrix},$$

where we exclude pairs corresponding to strings g that never occur. The tag system halts after the computation

$$\begin{array}{c} aaaa \\ aabc \\ bcbc \\ bca \\ aa \\ bc \\ a, \end{array}$$

which corresponds to the string

$$\begin{aligned} w &= \langle a | a | a | a | b | c | b | c | a | a | b | c | a \rangle \\ &= s_x s_{aa} s_{aa} s_{bc} s_{bc} s_{aa} s_{bc} s_a \\ &= t_x t_{aa} t_{aa} t_{bc} t_{bc} t_{aa} t_{bc} t_a. \end{aligned}$$

Any finite string that can be written as a concatenation of these pairs is either w , or w repeated some number of times. Thus x halts if and only if a corresponding string exists. This completes the reduction

$$\text{TAG SYSTEM HALTING} \leq \text{CORRESPONDENCE},$$

and proves that CORRESPONDENCE is undecidable.



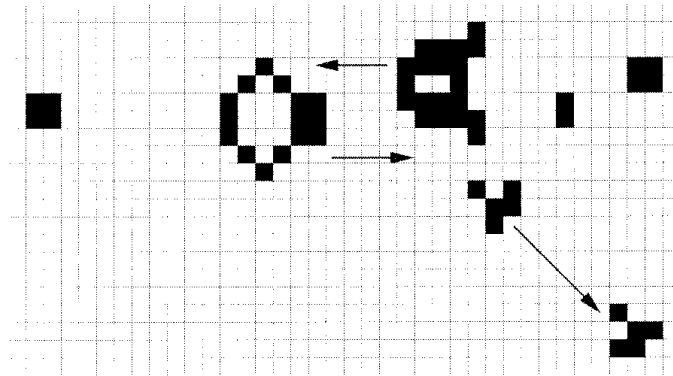


FIGURE 7.12: The glider gun, the first pattern found in Life where the number of live cells increases without bound. Its two central components shuttle back and forth, producing a glider every 30 steps.

7.6.4 The Meaning of Life

In Section 5.5.3, we encountered *cellular automata*—discrete universes whose laws of physics update each cell of a lattice simultaneously. Like our universe, where signals are limited by the speed of light, their dynamics is *local*. That is, the new state of each cell depends only on its own current state, and those of its neighbors. The physics of our universe is rich enough to build universal computers. Is this true of these toy universes as well?

One famous rule in which we can build computers is Conway’s “Game of Life.” In Figure 5.21 on page 159, we saw a *glider*—an elementary particle that travels one step diagonally every four time steps. There is also a *glider gun*, shown in Figure 7.12. It produces a glider every 30 steps, emitting a steady stream of them into the vacuum.

Depending on its precise timing, a collision between two gliders can make one of them bounce back the way it came, or annihilate both of them. We can use these collisions to perform Boolean operations, where we represent `true` and `false` as glider streams which are turned on or off. For instance, Figure 7.13 shows a NOT gate, where we intersect a stream A with the stream from a glider gun in a way that annihilates both. This produces a stream $\bar{A}$ which is on if and only if A is off.

Exercise 7.27 By drawing a diagram in the spirit of Figure 7.13, show how to convert two glider streams A and B to Boolean combinations such as $A \vee B$ and $A \wedge B$, using glider guns and collisions that annihilate both streams if they are present. Can you use a similar approach to perform fanout, i.e., to copy the truth value of a stream to two others, or do you need another kind of collision to do this?

Gliders are just the beginning. A dedicated community of “Life hackers” has invented oscillators, spaceships, puffer trains, buckaroos, breeders, switches, latches, adders, and memories. By putting these and other components together, it is possible to build an entire Turing machine in the Game of Life—a finite state control which sends and receives signals from a tape, updating its symbols and shifting it left and right. We show this astonishing construction in Figure 7.14.

In order to perform universal computation, this Turing machine must be equipped with an infinite tape, stretching to the horizon in both directions. Its initial state can be described with a finite number of bits, since a blank tape consists of a finite unit repeated periodically. However, since we are used to

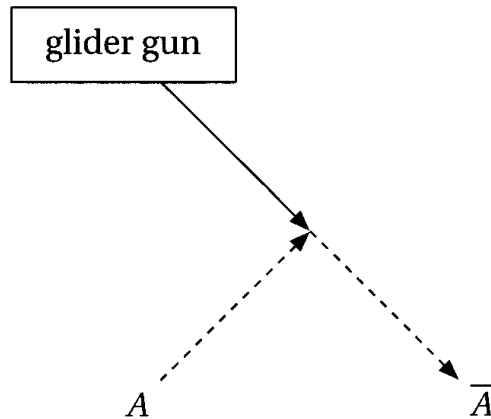


FIGURE 7.13: A schematic of a NOT gate in Life. The two streams of gliders annihilate if they are both turned on, so $\bar{A}$ is on if and only if A is off.

computers of finite physical extent, we might prefer a computer whose initial state consists of only a finite number of “live” cells, surrounded entirely by zeros.

There are two possible ways to do this. One is to design a flotilla of spaceships that move out into the lattice, firing gliders back toward the tape that collide and combine to form new tape cells. While this construction has not yet been completed, it seems perfectly possible, since glider collisions have been designed to build nearly anything.

Another type of universal computer, which can be built in Life with a finite number of live cells, is the counter machine discussed in Section 7.6.1. We can represent the value of each counter as the position of a stable 2×2 block. We can then push and pull these blocks, incrementing and decrementing the counter, by sending carefully timed sets of gliders at them.

In either of these constructions, we can arrange for a particular cell to turn on if and only if the Turing machine, or the counter machine, enters its HALT state. Therefore, the following problem is undecidable:

LIFE PREDICTION

Input: An initial configuration C and integers x, y

Question: If we start with C , will the cell at (x, y) ever turn on?

Thus the eventual fate of an initial configuration in Life is impossible to predict.

7.21

The Game of Life shows that we can perform universal computation in a simple two-dimensional universe. Is even one dimension enough? Turing machines are one-dimensional objects, and since the head only moves one step at a time, their dynamics is already local. So, designing one-dimensional CAs that simulate Turing machines is easy. One method is to combine the internal state with the tape symbol at the head's current location. If the machine has tape symbols A and internal states S , this gives the CA the set of states $Q = A \cup (A \times S)$. We then define the transition function f of the CA,

$$q_{x,t+1} = f(q_{x-1,t}, q_{x,t}, q_{x+1,t}),$$

so that it simulates the Turing machine at each step.

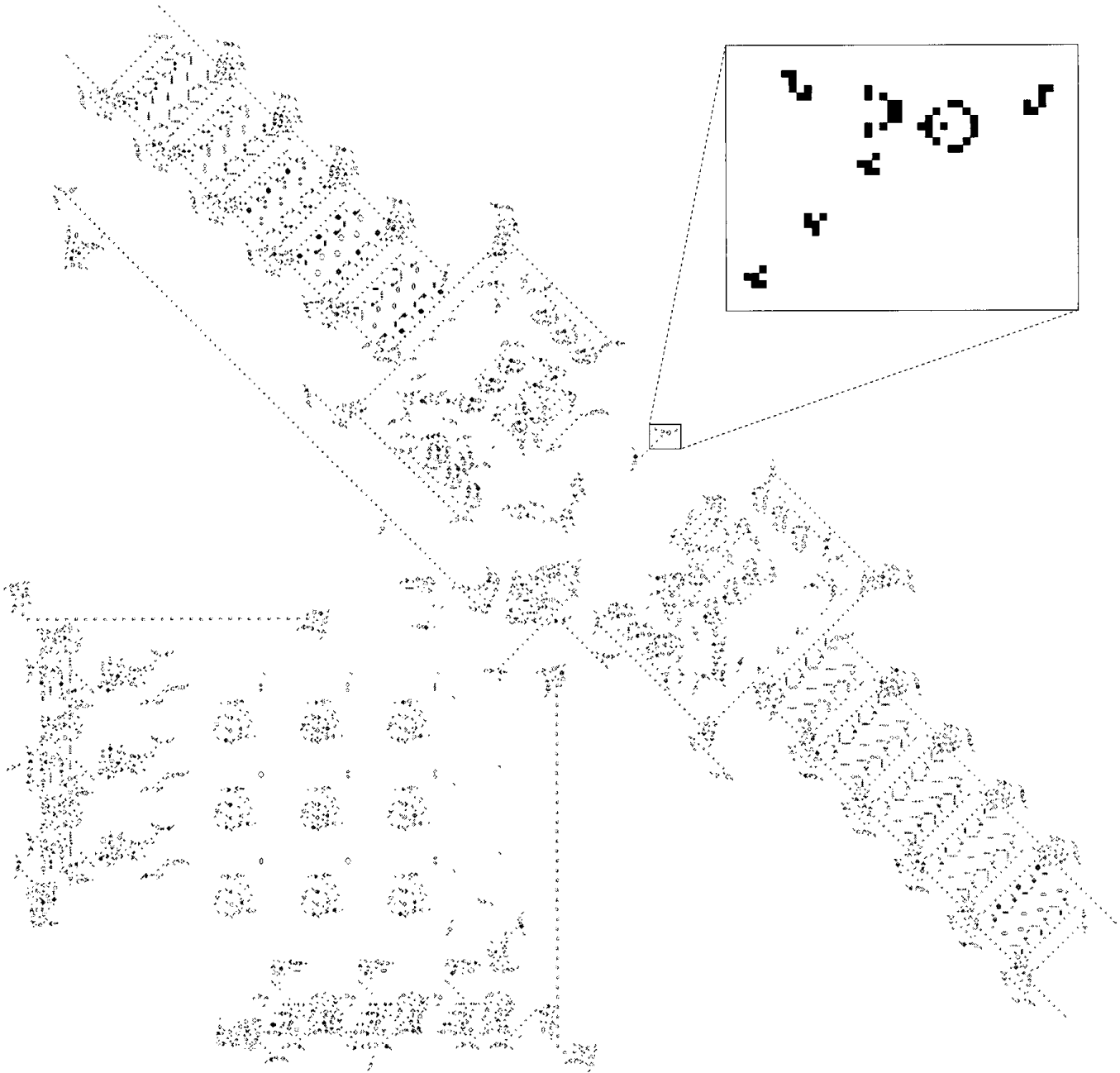


FIGURE 7.14: Constructing a Turing machine in the Game of Life. The head with its finite-state control is on the lower left, and the stack stretches from upper left to lower right. When animated, it is a truly impressive sight. We magnify one of its components, a glider gun, to give a sense of its scale.

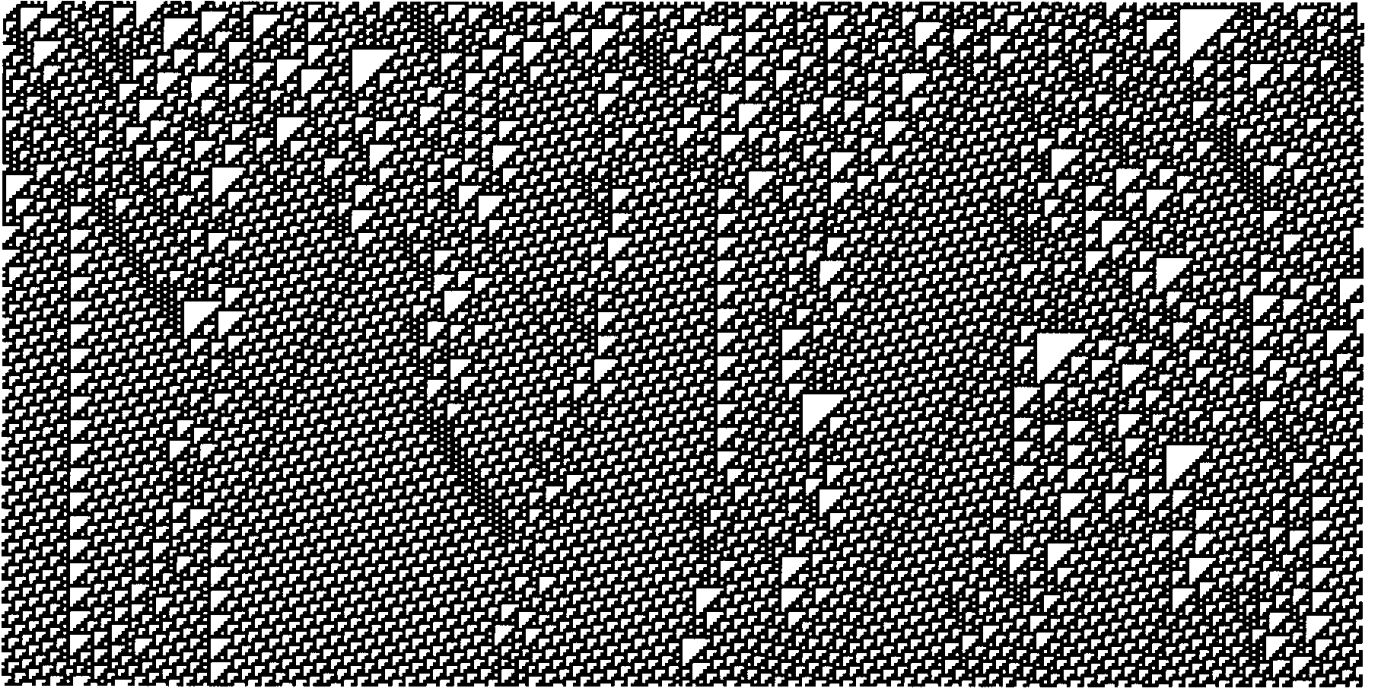


FIGURE 7.15: The evolution of elementary CA rule 110 from a random initial state, showing collisions between several types of gliders and periodic structures. Time increases downward, and each row of the diagram is a single step.

For instance, suppose that when the Turing machine is in state s and the tape symbol is b , it writes b' on the tape, changes its state to s' , and moves to the right. Then we would write, for any $a, c, d \in A$,

$$f(a, (b, s), c) = b' \quad \text{and} \quad f((b, s), c, d) = (c, s').$$

We also define $f(a, b, c) = b$ for any $a, b, c \in A$, since the tape symbols stay fixed if the head isn't around to modify them. Then one step of this CA would look like

...	a	(b, s)	c	d	...
...	a	b'	(c, s')	d	...

moving the head to the right and updating the tape. Of course, the parallel nature of a cellular automaton allows us to simulate many Turing machine heads simultaneously. We haven't specified here what happens when two heads collide.

If we start with a small universal Turing machine, i.e., one with a small number of states and symbols, this construction gives a universal CA with a fairly small number of states. But what is more surprising is that even “elementary” CAs—those with just two states and nearest-neighbor interactions—are capable of universal computation. Consider the following rule:

111	110	101	100	011	010	001	000
0	1	1	0	1	1	1	0

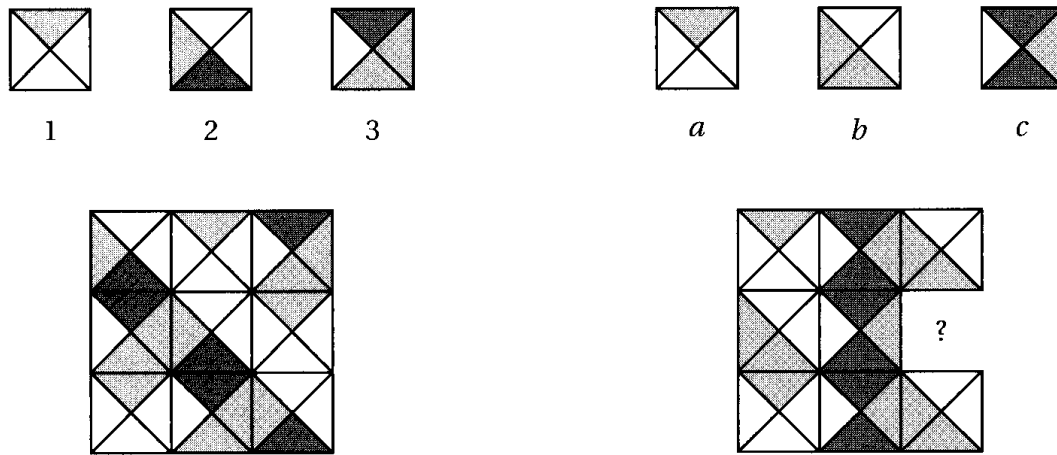


FIGURE 7.16: The set $\{1, 2, 3\}$ of Wang tiles allows us to tile the infinite plane, but the set $\{a, b, c\}$ does not. Telling which of these is the case is undecidable. Note that we don't, in general, allow tiles to be rotated or reflected.

This CA is called rule 110, since 01101110 is 110 in binary.

As Figure 7.15 shows, rule 110 has a wide variety of gliders and periodic structures. By using their collisions in exceedingly clever ways, we can simulate a kind of tag system, whose rules and initial string are given as part of the initial state. Like the Turing machine in Figure 7.14, the initial state of this simulation consists of a finite string, with spatially periodic patterns repeated to its left and right. Hence rule 110 is capable of universal computation, and the question of whether a given string of bits will ever appear in the lattice, given an initial state of this kind, is undecidable.

In turn, the smallest known universal Turing machines work by simulating rule 110. However, like the initial state of the CA, the initial tapes of these Turing machines are filled with periodic patterns to the left and right of the input string, as opposed to being filled with blanks. They never halt, but the question of whether a given string of bits will ever appear on their tape, immediately to the left, say, of their head, is undecidable.

Life and rule 110 show that even systems with very simple rules can perform universal computation, and thus generate undecidable behavior. Indeed, it's hard to imagine a system whose "laws of physics" are simpler without being trivial. We can build computers out of virtually anything, as long as their physics allows for particles that can carry information from place to place, and nontrivial interactions between these particles that let us process this information.

7.6.5 Tiling the Plane

The TILING problem asks whether we can cover the plane with tiles of a given set of types. In Section 5.3.4, we saw that for finite regions, this problem is NP-complete. Here we will show that for the *infinite* plane, it is undecidable—by showing how tiles can perform universal computation in space, rather than time.

We can fit tiles together according to shape or color. *Wang tiles* have a color along each edge, and the colors of neighboring tiles are required to match. Figure 7.16 shows two sets of these tiles. The set on the

7.22

7.15

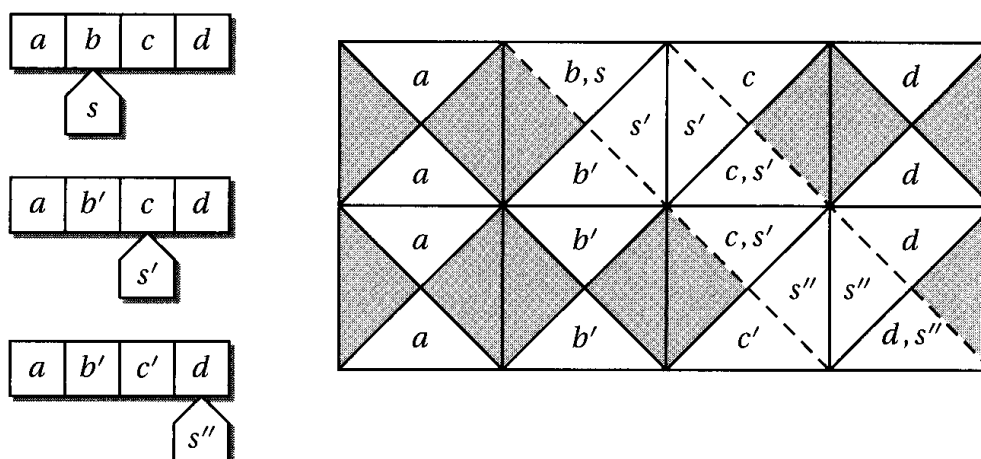


FIGURE 7.17: Simulating a Turing machine with Wang tiles. Plain tape symbols have gray on their sides, and transmit their symbols from top to bottom. Tiles representing the head transmit its state to the left or right, and also modify the tape symbol. The dashed lines show the head's path.

left can tile the infinite plane, but the set on the right always produces a gap that cannot be filled. The following problem asks which is which:

TILING

Input: A set of T Wang tiles

Question: Can we tile the infinite plane with tiles from T ?

Given a Turing machine, it's easy to design a set of Wang tiles such that each row corresponds to a step of the machine's computation. As Figure 7.17 shows, the tiles correspond to tape symbols, with special tiles marking the head's location and state. Each color corresponds to a tape symbol a , a head state s , or a pair (a, s) . Matching these colors transmits a tape symbol from each tile to the one below it, and transmits the head's state to the left or right.

Now suppose that there are no tiles in our set colored with the Turing machine's HALT state. In that case, if the machine halts, there is no way to complete the tiling. If we specify the tiles on one row, starting the machine in its initial state and giving it an input string, then we can fill in the tiles below that row, all the way out to infinity, if and only if the machine never halts. In fact, since the Halting Problem is undecidable even for blank tapes (see Exercise 7.20), it suffices to specify a single tile at the origin corresponding to the machine's head. This shows that the following variant of TILING is undecidable:

TILING COMPLETION

Input: A set T of Wang tiles, and specified tiles at a finite set of positions

Question: Can this tiling be completed so that it covers the infinite plane?

However, this construction doesn't show that TILING is undecidable. After all, it's easy to tile the plane with the tiles in Figure 7.17—just fill the plane with tape symbols, with no Turing machine head around

to modify them. Without specifying at least one initial tile, how can we make sure that, somewhere in the plane, there is actually a Turing machine performing a computation?

Let's pause for a moment to discuss an issue that may seem unrelated at first. The Wang tiles on the left of Figure 7.16 tile the plane periodically. In fact, the unique tiling they admit consists of the 3×3 block shown in the figure, repeated forever. Now consider the following exercise:

Exercise 7.28 Suppose that, for any set T of Wang tiles, either T allows a periodic tiling of the plane, or cannot tile the plane at all. Show that if this were true then TILING would be decidable.

Therefore, if TILING is undecidable, there must be a set of tiles that can tile the plane, but not periodically. We call such a set of tiles *aperiodic*.

Finding an aperiodic set of tiles takes some thought, but such sets do exist. One of them, consisting of six shapes and their rotations and reflections, is shown in Figure 7.18. If we decorate these tiles with stripes of two different colors, say black and gray, and if we require neighboring tiles to match both according to color and shape, they yield a fractal pattern of squares of size 2×2 , 4×4 , 8×8 , and so on.

We can use these squares as scaffolds on which to force a Turing machine to perform computations of increasing length. Specifically, we expand the set of tiles so that each one carries a tape symbol or a state-symbol pair. We then require the tile at the upper-left corner of each square to contain a head in its initial state, and the other tiles along the square's upper edge to encode a blank tape. By requiring these tiles to match like the Wang tiles described above, we force the Turing machine to run for 2 steps in each 2×2 square, 4 steps in each 4×4 square, and so on. (Using Exercise 7.18, we can assume that the machine will never move to the left of its initial position.) Since there are squares of arbitrarily large size, we can only tile the infinite plane if the machine runs forever.

We have a little more work to do. The problem is that these computations will overlap—since each tile is a part of an infinite number of squares of increasing size, which computation should it be a part of? We can avoid this problem with a clever construction shown in Figure 7.19. First, we focus on the $1/4$ of the tiles which protrude at their corners, i.e., the leftmost two shapes in Figure 7.18. Within each black square, we then pick out a subset of these tiles, namely the ones at the intersections of the rows and columns that don't cross a smaller black square.

In each black square of side 2^{2k} , this gives us a $2^k \times 2^k$ “board” of tiles. The tiles in each board are not adjacent; they are connected by the dotted lines in Figure 7.19. But we can simulate adjacency with additional tiles that transmit states and symbols along these dotted lines. These boards lie inside each other, but they don't overlap—so we can use each one to force the Turing machine to run for 2^k steps.

This reduces BLANK TAPE HALTING, or rather its complement, to TILING, and proves that the latter is undecidable.



7.23

7.6.6 From Chaos to Computation

Computation can also occur in continuous dynamical systems, where we start with an initial point and repeatedly apply a function f to it. Consider the following function, defined on the unit square $[0, 1] \times [0, 1]$:

$$f(x, y) = \begin{cases} (x/2, 2y) & \text{if } y < 1/2 \\ (x/2 + 1/2, 2y - 1) & \text{if } y \geq 1/2. \end{cases}$$

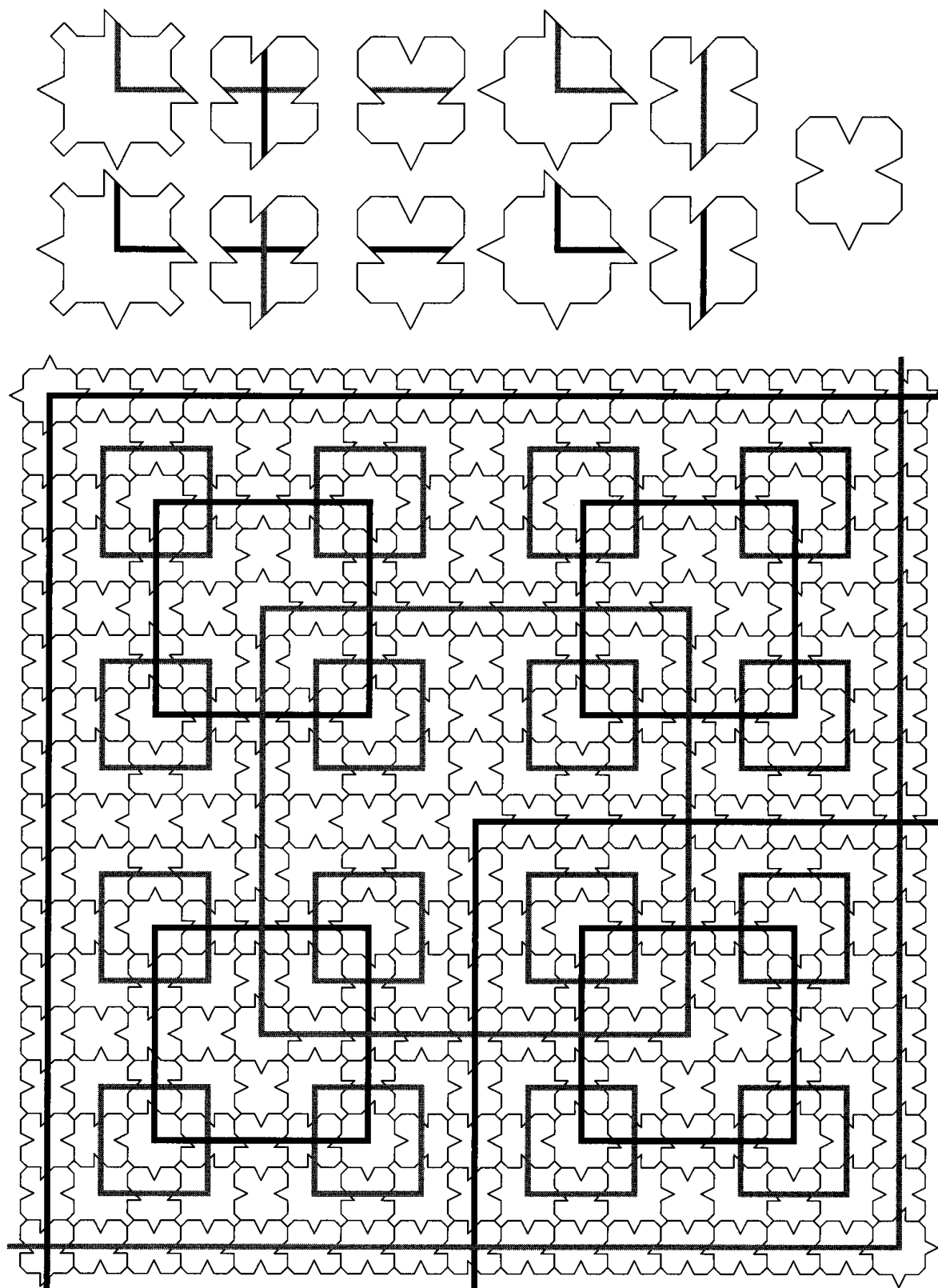


FIGURE 7.18: Robinson's aperiodic tiles. When decorated with black and gray stripes, they form a fractal hierarchy of squares whose sides are powers of 2. We then use these squares as scaffolds for Turing machine computations.

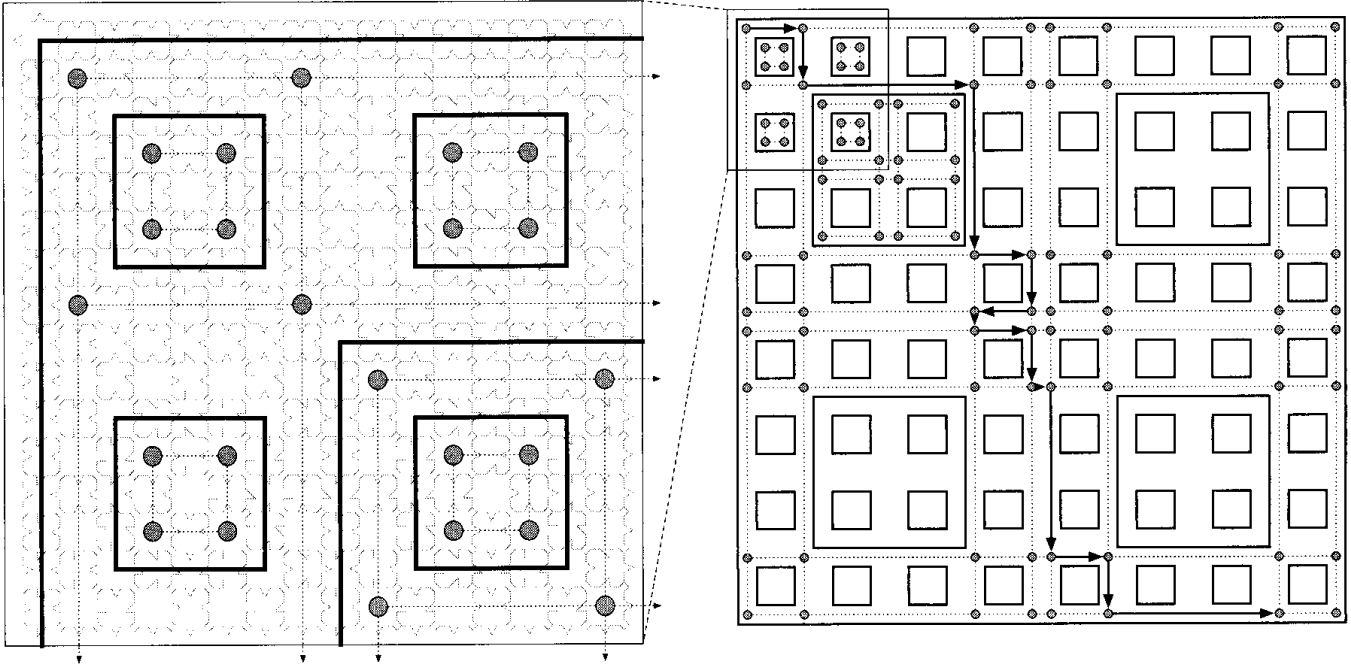


FIGURE 7.19: Each black square contains a “board” consisting of the tiles with protruding corners whose rows and columns are not interrupted by smaller black squares. Here these tiles are marked with gray circles, connected by dotted lines. On the left, the region shown in Figure 7.18. On the right, we zoom out, and show how black squares of side 4×4 , 16×16 , and 64×64 contain boards of size 2×2 , 4×4 , and 8×8 . We use additional types of tiles to transmit the machine’s states and symbols along the dotted lines, so that each $\ell \times \ell$ board simulates ℓ steps of the Turing machine. Shown in bold is a possible path that the machine might take as part of an 8-step simulation.

This is called the *Baker’s Map*. It stretches the unit square, cuts it, and then places the two stretched halves together, like a baker stretching and kneading dough. We show its action in Figure 7.20.

What does this map do in digital terms? Let’s write the binary expansions of x and y as $x = 0.x_1x_2x_3\dots$ and $y = 0.y_1y_2y_3\dots$ respectively, where

$$x = \sum_{i=1}^{\infty} 2^{-i} x_i \quad \text{and} \quad y = \sum_{i=1}^{\infty} 2^{-i} y_i.$$

Then f shifts y ’s bits towards the decimal point, shifts x ’s bits away from it, and moves the most significant bit of y to x :

$$f(0.x_1x_2x_3\dots, 0.y_1y_2y_3\dots) = (0.y_1x_1x_2\dots, 0.y_2y_3y_4\dots).$$

Suppose we start with two different initial points, (x, y) and (x', y') , and apply f repeatedly to both of them. If y and y' differ in their i th bit then this bit gets shifted towards the decimal point, doubling the distance $|y - y'|$ at each step. After i steps, y and y' disagree on their most significant bits, and the two points are in completely different parts of the square.

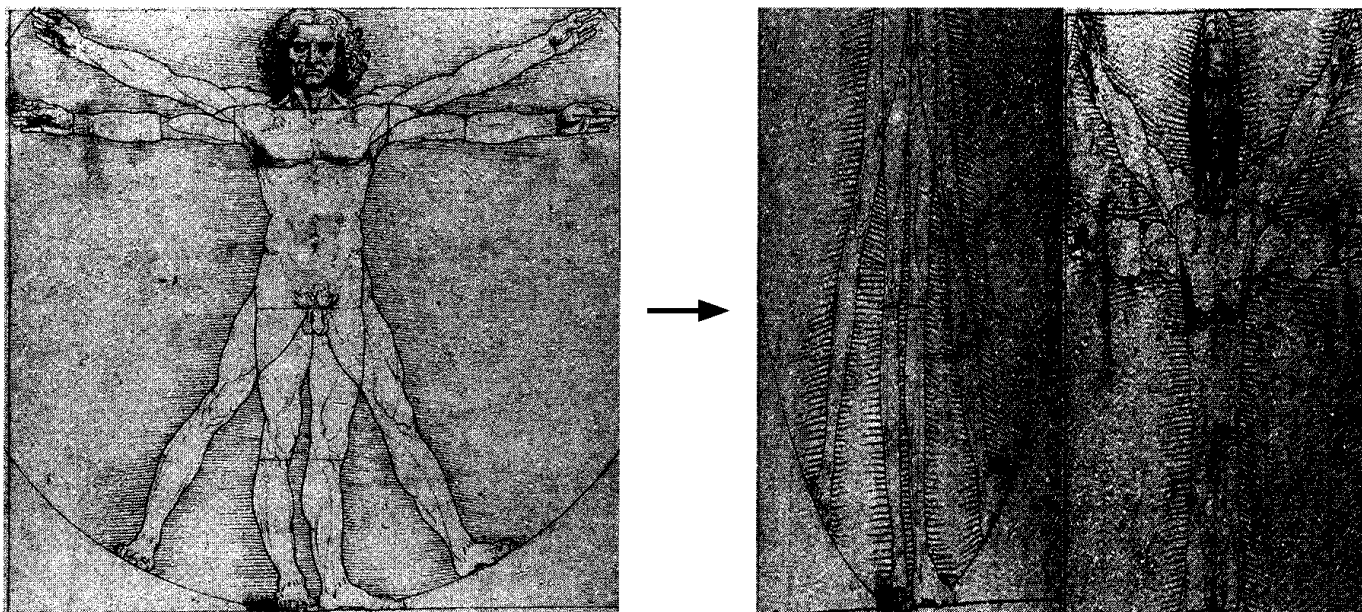


FIGURE 7.20: The Baker's Map f on the unit square. It doubles y , halves x , and moves y 's most significant bit to x . We can think of the coordinates (x, y) as a tape full of bits, $\dots y_3 y_2 y_1 . x_1 x_2 x_3 \dots$, where $0.x_1 x_2 x_3 \dots$ and $y = 0.y_1 y_2 y_3 \dots$ are the binary expansions of x and y respectively. Then f moves the decimal point, representing the head of a Turing machine, one step to the left.

This makes the Baker's Map a classic example of a chaotic dynamical system. It quickly magnifies any perturbations or uncertainties in its initial conditions, until it becomes completely unpredictable. If we only know the initial conditions to t bits of accuracy, we cannot predict the state of the system more than t steps in the future.

Exercise 7.29 Show that the Baker's Map has two unique fixed points and a unique period-2 orbit, i.e., a pair of points (x, y) and (x', y') such that $f(x, y) = (x', y')$ and $f(x', y') = (x, y)$. What are their digit sequences, and where are they in the unit square? How many period-3 orbits are there? What can you say about period- t orbits for larger t ?

How can maps like this simulate Turing machines? If we write the bits of y in reverse, we can think of (x, y) as an infinite tape whose tape symbols are 0 and 1,

$$\dots y_3 y_2 y_1 . x_1 x_2 x_3 \dots,$$

where the decimal point represents the machine's head. In that case, we have

$$f(\dots y_3 y_2 y_1 . x_1 x_2 x_3 \dots) = f(\dots y_3 y_2 . y_1 x_1 x_2 x_3 \dots),$$

so f moves the head one step to the left. Note the similarity to the arithmetization of the Turing machine on page 262, where we treated each half of the tape as the bit sequence of an integer. Both there and here, we move the machine's head by halving one of these numbers and doubling the other.

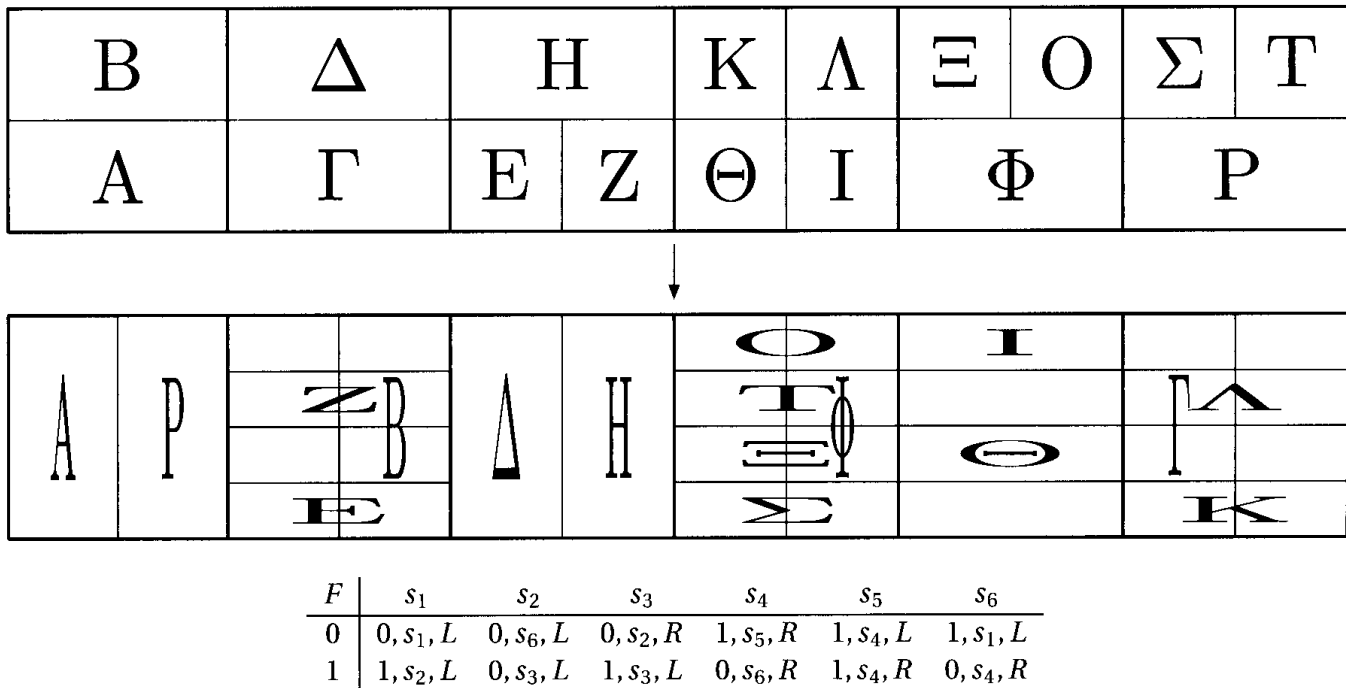


FIGURE 7.21: Above, an iterated map f on the rectangle $[0, 6] \times [0, 1]$ equivalent to a universal Turing machine with 6 states and 2 symbols. Each large square corresponds to one of the machine's states, and the current tape symbol is the most significant bit of y . Stretching vertically or squashing horizontally corresponds to moving the machine's head left or right on the tape. Below, the machine's transition function. Each triplet gives the new symbol, the new state, and whether the head moves left or right.

To complete the simulation, let's say that the tape symbol at the machine's current location is y_1 , which is 0 or 1 if we are in the lower or upper half of the square respectively. At each step, we can change y_1 by shifting up or down by $1/2$. We then move the head left or right on the tape by stretching vertically or squashing horizontally.

Finally, we define our function on a set of unit squares, one for each of the machine's internal states, and update the state by mapping pieces of one square to another. If the machine has s states, and we put these squares next to each other, we get a piecewise-continuous function f from the rectangle $[0, s] \times [0, 1]$ to itself. This function divides this rectangle into a finite number of pieces, stretches or squashes each one, and maps it back somewhere inside the rectangle.

In Figure 7.21, we carry out this construction for a universal Turing machine with 6 states and 2 tape symbols. This machine simulates cellular automaton rule 110, so the question of whether a particular finite string of bits will ever appear on its tape, at its head's current location, is undecidable. In our map, this corresponds to x and y lying in a pair of finite intervals, where their binary expansions start out with particular finite strings. Thus, given an initial point (x, y) , the question of whether it will ever land in a particular rectangle is undecidable.

As we discussed at the end of Section 7.6.4, the initial tape of this Turing machine is filled with periodic patterns to the left and right of its input. Such a tape corresponds to an initial point (x, y) with rational coordinates, since a real number is rational if and only if its binary expansion becomes periodic after a

finite number of bits. So, even if we know the initial point *exactly*, questions about f 's dynamics like the following are undecidable:

ITERATED MAP PREDICTION

Input: Rational numbers $x, y, x_{\text{left}}, x_{\text{right}}, y_{\text{bottom}}, y_{\text{top}}$

Question: Is there an integer t such that $f^t(x, y)$ lies in the rectangle $[x_{\text{left}}, x_{\text{right}}] \times [y_{\text{bottom}}, y_{\text{top}}]$?

Indeed, essentially any question about the long-term trajectory of such maps is undecidable, such as whether a given point is periodic, whether two initial points will ever collide, and so on. Thus these maps are unpredictable in a much more fundamental way than the Baker's Map is. Their behavior is computationally universal, as opposed to merely chaotic.

7.24

To sum up, iterated maps can simulate Turing machines, which simulate rule 110, which simulate tag systems, which simulate Collatz functions, which simulate FRACTRAN programs, which simulate counter machines, which can in turn simulate other Turing machines. . . a fitting chain of simulations with which to end our chapter.

Problems

As long as a branch of science offers an abundance of problems, so long is it alive; a lack of problems foreshadows extinction or the cessation of independent development.

David Hilbert, 1900

7.1 Computability is rare. Prove that the number of programs is countably infinite, but the number of functions $f: \mathbb{N} \rightarrow \mathbb{N}$ or $f: \mathbb{N} \rightarrow \{\text{true}, \text{false}\}$ is uncountably infinite. Therefore, almost all functions are uncomputable.

7.2 Goldbach's other conjecture. The mathematician Christian Goldbach (1690–1764) loved to invent mathematical conjectures, a passion he shared with his friend Leonhard Euler. His most famous conjecture, the one known today as *Goldbach's Conjecture*, states that every even number greater than or equal to 4 is the sum of two primes. In another conjecture, Goldbach claimed that every odd number can be written as the sum of a prime p (including $p = 1$ which was considered a prime back then) and two times the square of an integer $a \geq 0$, like $11 = 3 + 2 \times 2^2$ or $23 = 5 + 2 \times 3^2$. Euler himself checked that this conjecture holds for all odd numbers up to 2500, but he couldn't find a proof.

Write a computer program that halts if and only if Goldbach's other conjecture is false. Then run the program.

7.3 More undecidable problems. Show that it is undecidable, given the source code of a program Π , to tell whether or not any of the following is true:

1. Π halts on input 0.
2. Π computes a total function—that is, $\Pi(x)$ halts for all x .
3. $\Pi(x) = \text{true}$ for all x .
4. The set of x on which Π halts is finite.
5. There is an x such that $\Pi(x) = x$.

6. Π is equivalent to some other program Ψ . That is, even though Π and Ψ have different source codes, they compute the same partial function—for all x , either $\Pi(x)$ and $\Psi(x)$ both halt and return the same answer, or neither halts. Show that for any program Ψ , this problem is undecidable.

Prove each of these by reducing HALTING to them. That is, show how to convert an instance (Π, x) of HALTING to an instance of B . For instance, you can modify Π 's source code, or write a new program that calls B as a subroutine. Note that your reduction does not necessarily have to map yes-instances to yes-instances and no-instances to no-instances—all that matters is that if you could solve B , you could solve HALTING.

7.4 Rice's Theorem. We saw some examples of reductions from HALTING in the previous problem. Generalize these reductions and prove the following theorem:

Theorem 7.2 (Rice's Theorem) *Let P be a property of programs that depends only on the partial functions that they compute. Assume that there is at least one program Π_1 for which P is true, and at least one program Π_2 for which P is false. Then show that it is undecidable, given a program Π , to tell whether P is true for Π or not.*

Properties like these, that depend only on a program's output and not on its internal structure or dynamics, are called *semantic*.

Hint: show that for any such property P , we can reduce HALTING to the question of whether or not P is true. Note that in some cases your reduction may have to map yes-instances to no-instances and vice versa.



7.8

7.5 It's hard to tell if the only loops are simple. In Exercise 7.4 you showed that the Halting Problem is decidable for any program Π with the property that, for each x , $\Pi(x)$ either halts or falls into an endless loop. Now prove that telling whether a given program Π has this property is itself undecidable.

7.6 P vs. NP. We saw in Section 6.1.2 that if $P = NP$ then $\text{TIME}(G) = \text{NTIME}(G)$ for any class of functions $G(n)$ that is closed under composition with polynomials. This is certainly true if G is the class of all computable functions. On the other hand, we argued in Section 7.2.3 that Decidable and RE are analogous to P and NP respectively, and we proved using the Halting Problem that $\text{Decidable} \neq \text{RE}$. Why isn't this a proof that $P \neq NP$?

7.7 Domains and ranges. Given a partial function $f(x)$, we define its *domain* $\text{Dom } f$ as the set of x for which $f(x)$ is well-defined. In particular, if f is computed by some program, its domain is the set of inputs for which that program halts. Similarly, its *range* is the set of all its well-defined outputs, i.e., $\{f(x) : x \in \text{Dom } f\}$. Show that the following conditions are equivalent properties of a set S :

1. S is recursively enumerable.
2. S is the domain of some computable partial function.
3. S is the range of some computable partial function.

7.8 Enumerating in order. Consider a set $S \subseteq \mathbb{N}$ such that there is a program that prints out all the elements of S in increasing order. Show that S is decidable.

7.9 Decidable subsets of enumerable sets. Show that if S is infinite and recursively enumerable, it has an infinite decidable subset. Hint: consider Problem 7.8.

7.10 Total functions can't be enumerated. Show that the set of total computable functions is not recursively enumerable. In other words, show that there is no computable function U_{tot} such that $U_{\text{tot}}(i, x)$ is well-defined for all i and all x , with the property that, for every total computable function $f(x)$, there is an i such that $f(x) = U_{\text{tot}}(i, x)$ for all x . To put this differently, there is no program that prints out a list of all the programs that always halt.

7.11 We can't print the ones that don't halt. Show that the set of programs Π that halt when given themselves as input is recursively enumerable, but that its complement is not.

7.12 Hypercomputers. Show that if we have access to an oracle for the Halting Problem, the class of problems we can solve in finite time becomes $\Sigma_2 \cap \Pi_2$.

Hint: note that a problem is in $\Sigma_2 \cap \Pi_2$ if both its yes-instances and its no-instances can be described in the form $\exists y : \forall z : B(x, y, z)$ for some decidable B . (Compare Problem 6.11, which shows that $P^{NP} \subseteq \Sigma_2 P \cap \Pi_2 P$.)

7.13 Infinite sets of axioms. Show that Gödel's Incompleteness Theorem holds for formal systems with an infinite number of axioms, as long as the set of axioms is recursively enumerable.

7.14 Primitive logarithms. Starting with the results of Exercise 7.10, show that the function $\log_2(x) = \lfloor \log_2 x \rfloor$ is primitive recursive (with the convention $\log_2(0) = 0$). More generally, show that for any primitive recursive functions $g(y)$ and $t(x)$, the function

$$f(x) = \max\{y : y \leq t(x) \text{ and } g(y) \leq x\}$$

is primitive recursive. Hint: Note that $\log_2(x)$ is the inverse of 2^y , and the function $f(x)$ is the generalized inverse of $g(y)$.

7.15 Pairing up. In modern computer science, it is trivial to think of any finite collection of integers as a finite sequence of bits, and therefore as a single integer. In the world of recursive functions, we can do this with the functions $\text{pair} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$, $\text{left} : \mathbb{N} \rightarrow \mathbb{N}$, and $\text{right} : \mathbb{N} \rightarrow \mathbb{N}$ such that if $p = \text{pair}(x, y)$, then $\text{left}(p) = x$ and $\text{right}(p) = y$. One such pairing works as follows:

p	0	1	2	3	4	5	6	7	8	9	...
x	0	1	0	2	1	0	3	2	1	0	...
y	0	0	1	0	1	2	0	1	2	3	...

Show that pair , left , and right are primitive recursive. Hint: first figure out what $\text{pair}(x, 0)$ is. Then use the technique of Problem 7.14 to define left and right .

7.16 Recursion through iteration. Using the results of the previous problem, show that when defining primitive recursive functions, we really only need to think about functions of a single variable. In particular, show that a function $f(x)$ is primitive recursive if and only if it can be generated from 0, S , pair , left , and right , using composition and *iteration*, where iteration is defined as follows: if $f(x)$ and $g(x)$ are already defined, we can define a new function

$$h(y) = g^y(f(0)). \quad (7.29)$$

In fact, we can simplify this further and just take $h(y) = g^y(0)$.

7.17 Ackermann explodes. In this problem, we will prove that the Ackermann function grows more quickly than any primitive recursive function. We will focus on the following family of functions of a single variable:

$$G_n(y) = A(n, 2, y).$$

In particular,

$$G_1(y) = y + 2, \quad G_2(y) = 2y, \quad G_3(y) = 2^y, \quad G_4(y) = \underbrace{2^{2^{\cdot^{\cdot^2}}}}_y, \dots$$

For $n \geq 3$, we can write G_n iteratively as

$$G_n(y) = G_{n-1}^y(1) = G_{n-1}(G_n(y-1)).$$

Show that these functions *majorize* the primitive recursive functions in the following sense: for any primitive recursive function $f(y)$, there is an n such that $f(y) < G_n(y)$ for all $y \geq 3$. It follows that $G_y(y) = A(y, 2, y)$ grows faster than any primitive recursive function.

Hint: use the definition of the primitive recursive functions given by Problem 7.16. Clearly 0, S, pair, left, and right are all majorized by $G_n(y)$ for some n . Then show that if two functions f and g are majorized, so are the functions formed from them by composition and iteration. In other words, if $f(y) < G_n(y)$ and $g(y) < G_m(y)$ for all $y \geq 3$, then there is some k, ℓ such that $f(g(y)) < G_k(y)$ and $g^y(f(0)) < G_\ell(y)$. You may find it useful to prove that, for all $n \geq 2$ and $y \geq 3$,

$$G_n(y) \geq G_{n-1}(y+1). \quad (7.30)$$

7.18 Packing stacks into numbers. Show that we can represent a stack of arbitrary depth, where each cell contains an arbitrary integer, as a single integer—and that we can manipulate this state with functions push, pop, and isempty that are primitive recursive. Hint: represent the stack as a pair, consisting of the top integer and everything else.

7.19 Metaprograms, metametaprograms... Imagine computing a function $f(x)$ in the following way. We give an input x to a BLOOP program Π , which prints out another BLOOP program Π_x . We then run Π_x , which produces $f(x)$. Show that the class of functions that can be computed this way is larger than the primitive recursive functions. Does it include all computable functions? What about programs that print programs that print programs, and so on?

Along the same lines, show that the Ackermann function $A(n, x, y)$ can be implemented as a BLOOP program provided that we can call a universal interpreter for BLOOP programs as a subroutine. Why does this not prove that $A(n, x, y)$ is primitive recursive?

7.20 Primitive recursive time. Sketch a proof that a function $f(n)$ is primitive recursive if and only if it can be calculated in $t(n)$ time for some primitive recursive function $t(n)$.

7.21 Quines. Let's think of the combinator $\mathbf{Q} = \lambda x. xx$ in a slightly different way: as a program that takes a program Π and prints the source code—not the output—of the program $\Pi(\Pi)$. That is, if Π is a program that expects an input x , then $\mathbf{Q}(\Pi)$ is a version of Π which takes no input, and instead assumes that x is the source code of Π .

Show that $\mathbf{Q}(\mathbf{Q})$ is a *quine*—a program that, when we run it, prints out a copy of its own source code. If you are a good programmer, write $\mathbf{Q}$ in your favorite language. (Actually dealing with input and output conventions, line breaks, and so on can be quite tricky.) Then feed it to itself, run the resulting program $\mathbf{Q}(\mathbf{Q})$, and see if it works.



7.13

7.22 Adding in λ . Show that the λ -expressions **add** and **add'** defined in (7.16) and (7.17) are equivalent when applied to Church numerals. That is, show that for any $\bar{i}, \bar{j}$,

$$\mathbf{add} \bar{i} \bar{j} = \mathbf{add}' \bar{i} \bar{j}.$$

This is intuitively clear, but proving it takes some work. Hint: use induction. Start with the base case $\bar{i} = \bar{0}$, and then show that

$$\mathbf{add}(\mathbf{succ} \bar{i}) \bar{j} = \mathbf{succ}(\mathbf{add} \bar{i} \bar{j}) \text{ and } \mathbf{add}'(\mathbf{succ} \bar{i}) \bar{j} = \mathbf{succ}(\mathbf{add}' \bar{i} \bar{j})$$

where **succ** is the successor function defined in (7.15).

7.23 The wisdom teeth trick. Express the predecessor of a Church numeral in λ -calculus, i.e., find a combinator **pred** such that

$$\mathbf{pred} \bar{n} = \overline{n-1} \quad (n > 0)$$

$$\mathbf{pred} \bar{0} = \bar{0}.$$

As legend has it, Stephen Kleene solved this problem while having his wisdom teeth pulled, so his solution is sometimes referred to as the “wisdom teeth trick.” Here are some hints to help you find it without the use of novocaine.

First, we need a way to keep track of a pair of Church numerals without having them act on each other. The following combinators let us form pairs and extract their components:

$$\mathbf{pair} = \lambda xy f . fxy, \mathbf{first} = \lambda p . pT, \mathbf{second} = \lambda p . pF,$$

where T and F are defined as in (7.13). These are analogous to the primitive recursive functions of Problem 7.15, but in the λ -calculus we don't need to use arithmetic to do this.

Now write a combinator Φ that changes the pair (i, j) to $(j, j + 1)$. What happens if we apply Φ n times to $(0, 0)$?

7.24 Factorials in λ . Using the same pairing combinators as in the previous problem, write a λ -expression for the factorial function that doesn't use the Y -combinator.

7.25 Primitive recursion in λ . Now generalize the previous problem to show that if f and g are λ -definable, so is the function h defined from them using primitive recursion (7.6). Then use the λ -calculus's power of abstraction to show that the primitive recursion operator, which takes f and g as input and returns h , can itself be written as a combinator.

7.26 More fixed point combinators. Here are two other fixed-point combinators. One, discovered by Turing, is

$$\Theta = \eta\eta \quad \text{where} \quad \eta = \lambda xy . y(xxy).$$

Another, discovered by John Tromp, is

$$\mathbf{T} = \mathbf{T}_1\mathbf{T}_2 \quad \text{where} \quad \mathbf{T}_1 = \lambda xy . xyx \quad \text{and} \quad \mathbf{T}_2 = \lambda yx . y(xyx).$$

Prove that $\Theta \mathbf{R} = \mathbf{R}(\Theta \mathbf{R})$ for any $\mathbf{R}$, and similarly for $\mathbf{T}$.

7.27 Fixed point programs. Following the style of Problem 7.21, show how to interpret the fixed point combinator $\mathbf{Y}$ as a program that takes a program-modifying program $\mathbf{R}$ as input, and prints out the source code of a program f which is a fixed point of $\mathbf{R}$. We don't necessarily mean that $\mathbf{R}$ leaves f 's source code unchanged—we mean that it is a *functional* fixed point, in that f and $\mathbf{R}(f)$ compute the same partial function.

7.28 Two stacks suffice. As we have seen in several contexts, a *stack* is a data structure that works like a stack of plates. Each plate has a symbol written on it, which belongs to a finite alphabet A . We can “push” a symbol onto the stack, adding another plate on top; we can “pop” a symbol off the stack, removing the top plate and reading the symbol on it; or we can check to see if the stack is empty. But like a Turing machine's tape, we can only observe a stack in a restricted way. Only the top symbol is visible to us, and we have to pop it off if we want to see the symbols below it.

Consider the following kind of automaton. It has a finite set S of internal states, and access to k stacks. At each step, it can update its own internal state, and push or pop a symbol on each stack. It chooses these actions based only on its own internal state and the top symbols on the stacks, or whether or not they are empty. We can give such an automaton an input by putting a string of symbols on one of its stacks, and read its output from another stack when it enters a **HALT** state.

Show that if $k \geq 2$, such an automaton can simulate a Turing machine, and hence is computationally universal. Furthermore, show that if we are willing to accept an inefficient simulation, even a one-symbol alphabet $A = \{a\}$ suffices. Note that in that case, all that matters about a stack's state is its depth.

7.29 One queue suffices. Instead of stacks, suppose our automaton has access to data structures called *queues*. The only visible symbol is the one at the front of the queue, which is the one that gets removed if we pop a symbol. When we push a symbol, it gets added at the rear of the queue. We again have an automaton with a finite set S of internal states, which can push or pop a symbol on each queue or check to see if they are empty. Show that even one queue suffices to simulate a Turing machine.

7.30 Two, three, and four dimensions. Consider two-dimensional finite automata which are trapped inside some region in the planar grid. At each step, they can move one step in one of the cardinal directions based on their current state, and then update their state. Like the automata discussed in Section 7.6.1, they cannot write anything on the cells they visit, and the only way they can sense their current position is to tell when they bump into a wall. Thus their transition function can be written

$$F : \{\text{wall, not wall}\} \times S \rightarrow S \times \{\uparrow, \downarrow, \leftarrow, \rightarrow\},$$

where S is their finite set of states. We assume that whenever they bump into a wall, they bounce off it, back into the interior of the region.

Sketch definitions of two-dimensional automata that can determine

1. whether the region they are trapped in is a rectangle;
2. whether or not they are in a square;
3. whether or not they are in a square whose side is a power of 2.

Similarly, sketch a definition of a three-dimensional automaton that can tell whether it is in a cube whose side is a prime—and a four-dimensional automaton that can tell whether it is in a hypercube whose side is a perfect number, i.e., a number which is the sum of its divisors.

7.31 One-counter machines. Show that the Halting Problem for one-counter additive machines is decidable. Hint: first show that if the machine has n states and the counter reaches zero more than n times in the course of a computation, it will run forever.

7.32 FRACTRAN practice. Write a non-halting FRACTRAN program that generates the Fibonacci sequence in the following sense: if the initial value is $n = 6 = 2^1 3^1$, it generates $2^1 3^2$, $2^2 3^3$, $2^3 3^5$, and so on, so that the exponents of 2 and 3 are adjacent Fibonacci numbers whenever n has no other factors. The shortest such program we have found is

$$\frac{33}{14} \quad \frac{21}{22} \quad \frac{13}{7} \quad \frac{13}{11} \quad \frac{26}{85} \quad \frac{34}{65} \quad \frac{1}{13} \quad \frac{1}{17} \quad \frac{10}{3} \quad \frac{7}{1}$$

How does this program use the exponents of 2, 3, and 5, and larger primes to encode the state? Can you find a shorter program?

Then write a FRACTRAN program that, given the input 2^k in the form of its initial value n , halts if k is a power of 2, and otherwise falls into an infinite loop. Hint: you can arrange an infinite loop by putting a pair of fractions $p/q, q/p$ at the head of the program.

7.33 Euclid's algorithm in FRACTRAN. Write a FRACTRAN program that, given input $2^a 3^b 5$, halts with output $2^{\gcd(a,b)}$. Hint: start by implementing a (very short) FRACTRAN program for the map

$$2^a 3^b 5 \mapsto 2^{a-\min(a,b)} 3^{b-\min(a,b)} 7^{\min(a,b)} 5.$$

Extend this program to deal with the cases $a > b$, $a < b$, and $a = b$. Then iterate it and move the counters around so that leads to the state $2^b 3^{a \bmod b} 5$. Finally, arrange the program so that it strips off the 5 and terminates if $b = 0$.

7.34 Collatz with tags. Let g be a Collatz function with coefficients a_i and b_i defined as in (7.27). Show that there is a tag system with $\nu = q$ on the following set of $2q + 2$ symbols,

$$\{\alpha, \beta_0, \beta_1, \dots, \beta_{q-1}, \gamma, \delta_1, \delta_2, \dots, \delta_{q-1}\},$$

such that, for any x , the string $\alpha\gamma^x$ is mapped to $\alpha\gamma^{g(x)}$.

Hint: start with the rules

$$h(\alpha) = \beta_{q-1} \cdots \beta_1 \beta_0$$

$$h(\gamma) = \delta_{q-1} \cdots \delta_1 \delta_0.$$

Notice that if $x \equiv i \pmod q$, we will meet strings beginning with β_i and δ_i . Then devise rules of the form

$$h(\beta_i) = \gamma^r \alpha \gamma^s$$

$$h(\delta_i) = \gamma^t,$$

where r , s , and t depend on a_i and b_i for each $0 \leq i < q$.

7.35 Tagging Turing. Give another proof that NORMAL FORM HALTING is undecidable, by using a tag system in normal form to simulate Turing machines directly. Hint: pretend the tape is circular.

7.36 Rotating tiles. Show that TILING with Wang tiles remains undecidable even if rotations are allowed. If we change the matching rule so that colors come in complementary pairs, and neighboring tiles have to have complementary colors along their shared edge, show that TILING is undecidable even if we allow rotations and reflections.

7.37 Stripes imply rectangles. Suppose that a set of Wang tiles can tile the plane with horizontal periodicity. In other words, there is some integer ℓ such that the tiling remains the same if we shift the lattice ℓ sites to the left. Equivalently, the tiling consists of an infinite vertical strip of width ℓ , repeated endlessly to the left and right. Show that in this case, the same set of Wang tiles can tile the plane periodically, with a finite rectangle repeated forever.

Notes

7.1 The Difference Engine. Babbage's Difference Engine was designed to evaluate a polynomial $p(x)$ at a sequence of equidistant values $x, x + h, x + 2h, \dots$ for some step size h . As stated in the text for the case $h = 1$, if f is a polynomial of order n , its difference $\Delta f(x) = f(x + h) - f(x)$ is of order $n - 1$, until its n th-order difference is a constant. After manually setting the initial values $f(0), \Delta f(0), \dots, \Delta^n f(0)$, we update them as follows:

$$f(x + \delta) = f(x) + \Delta f(x)$$

$$\Delta f(x + \delta) = \Delta f(x) + \Delta^2 f(x)$$

$$\vdots$$

$$\Delta^{n-1} f(x + \delta) = \Delta^{n-1} f(x) + \Delta^n f.$$

Note that the machine never needs to multiply. If $\Delta^i f(x)$ is stored in the i th column, at each step it simply adds the current value of the i th column to the $(i - 1)$ st. The n th column stays constant, and the 0th column contains $f(x)$.

If Babbage had gotten his machine working, it is doubtful whether he would have simply used Taylor series. In many cases, we can obtain much better approximations by using low-degree polynomials whose coefficients are tuned to achieve the smallest possible error in a given interval. For instance, the approximation

$$\cos x \approx 1 - 0.4967x^2 + 0.03715x^4,$$

has an error of less than 10^{-3} on the interval $0 \leq x \leq \pi/2$, much smaller than that of the 4th-order Taylor series.

Babbage first proposed the Difference Engine in 1822. In 1847–1849 he designed an improved machine, “Difference Engine No. 2,” which would have evaluated polynomials up to order 7 with a precision of 31 digits. None of Babbage’s machines were completed during his lifetime, but in 1989, the London Science Museum began to build a working Difference Engine No. 2 according to Babbage’s design. It was completed in 1991, two hundred years after Charles Babbage’s birth, and worked perfectly.

7.2 Other pioneers of computing. Percy Ludgate was an Irish accountant, who independently of Babbage (at least at first) spent his spare time designing mechanical computers. His scheme for multiplication was quite idiosyncratic. Instead of storing a multiplication table, he assigned “index numbers”—or as they were called in a 1909 article about his work [133], “Irish logarithms”—to all products of one-digit numbers:

x	0	1	2	3	4	5	6	7	8	9	10	12	14	15	16	18	20	...
$\log x$	50	0	1	7	2	23	8	33	3	14	24	9	34	30	4	15	25	...

These numbers look quite strange, but they obey $\log xy = \log x + \log y$ for all $0 < x, y \leq 9$. Ludgate’s proposed machine could multiply large numbers quickly by computing all these double-digit products, and then carrying the tens in parallel. Zero is a special case, and corresponds to all logarithms greater than or equal to 50.

Ludgate’s scheme for division was also very original. To compute p/q , he first multiplied the numerator and denominator by a number close to $1/q$, giving $p/q = p'/(1 + \varepsilon)$ for a small ε . He then used the Taylor series

$$\frac{1}{1 + \varepsilon} = 1 - \varepsilon + \varepsilon^2 - \varepsilon^3 + \dots,$$

to approximate p/q to high precision. In modern terms, he found a reduction from DIVISION to MULTIPLICATION.

Another early figure was Leonardo Torres y Quevedo, a Spanish engineer. In the 1890s he designed a solver for algebraic equations, and he was also famous for building funiculars, airships, remote-controlled boards, and a machine that played a perfect Chess endgame of a king and rook against a human king. He knew of Babbage’s work, and in his *Ensayos sobre automática* he proposed building an Analytical Engine with floating-point arithmetic using electromechanical components. In 1920, he presented an “Electromechanical Arithmometer” in Paris, which allowed a user to input arithmetic problems through a keyboard, and computed and printed the results. In a *Scientific American* article about Torres’ creations, he had this to say about artificial intelligence:

There is no claim that [the chess player] will think or accomplish things where thought is necessary, but its inventor claims that the limits within which thought is really necessary need to be better defined, and that an automaton can do many things that are popularly classed with thought.

See Randell [668] for more discussion of this history, along with Vannevar Bush’s early work on analog and digital computers.

7.3 The Entscheidungsproblem. Hilbert’s Entscheidungsproblem was originally published in a paper with Ackermann [393]. It’s worth noting that Hilbert asked for a procedure which would determine whether a statement is true or false, as opposed to asking for a proof or disproof of it. The distinction between truth and provability took some time to emerge, culminating in Gödel’s result that there is an unbridgeable gap between them.

7.4 Sets, barbers, and adjectives. Russell’s paradox shows that so-called *naive set theory*, in which we are allowed to use any property to define the set of objects with that property, can lead to contradictions. He described his paradox in a letter to Gottlob Frege in 1902, just as Frege’s book—which used naive set theory to lay the foundations of arithmetic—was going to print. Frege hastily added an Appendix to his book, in which he wrote

Hardly anything more unfortunate can befall a scientific writer than to have one of the foundations of his edifice shaken after the work is finished. This was the position I was placed in by a letter of Mr. Bertrand Russell, just when the printing of this volume was nearing its completion.

While adopting a stratified set-theoretic universe gets rid of paradoxes like Russell's, it has other consequences as well. For instance, we do not speak of the set of all sets, since it would have to contain itself. Instead, we kick it upstairs by calling it a "proper class."

Russell's paradox has many equivalent forms. As he pointed out in [705], R is like the barber who shaves every man who doesn't shave himself, although this version of the paradox goes away if the barber doesn't shave at all. Another form concerns adjectives that describe themselves. For instance, "polysyllabic" is polysyllabic, "recondite" is recondite, "short" is short, and so on. Let's call such words *autological*, and let's call non-self-describing words like "monosyllabic" and "long" *heterological*. Is "heterological" heterological or not?

For another diagonal paradox, consider Berry's paradox. Define an integer x as follows: "the smallest integer that cannot be defined in less than thirteen words." From this we have to conclude that there is a difference between language and metalanguage—that "definability" is not a definable notion.

Similarly, let's say that 0 and 1 are interesting because they are the identities of addition and multiplication, 2 is interesting because it is the only even prime, and so on. At some point we get to the smallest uninteresting number—but surely that makes it interesting!

For a wonderful exploration of Russell's life, and the anguish and passion of his search for foundations, we recommend *Logicomix* by Doxiadis and Papadimitriou [245].

7.5 Hilbert's 10th Problem. When David Hilbert proposed his list of problems in 1900, probably every mathematician alive would have agreed that the 10th problem must have a solution. But after the appearance of undecidable problems in 1936, people began to wonder whether the solvability of Diophantine equations might be undecidable as well. In 1944, Emil Post declared that Hilbert's 10th problem "begs for an unsolvability proof," but it took decades for this proof to appear.

In 1961, Julia Robinson, Martin Davis, and Hilary Putnam [223] showed the undecidability of *exponential* Diophantine equations, i.e., integer equations where exponentiation and polynomials are allowed. Building on earlier work of Robinson [691], their proof worked by encoding the sequence of states of a Turing machine in an enormous integer, so that an integer solution exists if and only if the computation halts.

The challenge was then to show that exponentiation was not required—or rather, that it could be represented with purely polynomial Diophantine equations. In 1970, Yuri Matiyasevič [546] found a system of polynomials that encode the Fibonacci numbers. Using the fact that they grow exponentially (see Problem 2.4) he was able to complete the proof. A history and exposition can be found in [547].

Since then, the proof of undecidability has been simplified to some extent. The simplest proof, that of Jones and Matiyasevič [436, 437], builds integers that encode trajectories of the counter machines of Section 7.6.1.

7.6 Hilbert, Turing, Church: A Hilarious Tour of Computation. The phrase "strange loop," and the names BLOOP and FLOOP for the programming languages corresponding to primitive recursive and partial recursive functions, were introduced by Douglas Hofstadter in his Pulitzer Prize winning opus *Gödel, Escher, Bach: An Eternal Golden Braid* [399]. Both lighter and deeper than any textbook, this gorgeous book discusses self-reference and paradox in music, art and mathematics. In particular, it gives the clearest explanation of Turing's Halting Problem, and Gödel's Incompleteness Theorem, that we have seen. Buy it for your children and read it to them at bedtime.

7.7 The Continuum Hypothesis. Since real numbers, like subsets of the integers, can be represented as infinite binary strings, $2^{\aleph_0}$ is also the cardinality of the reals. Cantor conjectured that $2^{\aleph_0}$ is the smallest cardinal greater than $\aleph_0$, which is denoted $\aleph_1$. Proving or disproving this hypothesis was the first of Hilbert's problems in 1900. However, in 1940 Kurt Gödel showed that it cannot be proved using the standard Zermelo–Fraenkel axioms of set theory, and in 1963 Paul Cohen showed that it cannot be *disproved* either. Thus it is formally independent of these axioms.

A variety of other axioms have been put forward that would imply the Continuum Hypothesis or its negation. But until these axioms gain wide acceptance, its truth or falsity is a doctrinal question in the theology of infinite sets, on which reasonable mathematicians can disagree.

7.8 The Halting Problem and undecidability. The original Halting Problem was posed by Turing [782] for Turing machines, and by Church [173] for the existence of a normal form in the λ -calculus. A modern presentation, similar to the “software-level” one we give here—including the role played by an interpreter and the fact that BLOOP (by another name) cannot interpret itself—was given in 1972 by Hoare and Allison [395].

The fact that all nontrivial semantic properties of programs are undecidable, as stated in Problem 7.4, was proved by Rice [687]. Here “semantic” means a property of the function that a program computes, and “nontrivial” means that it is neither true for all computable partial functions, nor false for all of them.

As Problem 7.5 shows, some “dynamical” questions about programs are also undecidable, such as whether the only way that Π fails to halt is by falling into an endless loop. Other examples include whether Π contains “dead code,” i.e., lines that are never executed—or whether there is a shorter program that computes the same function that Π does.

7.9 Gödel's Second Incompleteness Theorem. The fact that no sufficiently powerful formal system is both consistent and complete—that is, that there are statements S such that neither S nor $\bar{S}$ is a theorem, or (even worse) they both are—is Gödel's First Incompleteness Theorem. His Second Incompleteness Theorem is, in some ways, even more severe: no formal system can prove its own consistency. In other words, it is impossible to prove, within the axioms of the system, that there is no statement S such that both S and $\bar{S}$ are theorems. Thus no formal system strong enough to do the mathematics we care about can prove, about itself, that it is free of paradoxes.

According to the rules of propositional logic, if one can prove a contradiction then one can prove anything. In other words, for any statements S and T , we have $(S \wedge \bar{S}) \Rightarrow T$. It follows that all a system needs to do in order to prove its own consistency is prove that some T is not a theorem. Since the set of theorems is undecidable (see Exercise 7.8), and since finite computations correspond to finite proofs, this rules out the possibility that the system can prove, for all nontheorems T , that T is a nontheorem. However, the Second Incompleteness Theorem implies the stronger statement that the system cannot do this for *even one* nontheorem.

7.10 Formal independence. If it is true, but unprovable, that Π never halts, then neither $\text{Halt s}(\Pi, x)$ nor $\overline{\text{Halt s}(\Pi, x)}$ is a theorem. In other words, the question of whether or not $\Pi(x)$ halts cannot be resolved by the axioms either way.

As a consequence, we have the freedom to add *either one* of these statements to our list of axioms, without causing an inconsistency. If we choose to believe that Π halts, even though it doesn't, we have the following strange situation. Since we can run Π for any finite amount of time, each of the statements

$$\begin{array}{c} \overline{\text{Halt sInTime}(\Pi, x, 1)} \\ \overline{\text{Halt sInTime}(\Pi, x, 2)} \\ \overline{\text{Halt sInTime}(\Pi, x, 3)} \\ \vdots \end{array}$$

is a theorem—and yet so is

$$\exists t : \text{Halt sInTime}(\Pi, x, t).$$

Such a system is called ω -inconsistent. It believes that there exists an integer t with a certain property, even though it knows that each particular t fails to qualify. Paraphrasing Scott Aaronson [1], if we ask such a system to tell us what t is, it cheerfully replies “why, the t such that $\Pi(x)$ halts after t steps, of course!”

7.11 Recursion and primitive recursion. The idea of defining functions recursively goes back to Dedekind and Peano in the late 1880s. In an effort to build an axiomatic foundation for number theory, they described how to define addition, multiplication, and so on inductively using the successor function. The phrase *primitive recursion* was coined by Rózsa Péter in 1935, who proved many important results about the class of primitive recursive functions. She also defined a hierarchy of more elaborate types of recursion, roughly equivalent to the hierarchy described in Problem 7.19. The equivalence between primitive recursive functions and programs with bounded loops—that is, programs written in BLOOP—was demonstrated by Meyer and Ritchie [561].

In the 1930s, Herbrand and Gödel defined recursive functions more generally, as solutions to systems of equations: for each x , the value $h(x)$ is defined if there is a unique y such that the statement $h(x) = y$ can be derived from the system. This derivation consists of a finite number of steps, where each step substitutes the right-hand side of one of the equations wherever its left-hand side appears. At the time, however, Gödel was not convinced that this concept of recursion “comprises all possible recursions.” In 1936, Kleene [480] showed that that this definition is equivalent to what we now call partial recursive functions, where we use the μ operator to solve equations of the form $f(\mathbf{x}, y) = 0$, and Exercise 7.12 is called Kleene’s Normal Form Theorem. Some of this history appears in Kleene [481].

7.12 Exploding functions. The Ackermann function, and a proof that it is not primitive recursive, was published by Hilbert’s student Wilhelm Ackermann in 1928 [20]. A similar function was published concurrently in a little-known paper by another of Hilbert’s students, Gabriel Sudan [765]; see [146].

Other recursive functions, with simpler proofs that they are not primitive recursive, were given by Rózsa Péter [641] and by Robinson [692]. The proof we give in Problem 7.17 that the Ackermann function majorizes every primitive recursive function essentially follows [692]. The kind of iterative implementation of Ackermann’s function which we give in Section 7.3, using a **while** loop and a stack, appears not to have been published before 1988 [349].

A popular way to write the sequence of binary operations starting with addition, multiplication, and exponentiation comes from Knuth. If we define $x \uparrow^1 y = x + y$ and

$$x \uparrow^{n+1} y = x \uparrow^n (\underbrace{x \uparrow^n (\cdots x)}_{n \text{ copies of } x}),$$

then $A(n, x, y) = x \uparrow^n y$. The operation $x \uparrow^4 y = \underbrace{x^{x^{\cdots x}}}_{y \text{ times}}$ is sometimes called “tetration.” This notation allows us to express very large numbers very compactly.

7.13 Quine and “quines.” The philosopher and logician Willard Van Orman Quine made numerous contributions to epistemology, set theory, and the philosophy of science. He also studied indirect versions of self-reference. Consider the following paradoxical sentence:

“Yields a falsehood when preceded by its quotation” yields a falsehood when preceded by its quotation.

This sentence manages to assert that it is false, without using the words “this sentence.” The first half acts as data and the second half acts as a function on this data, and together they expand to create another copy of the original sentence. In *Gödel, Escher, Bach*, Douglas Hofstadter draws numerous analogies between this kind of self-reference and self-reproduction. For instance, our genome (the first half) is the data that our ribosomes (the second half) read, translate into proteins, and unfold into another copy of ourselves.

In Quine’s honor, a computer program that “self-reproduces,” in the sense that it prints out a copy of its own source code, is called a *quine*. You can find quines in virtually every programming language ever invented—some readable, and some deliberately obfuscated—at Gary Thompson’s Quine Page, www.nyx.net/~gthompso/quine.htm.

7.14 The λ -calculus and combinatory logic. The notion of a combinator, and the realization that they allow functions to act directly on each other, was originally given by Moses Ilyich Schönfinkel in 1924 [717]. It was reinvented and extended by Haskell Curry [208], after whom the functional programming language HASKELL is named.

Schönfinkel and Curry showed that we can generate any combinator whatsoever from these two,

$$\mathbf{S} = \lambda x y z . (xz)(yz) \text{ and } \mathbf{K} = \lambda x y . x .$$

(Note that $\mathbf{K}$ was called $\mathbf{T}$ in Section 7.4.) For instance, the identity is $\lambda x . x = \mathbf{SKK}$, and the composition combinator **comp** from (7.18) is

$$\lambda f g x . f(gx) = \mathbf{S(KS)K}.$$

Thus $\mathbf{S}$ and $\mathbf{K}$ form one of the most compact systems capable of universal computation. If you think two combinators is still too many, you can generate them both from this one:

$$\mathbf{X} = \lambda x . x\mathbf{KSK},$$

since

$$\mathbf{K} = (\mathbf{XX})\mathbf{X} \text{ and } \mathbf{S} = \mathbf{X}(\mathbf{XX}).$$

In addition to the basic substitution rule, which is called β -reduction in the λ -calculus, Schönfinkel used the *axiom of extensionality* which states that two combinators are equivalent if they always carry out the same function on their inputs. However, this axiom, which is called η -reduction, isn't really necessary.

Alonzo Church proposed the λ -calculus in 1932 [171, 172] as a foundation for logic. His students Kleene and Rosser modified its original definition, and played crucial roles in its development. Church and Rosser [174] showed that, if a normal form exists, it is unique and any chain of reductions will lead to it. Many of the combinators for arithmetic on Church numerals are due to Rosser; those for predecessor and subtraction are due to Kleene, as is the proof that all partial recursive functions are λ -definable [479].

In the same year as Turing's work, Church [173] showed that the problem of whether a λ -expression has a normal form—or equivalently, whether a partial recursive function is defined for a given input—is undecidable. The fixed point combinator $\mathbf{Y}$ was discovered by Curry, and there are an infinite number of others.

The standard reference for the λ -calculus is Barendregt [79]; for a lighter introduction, see the lecture notes by Barendregt and Barendsen [80]. Smullyan gives a lovely exposition of combinatory logic in terms of birds in a forest responding to each others' calls in *To Mock a Mockingbird* [748].

The programming language LISP was invented by John McCarthy in 1958 [550], who later received the Turing Award. Inspired by the λ -calculus, it treats programs and data as the same kind of object, and lets the user define functions from a handful of simple primitives. Along with its descendants, such as HASKELL, SCHEME, and ML, LISP is considered a *functional* programming language because it computes by unfolding the definition of a function until it reaches a normal form. In contrast, *imperative* programming languages such as FORTRAN, C, and JAVA work primarily by ordering the computer to change the values of its variables, like a Turing machine updating its tape. Because of the grand unification of 1936, we know that these two types of programming language are equally powerful.

7.15 Small universal Turing machines. Turing's universal machine had a fairly large number of states and symbols. But since then, clever people have found much smaller ones—for instance, with just 6 states and 4 symbols. As Figure 7.22 shows, there is a tradeoff between states and symbols. Their product is essentially the size of the table of the transition function, and this product is roughly constant along the universal curve.

The smallest universal machines on record are those of Rogozhin [699], Kudlek and Rogozhin [499], and Neary and Woods [612]. Rather than simulating other Turing machines directly, these machines simulate tag systems like those described in Section 7.6.3. In some cases, a small amount of postprocessing is needed to extract the output from the tape.

If we modify our definitions a little, allowing the initial tape to be filled with periodic patterns to the left and right of the input word instead of with blanks, we can make these machines even smaller—with 3 states and 3 symbols, 2 states and 4 symbols, or 6 states and 2 symbols. We used the last of these for our iterated map in Section 7.6.6. These so-called *weakly* universal machines are also due to Neary and Woods [613], and they work by simulating cellular

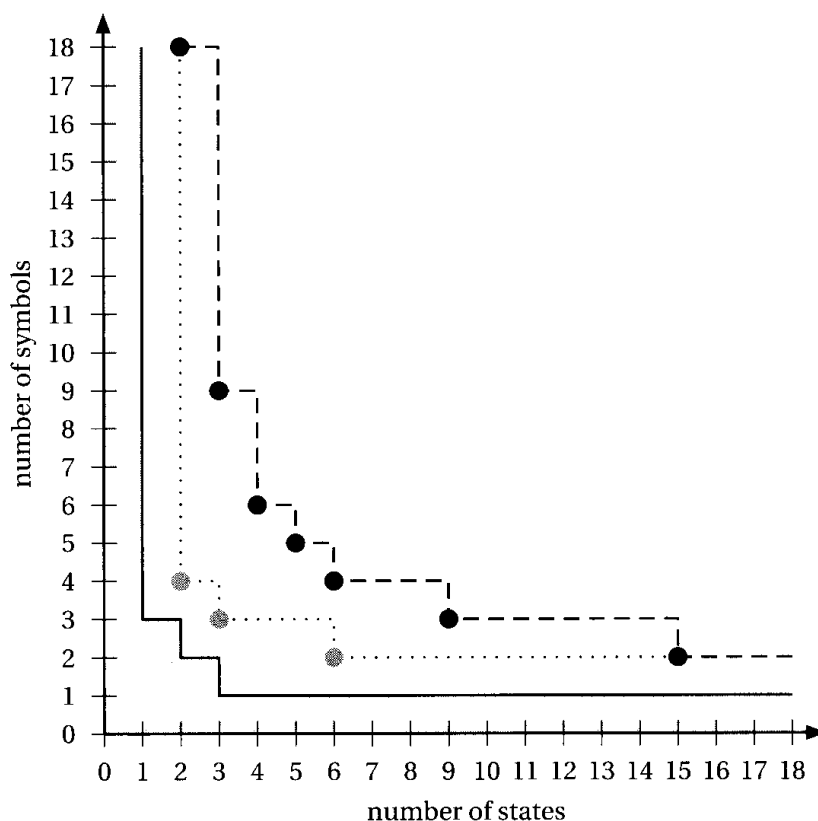


FIGURE 7.22: The smallest known universal Turing machines are indicated by black dots. Thus, everywhere at or above the dashed line, there is at least one universal Turing machine with that number of states and symbols. Weakly universal machines, where the tape is filled with a periodic pattern to the left and right of a finite initial word, rather than blanks, are indicated by gray dots. No universal machines exist at or below the solid line, because their Halting Problems are decidable.

automaton rule 110. They never halt, but the question of whether or not they will ever produce a certain string on their tape is undecidable.

7.16 Alan Turing and his machine. Our quotes are from Turing's 1936 paper "On computable numbers, with an application to the *Entscheidungsproblem*" [782]. It is reprinted, along with the other seminal papers of 1936 by Church, Kleene, and Post, in the collection of Davis [220]. For a guided tour through Turing's paper we recommend Petzold [642]. We also recommend Gandy [311], who describes the various strands of thought that came together in 1936. Gödel's quote is from his remarks to the Princeton Bicentennial Conference on Problems in Mathematics [220], and von Neumann's quote is from [800].

During World War II, Turing played a key role in the British effort to break the Nazi's Enigma code—a success which was, arguably, as important to the outcome of the war as the development of radar or the atomic bomb. He and his collaborators designed Colossus, the first large-scale electronic computer, slightly predating John von Neumann's work on ENIAC in the United States.

Turing also wrote presciently on the subject of artificial intelligence. In [783] he proposed the so-called Turing Test, in which a computer tries to fool an interviewer into believing that it is human. He began by describing a game in which a man tries to convince an interviewer that he is a woman. He argued that computers would someday pass this test, that "at the end of the century the use of words and general educated opinion will have altered so much

that one will be able to speak of machines thinking without expecting to be contradicted,” and that “machines will eventually compete with men in all purely intellectual fields.”

Turing believed that we would create such machines, not by writing a fixed program, but by setting a process of learning or evolution in motion. He described a simulated neural network, trained by giving it positive and negative feedback. He wrote, with a poignant tongue in his cheek,

It will not be possible to apply exactly the same teaching process to the machine as to a normal child. It will not, for instance, be provided with legs, so that it could not be asked to go out and fill the coal scuttle... however well these deficiencies might be overcome by clever engineering, one could not send the creature to school without the other children making excessive fun of it.

In a little-known 1948 report [785], Turing laid out several kinds of search, including “genetical or evolutionary search,” presaging the field of genetic algorithms. He also did foundational work on pattern formation, proposing and simulating nonlinear reaction-diffusion models for biological systems such as embryogenesis and phyllotaxis [784].

Alan Turing had the misfortune of being born into a time, and a society, in which homosexual relationships were illegal. After being convicted of “gross indecency,” he was stripped of his security clearance, thus ending his work on cryptography, and sentenced to injections of estrogen. Two years later, he committed suicide. Far too late, in 2009, British Prime Minister Gordon Brown apologized for the shameful treatment that Turing received. If it had not been for this tragedy, Turing might have played as great a role in the development of computer science as he did at its birth.

7.17 Busy beavers. The busy beaver function of Exercise 7.21 was invented by Tibor Rado [664]. He defined two functions, and proved that both are uncomputable: the maximum number of moves $S(n, m)$ a machine makes before halting, and the maximum number $\Sigma(n, m)$ of 1s it can write on the tape before halting. Even for small machines, these numbers are astronomical. For $n = 6$ states and $m = 2$ symbols, for instance, Pavel Kropitz has found a Turing machine showing that $S(n, m) \geq 7.4 \times 10^{36534}$.

7.18 The Church–Turing Thesis. Church and Turing were concerned with capturing the class of functions that are “effectively computable”—those for which there is an algorithm, each step of which could be carried out by a human with pencil and paper. Their thesis states that this class is synonymous with the Turing-computable functions. This is equivalent to von Neumann’s statement that a Turing machine can simulate any automaton, as long as each step of the automaton is something that a human could do.

The *physical Church–Turing Thesis* is a very different statement—namely, that any function which can be computed by a physical device can also be computed by a Turing machine, and more generally, that any physical system can be simulated by a Turing machine to any desired accuracy. This could be false, for instance, if the laws of physics allow for exotic processes that perform an infinite amount of computation in a finite time. We briefly address this thesis in Chapter 15 as part of our discussion of quantum computation.

The Church–Turing Thesis is often misunderstood as the claim that digital computers can simulate human intelligence. Of course, if the physical Church–Turing Thesis is true, the brain can be simulated by a computer since it is a finite physical system. The question of whether this simulation would be conscious in the same way that we are opens up an additional can of worms, such as whether a computer can “really” enjoy strawberries and cream, or “just simulate” that experience—and whether this distinction makes any sense. Turing deliberately sidestepped this question in [783] by defining the Turing test only in terms of a computer’s outward behavior, not its inner life.

In any case, while it is certainly connected to these questions, the original thesis does not address intelligence in general. It focuses on the set of decidable problems, those for which there is an algorithm—in Turing’s words, those “which can be solved by human clerical labour, working to fixed rules, and without understanding.”

7.19 Counter machines, FRACTRAN, and the Collatz Problem. Our definition of additive and multiplicative counter machines, and the fact that two-counter machines are universal, follows Minsky’s book [570]. FRACTRAN was introduced and shown to be computationally universal by John Horton Conway [195]; see Guy [352] and Havil [375] for

additional examples and discussion. Conway's proof that FRACTRAN is universal builds on his earlier proof [194] of universality for generalized Collatz problems.

The Collatz problem is named after its inventor Lothar Collatz. Among small numbers, one of the initial values of x that takes the longest to reach 1 is 27. It takes 111 steps, and reaches 9232 before it comes back down. One heuristic argument for the conjecture, which we take from Jeffrey Lagarias [505], is to consider the number of times we divide by 2 in between odd values of x . Clearly $3x + 1$ is even. If we assume that it is divisible by 4 with probability $1/2$, by 8 with probability $1/4$, and so on, just as if it were a random even integer, then we will divide by 2 once with probability $1/2$, by 4 with probability $1/4$, and so on, before multiplying by 3 the next time. The geometric mean of the ratio between the new value and the old one is then

$$\left(\frac{3}{2}\right)^{1/2} \left(\frac{3}{4}\right)^{1/4} \left(\frac{3}{8}\right)^{1/8} \cdots = \frac{3}{4} < 1.$$

This suggests that, “on average,” $g(x) < x$, and therefore that the trajectory of x cannot diverge. However, like many problems in mathematics, the question is whether the values of x can really be treated as random in this way.

The two-, three-, and four-dimensional finite automata of Problem 7.30 are explored in [451, 522].

7.20 Tag systems and Post's anticipation. Even though it took place in 1921, Emil Post's work on normal form and tag systems did not appear in print until 1943 [648]. He struggled with a particular example, where $\nu = 3$, $h(0) = 00$, and $h(1) = 1101$. Its general behavior is still not understood—there is neither an algorithm for its halting problem nor a proof that it is undecidable.

Minsky [569] proved that tag systems are universal using the counter simulation of Turing machines with tape alphabet $\{0, 1\}$, and Cocke and Minsky [184] showed this even in the case $\nu = 2$. The tag system which computes the $3x + 1$ function and the proof of universality using Collatz functions in Problem 7.34 are from Liesbeth De Mol [580], who explains the role that tag systems played in Post's thinking in [579].

After realizing that even tag systems possessed extremely complicated behavior, Post reversed his thinking, and used the diagonal method (Problem 7.11) to show that systems in normal form cannot prove all true propositions about themselves. Thus he proved—in essence—Gödel's and Turing's results a decade before they did. See Stillwell [760] for a discussion of Post's approach, which was arguably simpler than Gödel's.

In 1936, Post invented a machine essentially identical to the Turing machine [647], although he did not construct a universal machine or prove that the Halting Problem is undecidable. Later, he invented the notion of recursive enumerability [649]. His correspondence problem appeared in [650], and the undecidability proof we give here is from Minsky [570]. CORRESPONDENCE has been used to show that many other problems, ranging from automata theory to abstract algebra, are undecidable as well—including the problem of whether two four-dimensional manifolds are homeomorphic.

Owing to his anticipation of Gödel's and Turing's more famous works, Post is something of an unsung hero in logic and computer science. We agree with his sentiments when he says

...it seemed to us to be inevitable that these developments will result in a reversal of the entire axiomatic trend of the late 19th and early 20th centuries, with a return to meaning and truth.

7.21 The Game of Life. Like FRACTRAN, Life was invented (or should we say discovered?) by John Horton Conway. It first appeared in 1970, in Martin Gardner's “Mathematical Games” column in *Scientific American*. The first simulations were carried out on graph paper and Go boards, but it quickly became a favorite topic of the emerging Hacker community. That year, Bill Gosper won a \$50 prize from Conway by finding the glider gun in Figure 7.12, thus proving that the number of live sites can grow without bound.

The Turing machine shown in Figure 7.14 was constructed by Paul Rendell, who described it in [21]. Sliding block memories, which simulate counters by moving a 2×2 block, were invented by Dean Hickerson. A universal counter machine, consisting of a finite state control coupled to two or more of these blocks, was developed by Paul Chapman.

7.22 One-dimensional cellular automata. The universality of rule 110 was conjectured by Stephen Wolfram and proved by Matthew Cook [196]. Like Minsky's counter machines, Cook's original construction was exponentially inefficient. However, Neary and Woods [611] showed that the relevant tag systems, and therefore rule 110, can simulate a Turing machine with polynomial slowdown.

Wolfram [816] has conjectured that every cellular automaton is either easy to predict and analyze, or is computationally universal. Another candidate for universality among the "elementary" one-dimensional rules, with two states per site and nearest-neighbor interactions, is rule 54. See Hanson and Crutchfield [360] for a study of its gliders and their interactions.

7.23 Tilings. Wang tiles are named after Hao Wang, who showed in 1961 that TILING COMPLETION is undecidable [802]. He conjectured that any set of tiles can tile the plane periodically or not at all, and pointed out that in that case TILING would be decidable. In 1966, his student Robert Berger found a (very large) aperiodic set of tiles, and proved that TILING is in fact undecidable [105]. The much smaller set of tiles we show in Figure 7.18 was designed by Raphael Robinson [693].

Interestingly, the problem of whether a given set of Wang tiles can tile the plane *periodically* is also undecidable, since we can design the tiles so that a periodic tiling exists if and only if a given Turing machine halts. See Gurevich and Koriakov [351].

7.24 Iterated maps. The fact that simple piecewise-continuous maps of the plane can simulate Turing machines was shown by Moore [588, 589]. He showed that a number of questions about their behavior are undecidable, including whether a given point lies on a periodic orbit or whether or not its trajectory is chaotic. He also showed that these two-dimensional maps can be made infinitely differentiable, and that they can be embedded in continuous flows in three dimensions.

Similar results were obtained independently by Reif, Tygar, and Yoshida [678] for optical ray-tracing, Siegelmann and Sontag [738] for neural networks with piecewise linear activation functions, and Cosnard, Garzon, and Koiran [204] for maps of the plane.

The universal 6-state, 2-symbol Turing machine we simulate in Figure 7.21 is due to Neary and Woods; see Note 7.15.